

Jupyter Notebook Execution Report

Name: Rodrigo Tapiador Cano

Date: April 04, 2026

Cell 1: ■ Markdown

Forecasting de Demanda Eléctrica en España

Análisis de Series Temporales con Machine Learning y Variables Meteorológicas

Descripción del Proyecto

Este proyecto tiene como objetivo **predecir la demanda eléctrica en España** utilizando modelos estadísticos (SARIMA) y modelos de Machine Learning, incorporando **variables meteorológicas y energéticas** como factores explicativos clave.

Dataset: [Energy Consumption, Generation, Prices and Weather - Spain (Kaggle)](<https://www.kaggle.com/datasets/nicholasjhana/energy-consumption-generation-prices-and-weather>)

Contexto:

Según un paper de 2019, el forecasting en mercados energéticos es una de las áreas de mayor impacto del Machine/Deep Learning para la transición hacia una infraestructura eléctrica basada en renovables.

Contenido del dataset:

- **4 años de datos horarios** (2015-2018)
- Consumo eléctrico real y forecast del TSO
- **Generación por fuente:** solar, eólica, gas, carbón, nuclear, hidráulica
- **Precios spot** y day-ahead
- **Datos meteorológicos** de las 5 ciudades más grandes de España:
 - Madrid, Barcelona, Valencia, Sevilla, Bilbao
- Variables: temperatura, presión, humedad, velocidad del viento

Fuentes:

- ENTSOE (European Network of Transmission System Operators)
- Red Eléctrica de España (REE)

- Open Weather API

Objetivos Principales

1. **Limpieza y preparación** de datos con tratamiento correcto de valores nulos
2. **Análisis meteorológico**: Demostrar la relación entre clima y demanda eléctrica
3. **Feature Engineering avanzado**: Variables temporales, lags, rolling stats, y **weather features**
4. **Modelado estadístico**: SARIMA con análisis de tendencia, estacionalidad y residuo
5. **Machine Learning**: Random Forest y Gradient Boosting
6. **Análisis técnico**: Medias móviles, Golden Cross y volatilidad
7. **Comparación de modelos** y análisis de feature importance

Hipótesis clave:

> Las **variables meteorológicas** (especialmente temperatura) son factores explicativos fundamentales para la demanda eléctrica, junto con la información temporal (lags y estacionalidad).

Equipo de Trabajo y Responsabilidades

Persona 1: Preparación de Datos y EDA

Tareas:

- Carga y exploración inicial del dataset
- Conversión a series temporales (índice datetime)
- Análisis de valores nulos y estrategia de imputación
- Análisis exploratorio de demanda eléctrica
- Análisis meteorológico: correlación clima-demanda
- Visualizaciones de patrones diarios/semanales/estacionales

Entregables:

- Dataset limpio y correctamente indexado
- Informe de calidad de datos
- Gráficos exploratorios de demanda y temperatura

Persona 2: Análisis Temporal y Comparación de Modelos

Tareas:

- **Lags**: 1h, 24h, 168h (capturar autocorrelación)
- **Descomposición temporal**: tendencia, estacionalidad (period=24), residuo
- **Test de estacionariedad** (ADF)
- **ACF**
- **Modelo SARIMA**: determinación de órdenes (p,d,q)(P,D,Q,s)
- **Media móvil y Golden Cross** para precios eléctricos
- **Análisis de volatilidad**

Entregables:

- Modelo SARIMA con parámetros justificados
- Análisis de cruces de medias móviles en precios
- Demostración de que temperatura y lags son cruciales

Persona 3: Machine Learning

Tareas:

- **Modelos ML**: Random Forest y Gradient Boosting
- **Feature importance**: identificar variables más predictivas
- **Comparación final de modelos**: SARIMA vs RF vs GB
- Visualizaciones de predicciones vs valores reales

Entregables:

- Modelos ML con hiperparámetros optimizados
- Análisis de feature importance
- Tabla comparativa de métricas (RMSE, MAE, MAPE)
- Conclusiones sobre mejor modelo y variables importantes

Preguntas de Investigación

1. **¿Podemos predecir mejor que el TSO?**
2. **¿Cuál es la relación entre temperatura y demanda?**
3. **¿Qué features son más importantes?**: lags, temperatura, hora del día, generación?
4. **¿Qué modelo funciona mejor?**

Cell 2: ■ Markdown

1. Importación de Librerías

Importamos todas las librerías necesarias para el análisis completo.

Cell 3: ■ Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Cell 4: ■ Markdown

2. Carga y Exploración Inicial del Dataset

Cell 5: ■ Code

```
# 1. Cargar dataset de energía
df_energy = pd.read_csv('./data/energy_dataset.csv')
print(f"Energy dataset: {df_energy.shape[0]:,} filas × {df_energy.shape[1]} columnas")

# 2. Cargar dataset de clima
df_weather = pd.read_csv('./data/weather_features.csv')
print(f"Weather dataset: {df_weather.shape[0]:,} filas × {df_weather.shape[1]} columnas")

print("\n" + "=" * 80)
print("ESTRUCTURA DE LOS DATASETS:")
print("=" * 80)

print("\nEnergy Dataset:")
print(df_energy.info())
```



```
print("\nWeather Dataset:")
print(df_weather.info())
```

Output:

Energy dataset: 35,064 filas × 29 columnas

Weather dataset: 178,396 filas × 17 columnas

=====

ESTRUCTURA DE LOS DATASETS:

=====

Energy Dataset:

<class 'pandas.DataFrame'>

RangeIndex: 35064 entries, 0 to 35063

Data columns (total 29 columns):

#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	time	35064 non-null	str
1	generation biomass	35045 non-null	float64
2	generation fossil brown coal/lignite	35046 non-null	float64
3	generation fossil coal-derived gas	35046 non-null	float64
4	generation fossil gas	35046 non-null	float64
5	generation fossil hard coal	35046 non-null	float64
6	generation fossil oil	35045 non-null	float64
7	generation fossil oil shale	35046 non-null	float64
8	generation fossil peat	35046 non-null	float64
9	generation geothermal	35046 non-null	float64
10	generation hydro pumped storage aggregated	0 non-null	float64
11	generation hydro pumped storage consumption	35045 non-null	float64
12	generation hydro run-of-river and poundage	35045 non-null	float64
13	generation hydro water reservoir	35046 non-null	float64
14	generation marine	35045 non-null	float64
15	generation nuclear	35047 non-null	float64
16	generation other	35046 non-null	float64
17	generation other renewable	35046 non-null	float64
18	generation solar	35046 non-null	float64

```

19  generation waste          35045 non-null  float64
20  generation wind offshore  35046 non-null  float64
21  generation wind onshore   35046 non-null  float64
22  forecast solar day ahead   35064 non-null  float64
23  forecast wind offshore eday ahead  0 non-null      float64
24  forecast wind onshore day ahead  35064 non-null  float64
25  total load forecast        35064 non-null  float64
26  total load actual          35028 non-null  float64
27  price day ahead            35064 non-null  float64
28  price actual               35064 non-null  float64

```

dtypes: float64(28), str(1)

memory usage: 7.8 MB

None

Weather Dataset:

```
<class 'pandas.DataFrame'>
```

RangeIndex: 178396 entries, 0 to 178395

Data columns (total 17 columns):

#	Column	Non-Null Count	Dtype
0	dt_iso	178396 non-null	str
1	city_name	178396 non-null	str
2	temp	178396 non-null	float64
3	temp_min	178396 non-null	float64
4	temp_max	178396 non-null	float64
5	pressure	178396 non-null	int64
6	humidity	178396 non-null	int64
7	wind_speed	178396 non-null	int64
8	wind_deg	178396 non-null	int64
9	rain_1h	178396 non-null	float64
10	rain_3h	178396 non-null	float64
11	snow_3h	178396 non-null	float64
12	clouds_all	178396 non-null	int64
13	weather_id	178396 non-null	int64
14	weather_main	178396 non-null	str

```
15 weather_description 178396 non-null str
16 weather_icon         178396 non-null str
dtypes: float64(6), int64(6), str(5)
memory usage: 23.1 MB
None
```

Cell 6: ■ Markdown

Los datos cubren cuatro años horarios y combinan carga, generación y clima, suficiente base para comparar modelos estadísticos.

Cell 7: ■ Markdown

2b. Separación de Columnas de Forecast (Comparación con TSO)

Objetivo Principal

El propósito de este notebook es **desarrollar nuestro propio pronóstico de demanda eléctrica** utilizando modelos estadísticos (SARIMA) y Machine Learning. Las columnas de forecast del TSO las utilizaremos como **referencia de comparación** para comparar nuestra previsión con la oficial de red eléctrica.

Possible Data Leakage

Las columnas de **forecast del TSO** (pronóstico oficial del operador del sistema) representan una **potencial fuente de data leakage**:

- Si usamos estas columnas para entrenar nuestro modelo, estaríamos utilizando información que **no** estaría disponible en un escenario real de predicción en tiempo real.
- Esto podría inflar artificialmente la precisión de nuestro modelo.

Estrategia: Separar para Comparación Limpia

Acción: Extraer columnas de forecast a dataset separado **ANTES** del merge, para:

1. ✓ Entrenar nuestros modelos **sin estas columnas** (pronóstico limpio)
2. ✓ **Comparar nuestras predicciones** con el pronóstico oficial del TSO
3. ✓ Responder: **¿Predecimos mejor o peor que el TSO?**

Cell 8: ■ Code

```
# Identificar y separar columnas de forecast (TSO) antes del merge
```

```

forecast_cols = [col for col in df_energy.columns if 'forecast' in col.lower() or
'ahead' in col.lower()]

print(f"Columnas de forecast encontradas: {len(forecast_cols)}")
print("Columnas identificadas:")
for col in forecast_cols:
    print(f" - {col}")

# 1. Crear dataset separado con forecast (para comparación posterior)
df_forecast_tso = df_energy[['time'] + forecast_cols].copy()

print(f"\nDataset de forecast creado: {df_forecast_tso.shape}")
print(f"Rango: {df_forecast_tso['time'].min()} → {df_forecast_tso['time'].max()}")

# 2. Eliminar columnas de forecast del dataset principal (evitar data leakage)
df_energy_clean = df_energy.drop(columns=forecast_cols).copy()

print(f"\nDataset de energía limpio inicial: {df_energy_clean.shape}")

```

Output:

```

Columnas de forecast encontradas: 5
Columnas identificadas:
  - forecast solar day ahead
  - forecast wind offshore eday ahead
  - forecast wind onshore day ahead
  - total load forecast
  - price day ahead

Dataset de forecast creado: (35064, 6)
Rango: 2015-01-01 00:00:00+01:00 → 2018-12-31 23:00:00+01:00

Dataset de energía limpio inicial: (35064, 24)

```

Cell 9: ■ Markdown

Separar desde el inicio el forecast oficial del TSO evita leakage y deja preparado un benchmark externo limpio para evaluar después si nuestros modelos realmente aportan mejora.

Cell 10: ■ Code

```

# Convertir 'time' a datetime en ambos datasets originales limpios

```

```
from prophet.make_holidays import make_holidays_df
```

```
df_energy_clean['time'] = pd.to_datetime(df_energy_clean['time'], utc=True)
df_weather_clean = df_weather.copy()
df_weather_clean['time'] = pd.to_datetime(df_weather_clean['dt_iso'], utc=True)
df_weather_clean = df_weather_clean.drop(columns='dt_iso')
```

```
# Limpiar nombres de ciudades (Barcelona tiene un espacio inicial en el dataset original)
```

```
df_weather_clean['city_name'] = df_weather_clean['city_name'].str.strip()
```

```
def build_holiday_exog(index_or_series, country='ES', years_ahead=1,
                        timezone='Europe/Madrid'):
```

```
    dt_index = pd.DatetimeIndex(index_or_series)
```

```
    if dt_index.tz is None:
```

```
        local_index = dt_index.tz_localize(timezone)
```

```
    else:
```

```
        local_index = dt_index.tz_convert(timezone)
```

```
    local_dates = pd.DatetimeIndex(local_index.normalize().tz_localize(None))
```

```
    year_list = list(range(local_index.min().year, local_index.max().year + years_ahead
                            + 1))
```

```
    holidays_df = make_holidays_df(year_list=year_list, country=country)
```

```
    holiday_dates =
```

```
    pd.DatetimeIndex(pd.to_datetime(holidays_df['ds']).dt.normalize().unique())
```

```
    weekend_mask = pd.Series((local_index.dayofweek >= 5).astype(int),
                             index=dt_index)
```

```
    df_holidays = pd.DataFrame(index=dt_index)
```

```
    df_holidays.index.name = 'time'
```

```
    df_holidays['is_holiday'] = pd.Series(local_dates,
                                           index=dt_index).isin(holiday_dates).astype(int)
```

```
    df_holidays['is_pre_holiday'] = pd.Series(local_dates + pd.Timedelta(days=1),
                                               index=dt_index).isin(holiday_dates).astype(int)
```

```
    df_holidays['is_post_holiday'] = pd.Series(local_dates - pd.Timedelta(days=1),
                                                index=dt_index).isin(holiday_dates).astype(int)
```

```
    df_holidays['is_weekend_or_holiday'] = ((weekend_mask == 1) |
                                             (df_holidays['is_holiday'] == 1)).astype(int)
```

```
    return df_holidays.sort_index()
```

```
# DataFrame exogeno independiente para SARIMA; no se mezcla con los datasets de ML.
df_holidays_es = build_holiday_exog(df_energy_clean['time'], country='ES',
years_ahead=1)

print(f"Dataset energía limpio: {df_energy_clean.shape[0]:,} filas ×
{df_energy_clean.shape[1]} columnas")

print(f"Rango energía: {df_energy_clean['time'].min()} →
{df_energy_clean['time'].max()}")

print(f"\nDataset weather limpio: {df_weather_clean.shape[0]:,} filas ×
{df_weather_clean.shape[1]} columnas")

print(f"Rango weather: {df_weather_clean['time'].min()} →
{df_weather_clean['time'].max()}")

print(f"Memoria weather: {df_weather_clean.memory_usage(deep=True).sum() /
1024**2:.2f} MB")

print(f"\nExog holidays ES: {df_holidays_es.shape[0]:,} filas ×
{df_holidays_es.shape[1]} columnas")

print(f"Rango holidays: {df_holidays_es.index.min()} →
{df_holidays_es.index.max()}")

print(f"Columnas holidays: {list(df_holidays_es.columns)}")
```

Output:

```
Dataset energía limpio: 35,064 filas × 24 columnas
Rango energía: 2014-12-31 23:00:00+00:00 → 2018-12-31 22:00:00+00:00

Dataset weather limpio: 178,396 filas × 17 columnas
Rango weather: 2014-12-31 23:00:00+00:00 → 2018-12-31 22:00:00+00:00
Memoria weather: 55.69 MB

Exog holidays ES: 35,064 filas × 4 columnas
Rango holidays: 2014-12-31 23:00:00+00:00 → 2018-12-31 22:00:00+00:00
Columnas holidays: ['is_holiday', 'is_pre_holiday', 'is_post_holiday', 'is_weekend_or_holiday']

[STDERR]
Importing plotly failed. Interactive plots will not work.
```

Cell 11: ■ Markdown

Los datasets ya quedan alineados temporalmente y, además, se construye `df\_holidays\_es`, que servirá como calendario exógeno reproducible para SARIMA y Prophet.

Cell 12: ■ Markdown

2c. Eliminación de Columnas Innecesarias

Limpiamos `df_weather_clean`` antes del pivot para evitar arrastrar variables meteorológicas poco útiles al dataset final.

Columnas a Eliminar:

1. `temp_min`` y `temp_max``

- En datos `horarios`` no tiene sentido tener temperatura mínima/máxima

- Estos valores tienen sentido solo en `agregaciones diarias``

- Mejor usar `temp`` (temperatura actual por hora)

2. `weather_id``

- Identificador interno de la API de OpenWeather

- Redundante con las columnas categóricas `weather_main`` / `weather_description``

- No aporta información interpretable

> **Nota:** `weather_icon``, `weather_main`` y `weather_description`` se conservan en `df_weather_clean``, pero no se incluirán en el pivot numérico final.

Cell 13: ■ Code

```
# Eliminamos columnas innecesarias directamente sobre df_weather_clean
patterns_to_drop = ['temp_min', 'temp_max', 'weather_id']

cols_to_drop = [col for col in df_weather_clean.columns if col in patterns_to_drop]

print(f"Dataset weather antes: {df_weather_clean.shape}")
print(f"\nColumnas a eliminar ({len(cols_to_drop)}):")
for col in sorted(cols_to_drop):
    print(f" - {col}")

df_weather_clean = df_weather_clean.drop(columns=cols_to_drop)

print(f"\nDataset weather después: {df_weather_clean.shape}")
print(f"Columnas eliminadas: {len(cols_to_drop)}")
```

Output:

```
Dataset weather antes: (178396, 17)
```

Columnas a eliminar (3):

- temp_max
- temp_min
- weather_id

Dataset weather después: (178396, 14)

Columnas eliminadas: 3

Cell 14: ■ Markdown

Se eliminan variables meteorológicas redundantes o poco interpretables para no arrastrar ruido innecesario al pivot ni al dataset analítico final.

Cell 15: ■ Markdown

2d. Ajustes de Signo en Datos de Energía

El consumo de bombeo hidráulico (`generation hydro pumped storage consumption`) representa energía **consumida** (no generada). Por convención, los valores de consumo deben ser **negativos** para diferenciarlos de la generación. Este ajuste se hace sobre `df\_energy\_clean` antes de unir los datasets.

Cell 16: ■ Code

```
# Ajuste de signo: esta serie representa consumo y debe ser negativa
pumped_storage_col = 'generation hydro pumped storage consumption'
if pumped_storage_col in df_energy_clean.columns:
    df_energy_clean[pumped_storage_col] = -df_energy_clean[pumped_storage_col]
print(f"■ {pumped_storage_col} ajustada a valores negativos")
```

Output:

■ generation hydro pumped storage consumption ajustada a valores negativos

Cell 17: ■ Markdown

El bombeo hidráulico queda representado con el signo correcto como consumo, lo que hace más coherente cualquier agregación energética posterior.

Cell 18: ■ Markdown

3. Conversión a Series Temporales

****Fundamental:**** Convertir la columna `time` a formato datetime y establecerla como índice en `df\_energy\_clean` y `df\_weather\_clean` antes de cualquier imputación o merge.

Cell 19: ■ Code

```
# Establecer como índice y ordenar cronológicamente ambos datasets limpios
df_energy_clean = df_energy_clean.set_index('time').sort_index()
df_weather_clean = df_weather_clean.set_index('time').sort_index()

print("Dataset energía")
print(f"\nPrimera fecha: {df_energy_clean.index.min()}")
print(f"Última fecha: {df_energy_clean.index.max()}")
print(f"■ Total observaciones: {len(df_energy_clean):,}")

print("\nDataset weather")
print(f"Primera fecha: {df_weather_clean.index.min()}")
print(f"Última fecha: {df_weather_clean.index.max()}")
print(f"■ Total observaciones: {len(df_weather_clean):,}")

df_energy_clean.head(3)
```

Output:

Dataset energía

Primera fecha: 2014-12-31 23:00:00+00:00

Última fecha: 2018-12-31 22:00:00+00:00

■ Total observaciones: 35,064

Dataset weather

Primera fecha: 2014-12-31 23:00:00+00:00

Última fecha: 2018-12-31 22:00:00+00:00

■ Total observaciones: 178,396

	generation biomass	...	price actual
time		...	
2014-12-31 23:00:00+00:00	447.0	...	65.41
2015-01-01 00:00:00+00:00	449.0	...	64.92

2015-01-01 01:00:00+00:00

448.0 ...

64.48

[3 rows x 23 columns]

Cell 20: ■ Markdown

Ambos datasets pasan a estar ordenados como series temporales reales, condición necesaria para imputar, hacer merge y entrenar modelos cronológicos sin ambigüedad.

Cell 21: ■ Markdown

4. Análisis de Valores Nulos

Identificamos y cuantificamos los valores nulos en los datasets limpios por separado. Es importante distinguir entre **valores 0.0** (generación nula registrada) y **valores NaN** (datos no reportados).

Columnas con 0.0 (Generación Nula Real)

Estas columnas tienen **toda su serie en 0.0** porque España **no utiliza estas fuentes energéticas**:

1. **generation fossil oil shale** (Esquistos bituminosos)

España no usa esquistos para generación eléctrica comercial.

2. **generation fossil peat turb** (Turba)

No hay centrales de turba operativas en España.

3. **generation geothermal** (Geotermia)

A excepción de pequeños proyectos en Canarias, España no produce electricidad geotérmica a gran escala.

4. **generation marine** (Energía marina/undimotriz)

Tecnología experimental. No aporta energía significativa a la red nacional.

5. **generation wind offshore** (Eólica marina)

Durante 2015-2018 (periodo del dataset), la eólica marina era **inexistente** en España.

Columnas con NaN (Datos No Reportados)

Valores **NaN** significan que el dato **no existe o no se reportó**:

1. **generation hydro pumped storage aggregated**

Las empresas reportan por separado consumo (bombeo hacia arriba) y generación (turbinado). La columna "agregada" queda vacía porque España identifica la energía generada en otro concepto (generation hydro water reservoir) y da error la agregación.

Estrategia de Tratamiento

- **Columnas con todo 0.0**: Eliminar (no aportan variabilidad ni información predictiva)
- **Columnas con NaN puntuales**: Aplicar interpolación temporal o forward/backward fill
- **Columnas con todo NaN**: Eliminar (sin información útil)
- **Todo esto se realiza antes del merge final**

Cell 22: ■ Code

```
# Calcular porcentaje de nulos por columna en ambos datasets limpios
threshold = 70

null_percentage_energy = (df_energy_clean.isnull().sum() / len(df_energy_clean)) *
100

energy_cols_to_drop = null_percentage_energy[null_percentage_energy >=
threshold].index.tolist()

print(f"Dataset energía antes: {df_energy_clean.shape}")
print(f"\nColumnas de energía con más de {threshold}% de nulos
({len(energy_cols_to_drop)}):")
if energy_cols_to_drop:
for col in sorted(energy_cols_to_drop):
print(f" - {col}: {null_percentage_energy[col]:.2f}%")
else:
print(" (ninguna)")

df_energy_clean = df_energy_clean.drop(columns=energy_cols_to_drop)

print(f"\nDataset energía después: {df_energy_clean.shape}")
print(f"Columnas eliminadas en energía: {len(energy_cols_to_drop)}")

null_percentage_weather = (df_weather_clean.isnull().sum() / len(df_weather_clean))
* 100

weather_cols_to_drop = null_percentage_weather[null_percentage_weather >=
threshold].index.tolist()

print(f"\nDataset weather antes: {df_weather_clean.shape}")
print(f"\nColumnas de weather con más de {threshold}% de nulos
({len(weather_cols_to_drop)}):")
if weather_cols_to_drop:
for col in sorted(weather_cols_to_drop):
print(f" - {col}: {null_percentage_weather[col]:.2f}%")
```

```

else:
    print(" (ninguna)")

df_weather_clean = df_weather_clean.drop(columns=weather_cols_to_drop)

print(f"\nDataset weather después: {df_weather_clean.shape}")
print(f"Columnas eliminadas en weather: {len(weather_cols_to_drop)}")

```

Output:

```

Dataset energía antes: (35064, 23)

Columnas de energía con más de 70% de nulos (1):
  - generation hydro pumped storage aggregated: 100.00%

Dataset energía después: (35064, 22)
Columnas eliminadas en energía: 1

Dataset weather antes: (178396, 13)

Columnas de weather con más de 70% de nulos (0):
  (ninguna)

Dataset weather después: (178396, 13)
Columnas eliminadas en weather: 0

```

Cell 23: ■ Markdown

El filtrado por porcentaje de nulos retira series estructuralmente vacías y concentra el análisis en columnas con señal potencialmente útil.

Cell 24: ■ Markdown

5. Composición final de los datasets

1. **Eliminar columnas con todo 0.0**

No aportan variabilidad ni información (generación inexistente en España)

2. **Interpolación temporal** para valores nulos puntuales

Método apropiado para series temporales porque asume cambios graduales entre valores consecutivos

3. **Forward/Backward Fill** como respaldo

Para NaN al inicio/final de la serie donde la interpolación no funciona

Este punto queda dividido en tres bloques:

1. Imputación sobre `df\_energy\_clean` y `df\_weather\_clean`
2. Pivot de `df\_weather\_clean`
3. Merge final de los datasets limpios

Cell 25: ■ Code

```
# Eliminar columnas numéricas con todo 0.0 en df_energy_clean
energy_zero_cols = [
col for col in df_energy_clean.select_dtypes(include=[np.number]).columns
if df_energy_clean[col].notna().any() and df_energy_clean[col].dropna().eq(0).all()
]

print(f"Columnas de energía con todo 0.0 ({len(energy_zero_cols)}):")
if energy_zero_cols:
for col in energy_zero_cols:
print(f" - {col}")
df_energy_clean = df_energy_clean.drop(columns=energy_zero_cols)
else:
print(" (ninguna)")

# Eliminar columnas numéricas con todo 0.0 en df_weather_clean
weather_zero_cols = [
col for col in df_weather_clean.select_dtypes(include=[np.number]).columns
if df_weather_clean[col].notna().any() and
df_weather_clean[col].dropna().eq(0).all()
]

print(f"\nColumnas de weather con todo 0.0 ({len(weather_zero_cols)}):")
if weather_zero_cols:
for col in weather_zero_cols:
print(f" - {col}")
df_weather_clean = df_weather_clean.drop(columns=weather_zero_cols)
else:
print(" (ninguna)")

# Imputación en df_energy_clean
```

```

energy_numeric_cols = df_energy_clean.select_dtypes(include=[np.number]).columns
for col in energy_numeric_cols:
    if df_energy_clean[col].isna().sum() > 0:
        antes = df_energy_clean[col].isna().sum()
        df_energy_clean[col] = df_energy_clean[col].interpolate(method='linear',
            limit_direction='both')
        despues = df_energy_clean[col].isna().sum()
        print(f"ENERGY - {col}: {antes} → {despues} nulos")

# Imputación en df_weather_clean por ciudad antes del pivot
weather_numeric_cols = df_weather_clean.select_dtypes(include=[np.number]).columns
for col in weather_numeric_cols:
    if df_weather_clean[col].isna().sum() > 0:
        antes = df_weather_clean[col].isna().sum()
        df_weather_clean[col] = df_weather_clean.groupby('city_name')[col].transform(
            lambda s: s.interpolate(method='linear', limit_direction='both')
        )
        despues = df_weather_clean[col].isna().sum()
        print(f"WEATHER - {col}: {antes} → {despues} nulos")

```

Output:

Columnas de energía con todo 0.0 (6):

- generation fossil coal-derived gas
- generation fossil oil shale
- generation fossil peat
- generation geothermal
- generation marine
- generation wind offshore

Columnas de weather con todo 0.0 (0):

(ninguna)

ENERGY - generation biomass: 19 → 0 nulos

ENERGY - generation fossil brown coal/lignite: 18 → 0 nulos

ENERGY - generation fossil gas: 18 → 0 nulos

ENERGY - generation fossil hard coal: 18 → 0 nulos

ENERGY - generation fossil oil: 19 → 0 nulos

ENERGY - generation hydro pumped storage consumption: 19 → 0 nulos

```
ENERGY - generation hydro run-of-river and poundage: 19 → 0 nulos
ENERGY - generation hydro water reservoir: 18 → 0 nulos
ENERGY - generation nuclear: 17 → 0 nulos
ENERGY - generation other: 18 → 0 nulos
ENERGY - generation other renewable: 18 → 0 nulos
ENERGY - generation solar: 18 → 0 nulos
ENERGY - generation waste: 19 → 0 nulos
ENERGY - generation wind onshore: 18 → 0 nulos
ENERGY - total load actual: 36 → 0 nulos
WEATHER - clouds_all: 36 → 0 nulos
```

Cell 26: ■ Markdown

Tras eliminar columnas sin variabilidad e imputar huecos puntuales respetando la estructura temporal, los datasets quedan listos para el pivot meteorológico y la integración final.

Cell 27: ■ Markdown

5a. *Pivot del Dataset Meteorológico*

Una vez limpio `df\_weather\_clean`, transformamos las observaciones por ciudad a formato ancho para obtener una fila por timestamp y una columna por variable meteorológica y ciudad.

Cell 28: ■ Code

```
# Pivot final de weather ya limpio

df_weather_pivot = df_weather_clean.pivot_table(
    index=df_weather_clean.index,
    columns='city_name',
    values=weather_numeric_cols,
    aggfunc='mean'
)

df_weather_pivot.columns = [f"{var}_{city}" for var, city in
df_weather_pivot.columns]

print(f"Dataset weather pivotado: {df_weather_pivot.shape[0]:,} filas ×
{df_weather_pivot.shape[1]} columnas")

df_weather_pivot.head(3)
```

Output:

```
Dataset weather pivotado: 35,064 filas × 45 columnas

              clouds_all_Barcelona  ...  wind_speed_Valencia
time                               ...
2014-12-31 23:00:00+00:00          0.0  ...                1.0
2015-01-01 00:00:00+00:00          0.0  ...                1.0
2015-01-01 01:00:00+00:00          0.0  ...                0.0

[3 rows x 45 columns]
```

Cell 29: ■ Markdown

El clima queda consolidado en formato ancho, con una sola fila por hora, lo que simplifica tanto el análisis exploratorio como la construcción de features por ciudad.

Cell 30: ■ Markdown

5b. Merge Final de Datasets Limpios

Con `df\_energy\_clean` y `df\_weather\_pivot` ya preparados, realizamos la unión final por índice temporal para construir el dataset analítico completo.

Cell 31: ■ Code

```
# Merge final de los datasets limpios
df = df_energy_clean.join(df_weather_pivot, how='inner')

print(f"Dataset combinado final: {df.shape[0]:,} filas × {df.shape[1]} columnas")
print(f"Rango de fechas: {df.index.min()} → {df.index.max()}")
print(f"Memoria utilizada: {df.memory_usage(deep=True).sum() / 1024**2:.2f} MB")

cities = ['Valencia', 'Madrid', 'Bilbao', 'Barcelona', 'Seville']
weather_cols_created = sorted(col for col in df.columns if any(city in col for city
in cities))

print(f"\nColumnas meteorológicas creadas ({len(weather_cols_created)}):")
for col in weather_cols_created:
    print(f" - {col}")

df_clean = df.copy()
df_clean.info()
```


Output:

Dataset combinado final: 35,064 filas × 61 columnas

Rango de fechas: 2014-12-31 23:00:00+00:00 → 2018-12-31 22:00:00+00:00

Memoria utilizada: 16.59 MB

Columnas meteorológicas creadas (45):

- clouds_all_Barcelona
- clouds_all_Bilbao
- clouds_all_Madrid
- clouds_all_Seville
- clouds_all_Valencia
- humidity_Barcelona
- humidity_Bilbao
- humidity_Madrid
- humidity_Seville
- humidity_Valencia
- pressure_Barcelona
- pressure_Bilbao
- pressure_Madrid
- pressure_Seville
- pressure_Valencia
- rain_1h_Barcelona
- rain_1h_Bilbao
- rain_1h_Madrid
- rain_1h_Seville
- rain_1h_Valencia
- rain_3h_Barcelona
- rain_3h_Bilbao
- rain_3h_Madrid
- rain_3h_Seville
- rain_3h_Valencia
- snow_3h_Barcelona
- snow_3h_Bilbao
- snow_3h_Madrid
- snow_3h_Seville

- snow_3h_Valencia
- temp_Barcelona
- temp_Bilbao
- temp_Madrid
- temp_Seville
- temp_Valencia
- wind_deg_Barcelona
- wind_deg_Bilbao
- wind_deg_Madrid
- wind_deg_Seville
- wind_deg_Valencia
- wind_speed_Barcelona
- wind_speed_Bilbao
- wind_speed_Madrid
- wind_speed_Seville
- wind_speed_Valencia

```
<class 'pandas.DataFrame'>
```

DatetimeIndex: 35064 entries, 2014-12-31 23:00:00+00:00 to 2018-12-31 22:00:00+00:00

Data columns (total 61 columns):

#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	generation biomass	35064 non-null	float64
1	generation fossil brown coal/lignite	35064 non-null	float64
2	generation fossil gas	35064 non-null	float64
3	generation fossil hard coal	35064 non-null	float64
4	generation fossil oil	35064 non-null	float64
5	generation hydro pumped storage consumption	35064 non-null	float64
6	generation hydro run-of-river and poundage	35064 non-null	float64
7	generation hydro water reservoir	35064 non-null	float64
8	generation nuclear	35064 non-null	float64
9	generation other	35064 non-null	float64
10	generation other renewable	35064 non-null	float64
11	generation solar	35064 non-null	float64
12	generation waste	35064 non-null	float64

13	generation wind onshore	35064	non-null	float64
14	total load actual	35064	non-null	float64
15	price actual	35064	non-null	float64
16	clouds_all_Barcelona	35064	non-null	float64
17	clouds_all_Bilbao	35064	non-null	float64
18	clouds_all_Madrid	35064	non-null	float64
19	clouds_all_Seville	35064	non-null	float64
20	clouds_all_Valencia	35064	non-null	float64
21	humidity_Barcelona	35064	non-null	float64
22	humidity_Bilbao	35064	non-null	float64
23	humidity_Madrid	35064	non-null	float64
24	humidity_Seville	35064	non-null	float64
25	humidity_Valencia	35064	non-null	float64
26	pressure_Barcelona	35064	non-null	float64
27	pressure_Bilbao	35064	non-null	float64
28	pressure_Madrid	35064	non-null	float64
29	pressure_Seville	35064	non-null	float64
30	pressure_Valencia	35064	non-null	float64
31	rain_1h_Barcelona	35064	non-null	float64
32	rain_1h_Bilbao	35064	non-null	float64
33	rain_1h_Madrid	35064	non-null	float64
34	rain_1h_Seville	35064	non-null	float64
35	rain_1h_Valencia	35064	non-null	float64
36	rain_3h_Barcelona	35064	non-null	float64
37	rain_3h_Bilbao	35064	non-null	float64
38	rain_3h_Madrid	35064	non-null	float64
39	rain_3h_Seville	35064	non-null	float64
40	rain_3h_Valencia	35064	non-null	float64
41	snow_3h_Barcelona	35064	non-null	float64
42	snow_3h_Bilbao	35064	non-null	float64
43	snow_3h_Madrid	35064	non-null	float64
44	snow_3h_Seville	35064	non-null	float64

... (19 more lines truncated)

Cell 32: ■ Markdown

`df\_clean` pasa a ser el dataset analítico principal, porque reúne en una sola tabla la demanda, la generación y las variables meteorológicas alineadas por hora. Aunque inicialmente haremos el analisis sin variables meteorologicas por simplificar en los modelos menos complejos

Cell 33: ■ Markdown

5c. Division Train/Test Temporal

Reservamos el ultimo año de datos horarios como conjunto de test y dejamos el resto para train, manteniendo el orden temporal para evitar leakage.

Cell 34: ■ Code

```
# Division temporal train/test: ultimo año de datos para test
def split_last_year_hourly(df):
    df = df.sort_index().copy()
    test_start = df.index.max() - pd.DateOffset(years=1) + pd.Timedelta(hours=1)
    train = df.loc[df.index < test_start].copy()
    test = df.loc[df.index >= test_start].copy()
    return train, test, test_start

df_clean_train, df_clean_test, df_clean_test_start =
split_last_year_hourly(df_clean)

df_energy_clean_train, df_energy_clean_test, df_energy_clean_test_start =
split_last_year_hourly(df_energy_clean)

for name, original, train_df, test_df, test_start in [
    ('df_clean', df_clean, df_clean_train, df_clean_test, df_clean_test_start),
    ('df_energy_clean', df_energy_clean, df_energy_clean_train, df_energy_clean_test,
df_energy_clean_test_start),
]:
    print(f'\n{name}')
    print(f'Train: {train_df.shape[0]:,} filas | {train_df.index.min()} ->
{train_df.index.max()}')
    print(f'Test: {test_df.shape[0]:,} filas | {test_df.index.min()} ->
{test_df.index.max()}')
    print(f'Inicio test: {test_start}')
    print(f'Suma train + test = total: {len(train_df) + len(test_df) ==
len(original)}')
```

Output:

```
df_clean
Train: 26,304 filas | 2014-12-31 23:00:00+00:00 -> 2017-12-31 22:00:00+00:00
Test: 8,760 filas | 2017-12-31 23:00:00+00:00 -> 2018-12-31 22:00:00+00:00
Inicio test: 2017-12-31 23:00:00+00:00

df_energy_clean
Train: 26,304 filas | 2014-12-31 23:00:00+00:00 -> 2017-12-31 22:00:00+00:00
Test: 8,760 filas | 2017-12-31 23:00:00+00:00 -> 2018-12-31 22:00:00+00:00
Inicio test: 2017-12-31 23:00:00+00:00

Suma train + test = total: True
```

Cell 35: ■ Markdown

El corte cronológico deja reservado el último año como test y hace válidas las comparaciones posteriores entre modelos, evitando leakage temporal.

Cell 36: ■ Markdown

6. Análisis Meteorológico

Antes de construir modelos conviene entender **qué información meteorológica aporta señal real** a la demanda y cuál añade solo ruido.

En este bloque no buscamos “hacer climatología”, sino responder preguntas útiles para el forecasting:

1. ¿Qué resumen meteorológico representa mejor a España en cada hora?
2. ¿La relación temperatura-demanda es lineal o no lineal?
3. ¿Cómo cambia el efecto del clima según la hora y el tipo de día?
4. ¿Qué variables meteorológicas parecen más prometedoras para el feature engineering?

El objetivo es que el paso al modelado quede justificado con evidencia y no solo por intuición.

Cell 37: ■ Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```

import seaborn as sns

sns.set_theme(style='whitegrid')

target = 'total load actual'

def get_prefixed_columns(df, prefixes):
    return sorted(
        col for col in df.columns
        if any(col.startswith(prefix) for prefix in prefixes)
    )

temp_cols = get_prefixed_columns(df_clean, ['temp_'])
humidity_cols = get_prefixed_columns(df_clean, ['humidity_'])
pressure_cols = get_prefixed_columns(df_clean, ['pressure_'])
wind_speed_cols = get_prefixed_columns(df_clean, ['wind_speed_'])
cloud_cols = get_prefixed_columns(df_clean, ['clouds_all_'])
rain_cols = get_prefixed_columns(df_clean, ['rain_'])

weather_cols_for_analysis = sorted(
    set(temp_cols + humidity_cols + pressure_cols + wind_speed_cols + cloud_cols +
        rain_cols)
)

df_weather_analysis = df_clean[[target] +
    weather_cols_for_analysis].copy().sort_index()

if not temp_cols:
    raise ValueError('No se han encontrado columnas de temperatura en df_clean para el
    análisis meteorológico.')

df_weather_analysis['temp_mean_spain'] =
df_weather_analysis[temp_cols].mean(axis=1)
df_weather_analysis['temp_std_spain'] = df_weather_analysis[temp_cols].std(axis=1)
df_weather_analysis['temp_range_spain'] =
df_weather_analysis[temp_cols].max(axis=1) -
df_weather_analysis[temp_cols].min(axis=1)

if humidity_cols:
    df_weather_analysis['humidity_mean_spain'] =
df_weather_analysis[humidity_cols].mean(axis=1)

if pressure_cols:

```

```
df_weather_analysis['pressure_mean_spain'] =  
df_weather_analysis[pressure_cols].mean(axis=1)
```

```
if wind_speed_cols:
```

```
df_weather_analysis['wind_speed_mean_spain'] =  
df_weather_analysis[wind_speed_cols].mean(axis=1)
```

```
if cloud_cols:
```

```
df_weather_analysis['clouds_mean_spain'] =  
df_weather_analysis[cloud_cols].mean(axis=1)
```

```
if rain_cols:
```

```
df_weather_analysis['rain_mean_spain'] =  
df_weather_analysis[rain_cols].mean(axis=1)
```

```
df_weather_analysis['hour'] = df_weather_analysis.index.hour
```

```
df_weather_analysis['day_of_week'] = df_weather_analysis.index.dayofweek
```

```
df_weather_analysis['is_weekend'] = (df_weather_analysis['day_of_week'] >= 5).astype(int)
```

```
df_weather_analysis['month'] = df_weather_analysis.index.month
```

```
df_weather_analysis['year'] = df_weather_analysis.index.year
```

```
temp_q1, temp_q2 = df_weather_analysis['temp_mean_spain'].quantile([0.33, 0.66])
```

```
df_weather_analysis['temp_regime'] = pd.cut(  
df_weather_analysis['temp_mean_spain'],  
bins=[-np.inf, temp_q1, temp_q2, np.inf],  
labels=['Frio', 'Templado', 'Calido']  
)
```

```
aggregate_weather_signals = [  
col for col in [  
'temp_mean_spain',  
'temp_std_spain',  
'temp_range_spain',  
'humidity_mean_spain',  
'pressure_mean_spain',  
'wind_speed_mean_spain',  
'clouds_mean_spain',  
'rain_mean_spain'  
]  
if col in df_weather_analysis.columns
```

```
]
```

```
corr_summary = (  
df_weather_analysis[[target] + aggregate_weather_signals]  
.corr(numeric_only=True)[target]  
.drop(target)  
.sort_values(key=lambda s: s.abs(), ascending=False)  
.to_frame('corr_con_demanda')  
.round(3)  
)  
  
print('Dataset meteorológico para análisis preparado.')  
print(f'Filas: {df_weather_analysis.shape[0]:,}')  
print(f'Temperaturas por ciudad disponibles: {len(temp_cols)}')  
print(f'Señales agregadas disponibles: {aggregate_weather_signals}')  
display(corr_summary)
```

Output:

Dataset meteorológico para análisis preparado.

Filas: 35,064

Temperaturas por ciudad disponibles: 5

Señales agregadas disponibles: ['temp_mean_spain', 'temp_std_spain', 'temp_range_spain', 'humidit

Index	corr_con_demanda
humidity_mean_spain	-0.371
temp_mean_spain	0.205
wind_speed_mean_spain	0.196
temp_std_spain	0.075
temp_range_spain	0.07
rain_mean_spain	0.019
clouds_mean_spain	0.016
pressure_mean_spain	-0.002

Cell 38: ■ Markdown

6a. Construcción de Señales Meteorológicas Agregadas

El dataset original trae observaciones por ciudad. Eso es útil para ML, pero para el análisis exploratorio conviene resumir primero el clima en unas pocas señales nacionales:

- `temp_mean_spain`: temperatura media horaria del conjunto de ciudades
- `temp_std_spain` / `temp_range_spain`: dispersión geográfica del clima
- `medias nacionales` de humedad, presión, viento, nubosidad y lluvia

Así podemos responder dos cosas: si el clima acompaña a la demanda a nivel agregado y qué variables merecen la pena conservar más adelante.

Cell 39: ■ Code

```
weekly_view = (
df_weather_analysis[[target, 'temp_mean_spain']]
.resample('W')
.mean()
)

monthly_profile = (
df_weather_analysis.groupby('month')[[target, 'temp_mean_spain']]
.mean()
.reindex(range(1, 13))
)

month_labels = ['Ene', 'Feb', 'Mar', 'Abr', 'May', 'Jun', 'Jul', 'Ago', 'Sep',
'Oct', 'Nov', 'Dic']

fig, axes = plt.subplots(2, 1, figsize=(15, 10), constrained_layout=True)

axes[0].plot(weekly_view.index, weekly_view[target], color='steelblue',
linewidth=1.8, label='Demanda semanal')
ax0b = axes[0].twinx()
ax0b.plot(weekly_view.index, weekly_view['temp_mean_spain'], color='tomato',
linewidth=1.5, alpha=0.9, label='Temperatura media semanal')
axes[0].set_title('Co-movimiento entre demanda y temperatura (media semanal)')
axes[0].set_ylabel('MW')
ax0b.set_ylabel('Temperatura (K)')
axes[0].grid(alpha=0.25)
lines1, labels1 = axes[0].get_legend_handles_labels()
lines2, labels2 = ax0b.get_legend_handles_labels()
```

```

axes[0].legend(lines1 + lines2, labels1 + labels2, loc='upper right')

axes[1].plot(monthly_profile.index, monthly_profile[target], marker='o',
color='steelblue', linewidth=2, label='Demanda media')
ax1b = axes[1].twinx()
ax1b.plot(monthly_profile.index, monthly_profile['temp_mean_spain'], marker='o',
color='tomato', linewidth=2, label='Temperatura media')
axes[1].set_xticks(range(1, 13))
axes[1].set_xticklabels(month_labels)
axes[1].set_title('Perfil medio mensual: demanda y temperatura')
axes[1].set_ylabel('MW')
ax1b.set_ylabel('Temperatura (K)')
axes[1].grid(alpha=0.25)
lines1, labels1 = axes[1].get_legend_handles_labels()
lines2, labels2 = ax1b.get_legend_handles_labels()
axes[1].legend(lines1 + lines2, labels1 + labels2, loc='upper right')

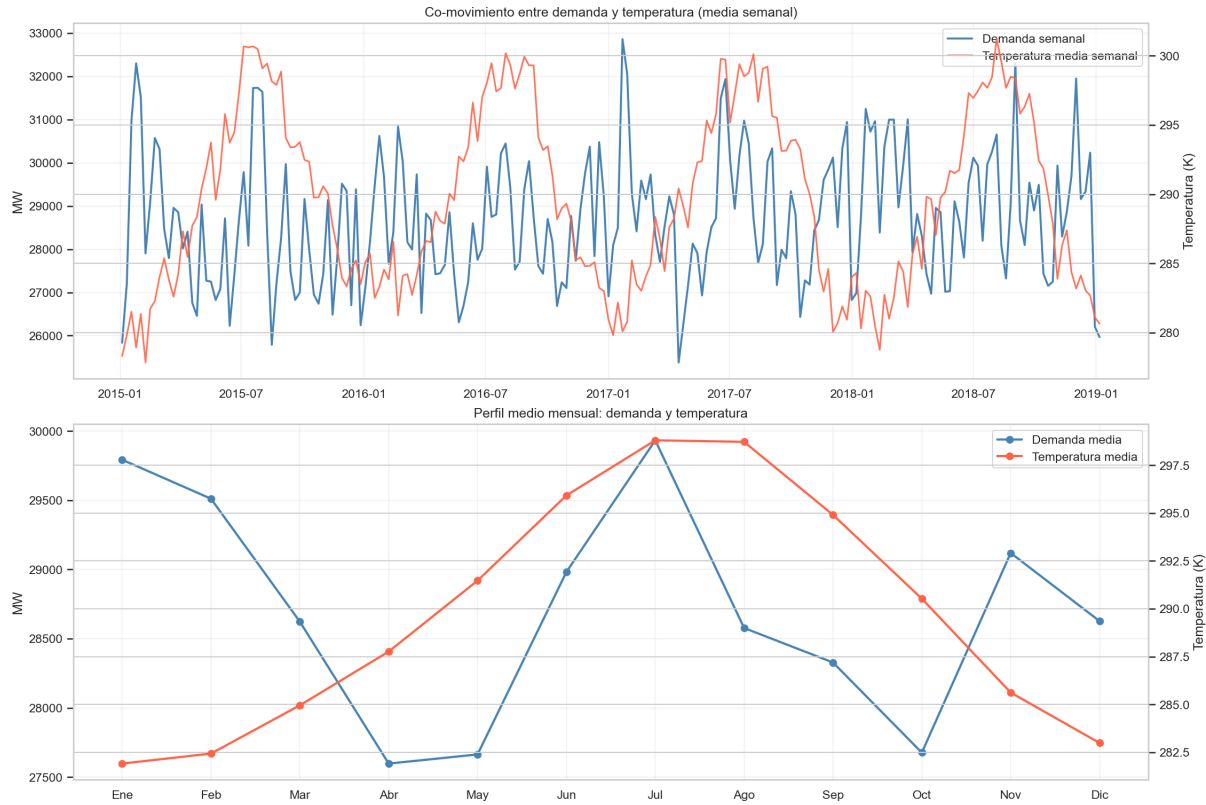
plt.show()

```

Output:

[STDERR]

<string>:1: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown



Cell 40: ■ Markdown

El repunte de la demanda en julio, aun con temperaturas muy altas, se explica por un efecto combinado: el calor incrementa el uso de climatización (especialmente aire acondicionado) y, al mismo tiempo, la actividad económica todavía se mantiene más alta que en agosto. De hecho, con temperaturas similares entre julio y agosto, julio presenta mayor consumo, sobre todo en horas diurnas y laborales. Esto sugiere que la demanda no depende solo del clima, sino también del calendario y de los patrones de actividad, por lo que conviene modelar su interacción (temperatura, hora y tipo de día) en el forecasting.

Cell 41: ■ Markdown

6b. Clima y Calendario Actúan Juntos

La demanda no responde igual a la temperatura en todas las horas ni en todos los días:

- un lunes frío a las 8:00 no se comporta igual que un domingo templado a las 8:00
- el patrón horario laboral y el patrón meteorológico se superponen

Este gráfico ayuda a ver si el clima cambia la **altura** de la curva diaria y si el efecto es distinto en laborales y fines de semana.

Cell 42: ■ Code

```
profile_cols = [target, 'hour', 'is_weekend']
df_profile = df_weather_analysis[profile_cols].dropna().copy()
```

```
laborable_profile = (
df_profile[df_profile['is_weekend'] == 0]
.groupby(['hour'], observed=True)[target]
.mean()
.reset_index()
)
```

```
weekend_profile = (
df_profile[df_profile['is_weekend'] == 1]
.groupby(['hour'], observed=True)[target]
.mean()
.reset_index()
)
```

```
fig, axes = plt.subplots(1, 2, figsize=(16, 6), sharey=True,
constrained_layout=True)
```

```
sns.lineplot(
data=laborable_profile,
x='hour',
y=target,
palette=['#1d3557', '#457b9d', '#e76f51'],
linewidth=2,
ax=axes[0]
)
axes[0].set_title('Perfil horario medio en laborables')
axes[0].set_xlabel('Hora del día')
axes[0].set_ylabel('Demanda (MW)')
axes[0].grid(alpha=0.25)
```

```
sns.lineplot(
data=weekend_profile,
x='hour',
y=target,
```

```

palette=['#1d3557', '#457b9d', '#e76f51'],
linewidth=2,
ax=axes[1]
)
axes[1].set_title('Perfil horario medio en fines de semana')
axes[1].set_xlabel('Hora del día')
axes[1].set_ylabel('')
axes[1].grid(alpha=0.25)

plt.show()

```

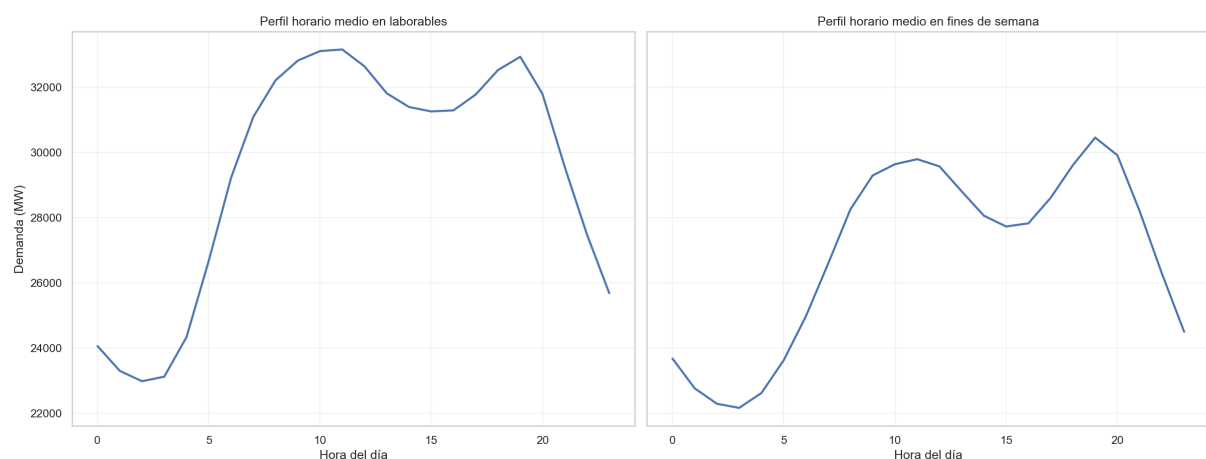
Output:

[STDERR]

```

<string>;20: UserWarning: Ignoring `palette` because no `hue` variable has been assigned.
<string>;33: UserWarning: Ignoring `palette` because no `hue` variable has been assigned.
<string>;1: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown

```



Cell 43: ■ Markdown

Estas curvas muestran cómo clima y calendario se combinan para explicar la demanda:

- En ambos paneles se mantiene la forma diaria típica (subida por la mañana, meseta diurna y descenso nocturno), lo que confirma un patrón horario estructural.
- En días laborables la separación entre curvas es más marcada en horas centrales, indicando que el efecto térmico se amplifica cuando coincide con actividad económica.
- En fines de semana el nivel total baja, pero el calor sigue desplazando la curva hacia arriba, por lo que el impacto meteorológico no desaparece.

Conclusión operativa: la demanda no depende solo de la temperatura ni solo del calendario; depende de su interacción. Por eso, para forecasting conviene incluir variables no lineales de temperatura y su cruce con hour e is_weekend.

Cell 44: ■ Markdown

7. Feature Engineering Completo

El **feature engineering** es **CRITICO** para el éxito de los modelos ML. Crearemos:

1. Features temporales (hora, día, mes)
2. Lags (autocorrelación)
3. Rolling statistics (tendencias locales)
4. **Weather features** (lags y rolling de temperatura)
5. Variables energéticas laggeadas (renovables vs fósiles)

> **Nota:** El calendario de festivos se mantiene como dataframe exógeno independiente (`df_holidays_es`) para SARIMA.

Cell 45: ■ Markdown

7. Feature Engineering - Parte A: Variables Temporales

La creación de features temporales es **fundamental** para capturar patrones diarios, semanales y estacionales en la demanda eléctrica.

Features cíclicas: Las transformaciones sin/cos permiten que los modelos ML entiendan que la hora 23 está cerca de la hora 0.

Cell 46: ■ Code

```
import numpy as np
import pandas as pd

# Copias para ML; mantenemos los datasets limpios sin tocar para ARIMA/SARIMA
# df_clean y df_energy_clean seguirán siendo la base para modelos estadísticos.
# Los festivos se mantienen aparte en df_holidays_es para utilizarlos como exog.
df_clean_fe = df_clean.copy()
df_energy_clean_fe = df_energy_clean.copy()
```

```
feature_groups_clean = {group: [] for group in ['temporal', 'lags', 'rolling',
'weather', 'energy']}
feature_groups_energy = {group: [] for group in ['temporal', 'lags', 'rolling',
'weather', 'energy']}
```

```
def add_temporal_features(df):
df = df.copy()
df['hour'] = df.index.hour
df['day_of_week'] = df.index.dayofweek
df['month'] = df.index.month
df['quarter'] = df.index.quarter
df['day_of_year'] = df.index.dayofyear
df['is_weekend'] = (df['day_of_week'] >= 5).astype(int)
df['hour_sin'] = np.sin(2 * np.pi * df['hour'] / 24)
df['hour_cos'] = np.cos(2 * np.pi * df['hour'] / 24)
df['day_of_week_sin'] = np.sin(2 * np.pi * df['day_of_week'] / 7)
df['day_of_week_cos'] = np.cos(2 * np.pi * df['day_of_week'] / 7)
df['month_sin'] = np.sin(2 * np.pi * df['month'] / 12)
df['month_cos'] = np.cos(2 * np.pi * df['month'] / 12)
```

```
temporal_cols = [
'hour', 'day_of_week', 'month', 'quarter', 'day_of_year', 'is_weekend',
'hour_sin', 'hour_cos', 'day_of_week_sin', 'day_of_week_cos',
'month_sin', 'month_cos'
]
return df, temporal_cols
```

```
df_clean_fe, temporal_cols = add_temporal_features(df_clean_fe)
df_energy_clean_fe, _ = add_temporal_features(df_energy_clean_fe)
```

```
feature_groups_clean['temporal'] = temporal_cols.copy()
feature_groups_energy['temporal'] = temporal_cols.copy()
```

```
print('Feature engineering inicializado sobre copias de los datasets limpios.')
print(f'df_clean base: {df_clean.shape} -> df_clean_fe: {df_clean_fe.shape}')
print(f'df_energy_clean base: {df_energy_clean.shape} -> df_energy_clean_fe:
{df_energy_clean_fe.shape}')
print(f'Variables temporales creadas: {len(temporal_cols)}')
```

```
print('Festivos disponibles aparte en df_holidays_es para modelos con exogenas')
```

Output:

```
Feature engineering inicializado sobre copias de los datasets limpios.  
df_clean base:          (35064, 61) -&gt; df_clean_fe: (35064, 73)  
df_energy_clean base: (35064, 16) -&gt; df_energy_clean_fe: (35064, 28)  
Variables temporales creadas: 12  
Festivos disponibles aparte en df_holidays_es para modelos con exogenas
```

Cell 47: ■ Markdown

Quedan creadas dos bases paralelas para ML y se incorporan variables temporales que recogen calendario diario, semanal y estacional sin tocar los datasets reservados a ARIMA/SARIMA.

Cell 48: ■ Markdown

8. Feature Engineering - Parte B: Lags (Autocorrelación)

Los **lags** capturan la autocorrelación temporal: el valor actual depende de valores pasados.

Lags clave:

- **lag_1**: Valor hace 1 hora
- **lag_24**: Valor hace 24 horas (ayer misma hora)
- **lag_168**: Valor hace 168 horas (semana pasada misma hora)

Cell 49: ■ Code

```
def add_load_lag_features(df, target_col='total load actual', lags=(1, 24, 168)):  
    df = df.copy()  
    lag_cols = []  
    for lag in lags:  
        col = f'load_lag_{lag}'  
        df[col] = df[target_col].shift(lag)  
        lag_cols.append(col)  
    return df, lag_cols
```

```
df_clean_fe, lag_cols = add_load_lag_features(df_clean_fe)
```



```
df_energy_clean_fe, _ = add_load_lag_features(df_energy_clean_fe)

feature_groups_clean['lags'] = lag_cols.copy()
feature_groups_energy['lags'] = lag_cols.copy()

print('Lags de carga creados en ambos datasets ML:')
print(lag_cols)
print(f'df_clean_fe columnas: {df_clean_fe.shape[1]}')
print(f'df_energy_clean_fe columnas: {df_energy_clean_fe.shape[1]}')
```

Output:

```
Lags de carga creados en ambos datasets ML:
['load_lag_1', 'load_lag_24', 'load_lag_168']
df_clean_fe columnas: 76
df_energy_clean_fe columnas: 31
```

Cell 50: ■ Markdown

Los lags introducen memoria explícita de la demanda, una señal clave en series horarias eléctricas donde la persistencia a 1h, 24h y 168h suele ser muy fuerte.

Cell 51: ■ Markdown

9. Feature Engineering - Parte C: Rolling Statistics

Las **rolling statistics** (estadísticas móviles) capturan tendencias y volatilidad reciente.

Ventanas importantes:

- **24 horas**: Captura tendencia/volatilidad del último día
- **168 horas**: Captura tendencia/volatilidad de la última semana
- **720 horas**: Captura tendencia/volatilidad del último mes

Cell 52: ■ Code

```
# Definimos las ventanas en horas
ventanas = [24, 168, 720]

for v in ventanas:
```

```
# Media móvil: tendencia suavizada
df_energy_clean[f'rolling_mean_{v}'] = df_energy_clean['total load
actual'].rolling(window=v).mean()

# Desviación típica móvil: volatilidad (incertidumbre)
df_energy_clean[f'rolling_std_{v}'] = df_energy_clean['total load
actual'].rolling(window=v).std()

# Eliminamos los nulos creados por las ventanas (NaN al inicio)
df_energy_clean.dropna(inplace=True)

print(f"Nuevas variables creadas para ventanas de: {ventanas} horas")
```

Output:

```
Nuevas variables creadas para ventanas de: [24, 168, 720] horas
```

Cell 53: ■ Markdown

1. ¿Por qué 3 ventanas diferentes?

Media 24h (Diaria): Elimina el "ruido" de si a las 3 p.m. se encendieron todos los hornos de España. Te muestra si hoy ha sido un día más caluroso o productivo que ayer. Es tu indicador de corto plazo.

Media 168h (Semanal): Elimina el efecto "fin de semana". Si comparas la demanda de un martes con la de un domingo, la gráfica se vuelve loca. Esta media suaviza esa diferencia y te dice si la semana en general está teniendo un consumo alto.

Media 720h (Mensual): Esta es la que marca la estacionalidad pura. Ignora si es lunes o domingo, o si hubo un festivo. Solo te dice: "Estamos en invierno y la calefacción está tirando de la red" o "Estamos en primavera y el consumo ha bajado".

2. ¿Qué aporta a tu modelo de Forecasting?

Si le das al modelo (Prophet o SARIMA) la columna de rolling_mean_24, el modelo entiende mejor el momento en el que está.

Ayuda a detectar anomalías: Si la demanda real se separa mucho de la media de 168h, algo raro está pasando (un festivo nacional no registrado, una huelga, o un temporal extremo).

Cell 54: ■ Code

```
plt.figure(figsize=(15, 6))
plt.plot(df_energy_clean.index, df_energy_clean['total load actual'], alpha=0.3,
label='Real (Horario)')
plt.plot(df_energy_clean.index, df_energy_clean['rolling_mean_24'], label='Media
24h', linewidth=2)
```

```
plt.plot(df_energy_clean.index, df_energy_clean['rolling_mean_168'], label='Media Semanal', linewidth=3)

plt.plot(df_energy_clean.index, df_energy_clean['rolling_mean_720'], label='Media Mensual', linewidth=4)

plt.title("Análisis de Tendencias: Descomposición de la Demanda mediante Medias Móviles")

plt.ylabel("Demanda (MW)")

plt.xlabel("Fecha")

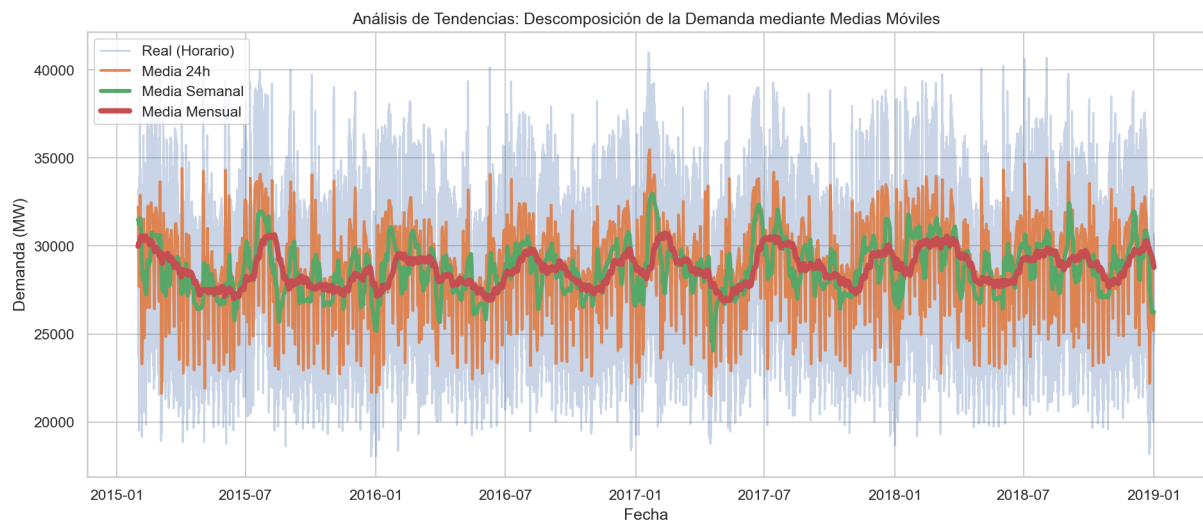
plt.legend()

plt.show()
```

Output:

[STDERR]

<string>:1: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown



Cell 55: ■ Markdown

La visualización del histórico completo permite identificar una tendencia base estable con ciclos estacionales anuales. La aplicación de la media mensual (720h) actúa como un filtro que elimina la volatilidad de corto plazo, revelando que los picos de demanda máxima se concentran en los meses de invierno (enero-febrero) y verano (julio-agosto). Su trabajo es decirnos en qué estación estamos. Solo cambia cuando el clima obliga a un cambio de hábitos masivo: sube en picado cuando encendemos los aires acondicionados en verano y vuelve a subir cuando llega el frío de diciembre.

Cell 56: ■ Code

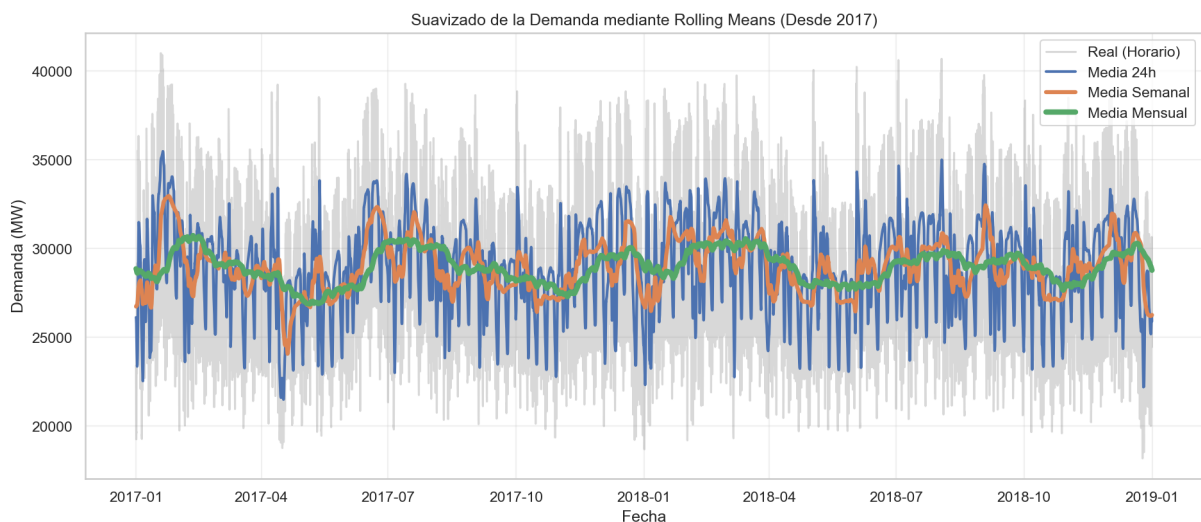
```
import matplotlib.pyplot as plt
```

```
# 1. Filtrar los datos para mostrar solo de 2017 en adelante
df_plot = df_energy_clean.loc['2017-01-01':]

# 2. Crear la figura
plt.figure(figsize=(15, 6))

# Graficar usando el DataFrame filtrado
plt.plot(df_plot.index, df_plot['total load actual'], alpha=0.3, label='Real (Horario)', color='gray')
plt.plot(df_plot.index, df_plot['rolling_mean_24'], label='Media 24h', linewidth=2)
plt.plot(df_plot.index, df_plot['rolling_mean_168'], label='Media Semanal', linewidth=3)
plt.plot(df_plot.index, df_plot['rolling_mean_720'], label='Media Mensual', linewidth=4)

# Configuración de la gráfica
plt.title("Suavizado de la Demanda mediante Rolling Means (Desde 2017)")
plt.ylabel("Demanda (MW)")
plt.xlabel("Fecha")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```



Cell 57: ■ Markdown

- La línea de 24h (Azul): El pulso del día a día. Esta media es la más nerviosa porque reacciona rápido. Nos dice cómo ha ido el consumo en las últimas horas comparado con el día anterior. Si ves que sube

de repente, es probable que haya entrado un frente de frío o calor. Es muy útil para ver la tendencia inmediata, pero todavía "sufre" mucho los bajones de los fines de semana.

- La línea de 168h (Naranja): El ritmo de la semana. Al promediar una semana entera, borra el efecto del sábado y el domingo. Mientras que la demanda real se desploma cada fin de semana, esta línea naranja se mantiene firme, mostrándonos si la actividad económica y social de esa semana en concreto está subiendo o bajando. Es el mejor indicador para saber si estamos en una semana de mucha actividad industrial o si el país ha bajado el ritmo. Los picos más altos se puede apreciar que coinciden con enero/febrero y en los meses de verano.

Cell 58: ■ Code

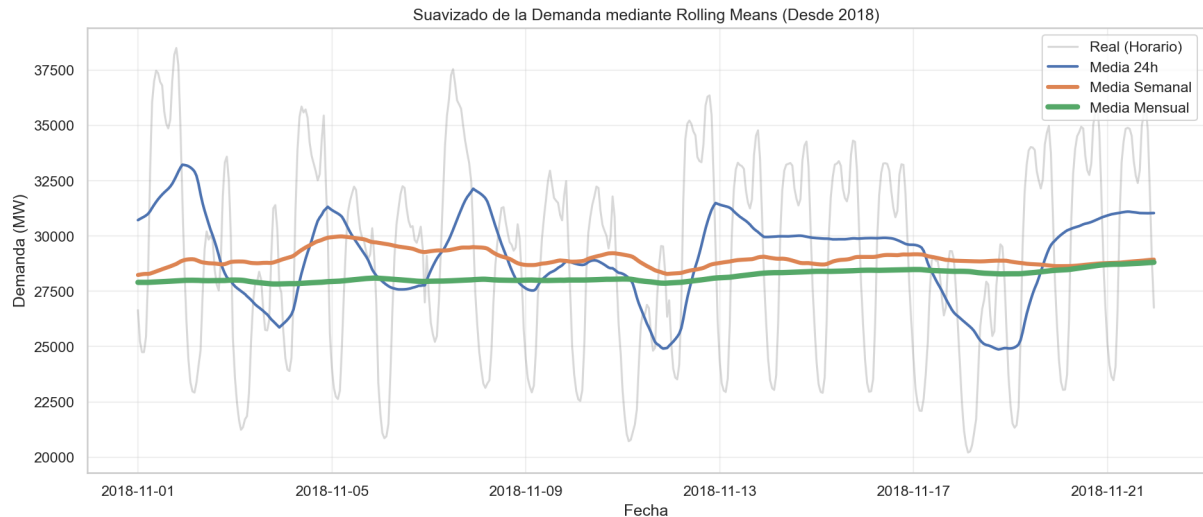
```
import matplotlib.pyplot as plt

# 1. Filtrar los datos para mostrar solo de 2017 en adelante
df_plot = df_energy_clean.loc['2018-11-01':'2018-11-21']

# 2. Crear la figura
plt.figure(figsize=(15, 6))

# Graficar usando el DataFrame filtrado
plt.plot(df_plot.index, df_plot['total load actual'], alpha=0.3, label='Real (Horario)', color='gray')
plt.plot(df_plot.index, df_plot['rolling_mean_24'], label='Media 24h', linewidth=2)
plt.plot(df_plot.index, df_plot['rolling_mean_168'], label='Media Semanal', linewidth=3)
plt.plot(df_plot.index, df_plot['rolling_mean_720'], label='Media Mensual', linewidth=4)

# Configuración de la gráfica
plt.title("Suavizado de la Demanda mediante Rolling Means (Desde 2018)")
plt.ylabel("Demanda (MW)")
plt.xlabel("Fecha")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```



Cell 59: ■ Markdown

En el patrón semanal cada "valle" o caída profunda que se repite rítmicamente suele representar el fin de semana.

Si contaras los picos entre una caída grande y la siguiente, verías que hay grupos de 5 días laborales más o menos.

Cell 60: ■ Code

```
import matplotlib.pyplot as plt

# 1. Crear la figura y el primer eje (para la demanda real)
fig, ax1 = plt.subplots(figsize=(15, 7))

# Graficamos la demanda real (Eje Y Izquierdo)
ax1.plot(df_energy_clean.index, df_energy_clean['total load actual'],
alpha=0.2, color='gray', label='Demanda Real (Horario)')
ax1.set_ylabel('Demanda (MW)', fontsize=12)
ax1.tick_params(axis='y')

# 2. Crear el segundo eje Y (para las volatilidades)
ax2 = ax1.twinx()

# Graficamos las volatilidades (Eje Y Derecho)
ax2.plot(df_energy_clean.index, df_energy_clean['rolling_std_24'],
label='Volatilidad Diaria (24h)', linewidth=1, alpha=0.8)
```

```

ax2.plot(df_energy_clean.index, df_energy_clean['rolling_std_168'],
label='Volatilidad Semanal (168h)', linewidth=2)

ax2.plot(df_energy_clean.index, df_energy_clean['rolling_std_720'],
label='Volatilidad Mensual (720h)', linewidth=2.5)

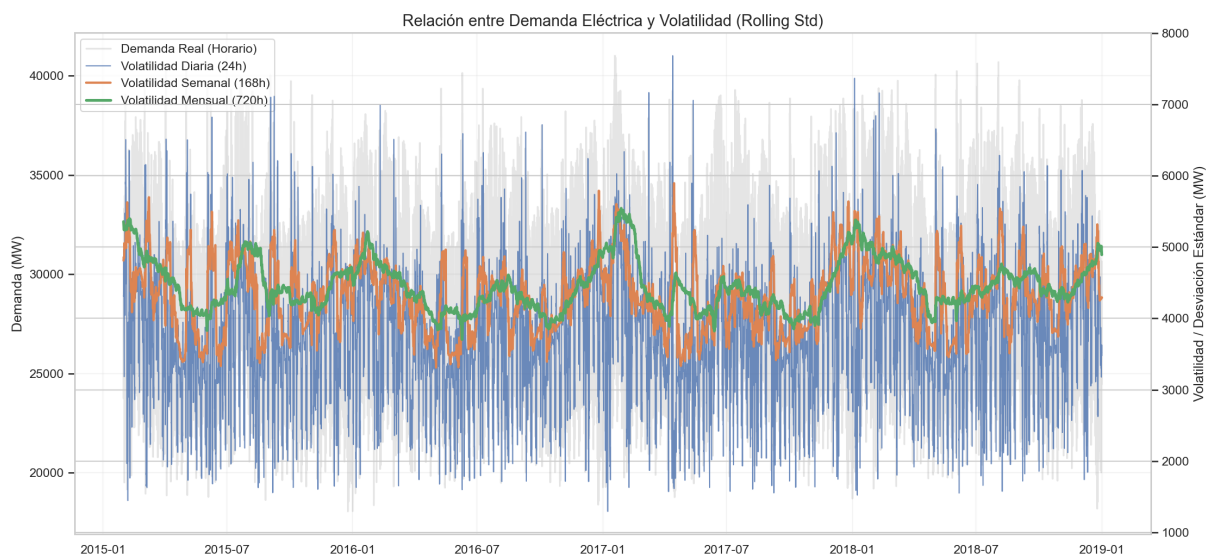
# Configuración del eje secundario
ax2.set_ylabel('Volatilidad / Desviación Estándar (MW)', fontsize=12)
ax2.tick_params(axis='y')

# 3. Títulos y leyenda
plt.title("Relación entre Demanda Eléctrica y Volatilidad (Rolling Std)",
fontsize=14)
ax1.grid(True, alpha=0.2)

# Unimos las leyendas de ambos ejes para que salgan en un solo cuadro
lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines1 + lines2, labels1 + labels2, loc='upper left')

plt.tight_layout()
plt.show()

```



Cell 61: ■ Markdown

- Meses de Alta Volatilidad (Invierno y Verano): Durante enero, febrero, julio y agosto, la volatilidad se dispara. Esto ocurre porque la demanda depende totalmente del clima extremo. Un día especialmente frío o una ola de calor repentina provocan cambios bruscos en el uso de calefacción y aire

acondicionado, haciendo que la curva de demanda sea mucho más errática y difícil de predecir.

- Meses de Baja Volatilidad (Primavera y Otoño): En meses como mayo u octubre (las llamadas "estaciones valle"), la volatilidad cae. El clima es templado, no hay un uso intensivo de climatización y el consumo se vuelve más rutinario y predecible. Aquí es donde nuestros modelos suelen tener sus mejores métricas.

- El factor "Incertidumbre": Esta gráfica nos advierte de que el riesgo de error de nuestro modelo es mayor en los picos estacionales. Para un gestor de red eléctrica, esto significa que en enero debe tener más "reservas" de energía preparadas, porque la probabilidad de que la demanda real se desvíe del forecast es mucho más alta que en primavera.

Cell 62: ■ Code

```
import matplotlib.pyplot as plt

# 1. Filtrar los datos desde 2018 en adelante
df_plot = df_energy_clean.loc['2017-01-01:']

# 2. Crear la figura y el primer eje (para la demanda real)
fig, ax1 = plt.subplots(figsize=(15, 7))

# Graficamos la demanda real (Eje Y Izquierdo) usando el DF filtrado
ax1.plot(df_plot.index, df_plot['total load actual'],
alpha=0.15, color='gray', label='Demanda Real (Horario)')
ax1.set_ylabel('Demanda (MW)', fontsize=12)
ax1.tick_params(axis='y')

# 3. Crear el segundo eje Y (para las volatilidades)
ax2 = ax1.twinx()

# Graficamos las volatilidades (Eje Y Derecho)
ax2.plot(df_plot.index, df_plot['rolling_std_24'],
label='Volatilidad Diaria (24h)', linewidth=1, alpha=0.7)

ax2.plot(df_plot.index, df_plot['rolling_std_168'],
label='Volatilidad Semanal (168h)', linewidth=2)

ax2.plot(df_plot.index, df_plot['rolling_std_720'],
label='Volatilidad Mensual (720h)', linewidth=3, color='green')

# Configuración del eje secundario
ax2.set_ylabel('Volatilidad / Desviación Estándar (MW)', fontsize=12)
ax2.tick_params(axis='y')
```



```
# 4. Títulos y leyenda
```

```
plt.title("Relación entre Demanda Eléctrica y Volatilidad (Desde 2017)",  
fontsize=14)
```

```
ax1.grid(True, alpha=0.2)
```

```
# Unimos las leyendas de ambos ejes
```

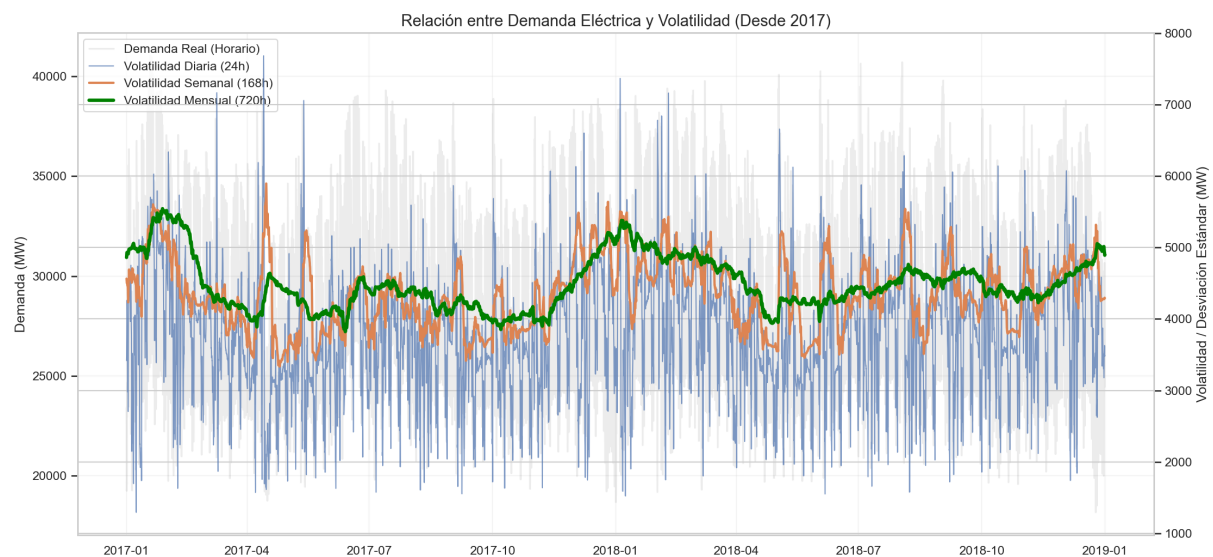
```
lines1, labels1 = ax1.get_legend_handles_labels()
```

```
lines2, labels2 = ax2.get_legend_handles_labels()
```

```
ax1.legend(lines1 + lines2, labels1 + labels2, loc='upper left')
```

```
plt.tight_layout()
```

```
plt.show()
```



Cell 63: ■ Markdown

- Los picos de incertidumbre: Si te fijas, la volatilidad no es plana. Hay momentos (especialmente en invierno y verano) donde las líneas de 24h y 168h se disparan. Esto nos dice que en las épocas de mayor consumo, la demanda se vuelve más errática y "nerviosa".

- La volatilidad de 168h (Semanal): Esta línea es clave porque nos marca los periodos donde el ritmo de la semana se vuelve impredecible. Por ejemplo, en diciembre/enero/febrero la volatilidad sube muchísimo no porque se consuma más (que también), sino porque los festivos rompen el esquema habitual y el modelo tiene más riesgo de fallar.

Cell 64: ■ Markdown

10. Feature Engineering - Parte D: ****Weather Features****

Las ****weather features son FUNDAMENTALES**** para predecir demanda eléctrica. La temperatura afecta directamente el consumo (calefacción en invierno, aire acondicionado en verano).

****Features meteorológicas clave:****

- Lags de temperatura (temp_lag24, temp_lag168)
- Rolling statistics de temperatura
- Interacciones con hora del día

Cell 65: ■ Code

```
def add_weather_features(df):
    df = df.copy()
    base_weather_cols = sorted(
        col for col in df.columns
        if col.startswith(('temp_', 'humidity_', 'pressure_', 'wind_speed_', 'wind_deg_',
                           'rain_', 'snow_', 'clouds_all_'))
    )
    temp_cols = [col for col in base_weather_cols if col.startswith('temp_')]
    weather_cols = base_weather_cols.copy()

    if not temp_cols:
        return df, weather_cols

    df['temp_mean_spain'] = df[temp_cols].mean(axis=1)
    df['temp_std_spain'] = df[temp_cols].std(axis=1)
    df['temp_range_spain'] = df[temp_cols].max(axis=1) - df[temp_cols].min(axis=1)
    df['temp_mean_spain_rolling_mean_24h'] =
    df['temp_mean_spain'].shift(1).rolling(window=24).mean()
    df['temp_mean_spain_rolling_mean_168h'] =
    df['temp_mean_spain'].shift(1).rolling(window=168).mean()

    engineered_weather_cols = [
        'temp_mean_spain',
        'temp_std_spain',
        'temp_range_spain',
        'temp_mean_spain_rolling_mean_24h',
        'temp_mean_spain_rolling_mean_168h'
```

```

]

for col in temp_cols:
    lag24 = f'{col}_lag_24'
    lag168 = f'{col}_lag_168'
    roll24 = f'{col}_rolling_mean_24h'
    df[lag24] = df[col].shift(24)
    df[lag168] = df[col].shift(168)
    df[roll24] = df[col].shift(1).rolling(window=24).mean()
    engineered_weather_cols.extend([lag24, lag168, roll24])

df['temp_mean_x_hour_sin'] = df['temp_mean_spain'] * df['hour_sin']
df['temp_mean_x_hour_cos'] = df['temp_mean_spain'] * df['hour_cos']
engineered_weather_cols.extend(['temp_mean_x_hour_sin', 'temp_mean_x_hour_cos'])

weather_cols.extend(engineered_weather_cols)
weather_cols = list(dict.fromkeys(weather_cols))
return df, weather_cols

df_clean_fe, weather_cols = add_weather_features(df_clean_fe)
feature_groups_clean['weather'] = weather_cols.copy()
feature_groups_energy['weather'] = []

print('Weather features anadidas al dataset combinado para ML.')
print(f'Variables weather totales disponibles en df_clean_fe: {len(weather_cols)}')
print('df_energy_clean_fe no recibe weather features porque no contiene variables meteorologicas.')

```

Output:

```

Weather features anadidas al dataset combinado para ML.
Variables weather totales disponibles en df_clean_fe: 67
df_energy_clean_fe no recibe weather features porque no contiene variables meteorologicas.

```

Cell 66: ■ Markdown

Las variables meteorológicas se añaden solo a `df\_clean\_fe`, lo que deja una comparación limpia entre modelos con y sin weather sin duplicar lógica más adelante.

Cell 67: ■ Markdown

11. Feature Engineering - Parte E: Variables Energéticas

Creamos agregaciones de las fuentes de generación eléctrica para simplificar el análisis.

****Importante:**** para evitar leakage en forecasting, las features energéticas se construyen con ****lags temporales**** y no con valores contemporáneos de la misma hora objetivo.

Cell 68: ■ Code

```
def add_energy_features(df, lags=(1, 24, 168)):
    df = df.copy()

    base_energy_cols = sorted(col for col in df.columns if col.startswith('generation
'))
    fossil_cols = [
        'generation fossil brown coal/lignite',
        'generation fossil gas',
        'generation fossil hard coal',
        'generation fossil oil'
    ]
    renewable_cols = [
        'generation biomass',
        'generation hydro run-of-river and poundage',
        'generation hydro water reservoir',
        'generation other renewable',
        'generation solar',
        'generation wind onshore'
    ]
    hydro_cols = [
        'generation hydro pumped storage consumption',
        'generation hydro run-of-river and poundage',
        'generation hydro water reservoir'
    ]
    nuclear_cols = ['generation nuclear']

    fossil_existing = [col for col in fossil_cols if col in df.columns]
```

```

renewable_existing = [col for col in renewable_cols if col in df.columns]
hydro_existing = [col for col in hydro_cols if col in df.columns]
nuclear_existing = [col for col in nuclear_cols if col in df.columns]

```

```

fossil_total = df[fossil_existing].sum(axis=1) if fossil_existing else
pd.Series(0.0, index=df.index)
renewable_total = df[renewable_existing].sum(axis=1) if renewable_existing else
pd.Series(0.0, index=df.index)
hydro_total = df[hydro_existing].sum(axis=1) if hydro_existing else pd.Series(0.0,
index=df.index)
nuclear_total = df[nuclear_existing].sum(axis=1) if nuclear_existing else
pd.Series(0.0, index=df.index)
low_carbon_total = renewable_total + nuclear_total
generation_total = df[base_energy_cols].sum(axis=1) if base_energy_cols else
pd.Series(0.0, index=df.index)

```

```

aggregate_series = {
'generation_fossil_total': fossil_total,
'generation_renewable_total': renewable_total,
'generation_hydro_total': hydro_total,
'generation_nuclear_total': nuclear_total,
'generation_low_carbon_total': low_carbon_total,
'generation_total': generation_total,
}

```

```

generation_denominator = generation_total.replace(0, np.nan)
share_series = {
'renewable_share_of_generation': renewable_total / generation_denominator,
'fossil_share_of_generation': fossil_total / generation_denominator,
'low_carbon_share_of_generation': low_carbon_total / generation_denominator,
}

```

```

engineered_energy_cols = []
for feature_name, series in (**aggregate_series, **share_series).items():
for lag in lags:
col = f'{feature_name}_lag_{lag}'
df[col] = series.shift(lag)
engineered_energy_cols.append(col)

```

```

energy_cols = list(dict.fromkeys(engineered_energy_cols))
return df, energy_cols

df_clean_fe, energy_cols = add_energy_features(df_clean_fe)
df_energy_clean_fe, _ = add_energy_features(df_energy_clean_fe)

feature_groups_clean['energy'] = energy_cols.copy()
feature_groups_energy['energy'] = energy_cols.copy()

all_feature_cols_clean = list(dict.fromkeys([col for cols in
feature_groups_clean.values() for col in cols]))
all_feature_cols_energy = list(dict.fromkeys([col for cols in
feature_groups_energy.values() for col in cols]))

print('Variables energeticas agregadas en ambas copias para ML.')
print(f'df_clean_fe final: {df_clean_fe.shape} | features disponibles:
{len(all_feature_cols_clean)}')
print(f'df_energy_clean_fe final: {df_energy_clean_fe.shape} | features
disponibles: {len(all_feature_cols_energy)}')
print('Datasets finales:')
print(' - Base para ARIMA/SARIMA: df_clean, df_energy_clean')
print(' - Con feature engineering para ML: df_clean_fe, df_energy_clean_fe')

```

Output:

```

Variables energeticas agregadas en ambas copias para ML.
df_clean_fe final: (35064, 125) | features disponibles: 109
df_energy_clean_fe final: (35064, 58) | features disponibles: 42
Datasets finales:
- Base para ARIMA/SARIMA: df_clean, df_energy_clean
- Con feature engineering para ML: df_clean_fe, df_energy_clean_fe

```

Cell 69: ■ Markdown

El feature engineering queda cerrado con agregados energéticos laggeados y ya están definidas las dos familias finales de datasets: base estadística y base ML.

Cell 70: ■ Markdown

12. Descomposición Temporal (Tendencia, Estacionalidad, Residuo)

La descomposición temporal permite separar la serie en:

- **Tendencia**: Patrón de largo plazo
- **Estacionalidad**: Patrón repetitivo (diario/semanal)
- **Residuo**: Variación aleatoria

Cell 71: ■ Code

```
serie_energia = df_energy_clean['total load actual']
```

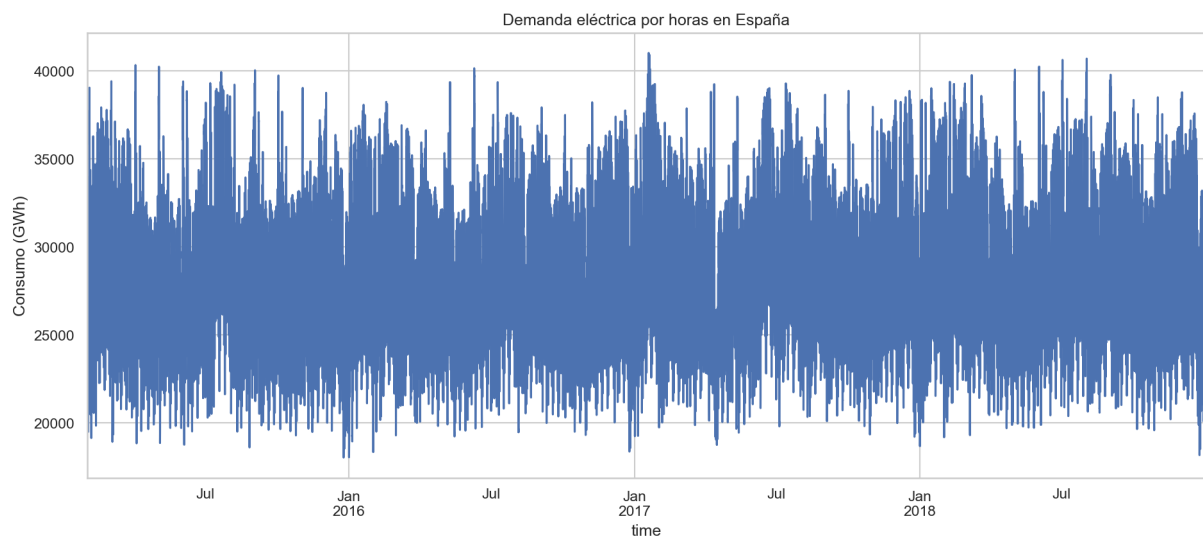
Cell 72: ■ Code

```
plt.figure(figsize=(15, 6))
serie_energia.plot(title='Demanda eléctrica por horas en España')
plt.ylabel('Consumo (GWh)')
plt.show()
```

Output:

[STDERR]

<string>;1: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown



Cell 73: ■ Markdown

La serie horaria confirma una variabilidad fuerte de corto plazo, por lo que cualquier modelo competitivo debe capturar bien ciclos diarios, semanales y estacionales.

Cell 74: ■ Markdown

Los datos diarios son muy ruidosos. Se remuestrean los datos a una frecuencia **diaria ('D')**, **semanal ('W')** y **mensual ('M')**. Para cada una, grafica el consumo medio.

Cell 75: ■ Code

```
df_diario = serie_energia.resample('D').mean()
df_semanal = serie_energia.resample('W').mean()
df_mensual = serie_energia.resample('ME').mean()

fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(15, 12))

df_diario.plot(ax=ax1, title='Demanda Media Diaria')
ax1.set_ylabel('Demanda (MW)')

df_semanal.plot(ax=ax2, title='Demanda Media Semanal')
ax2.set_ylabel('Demanda (MW)')

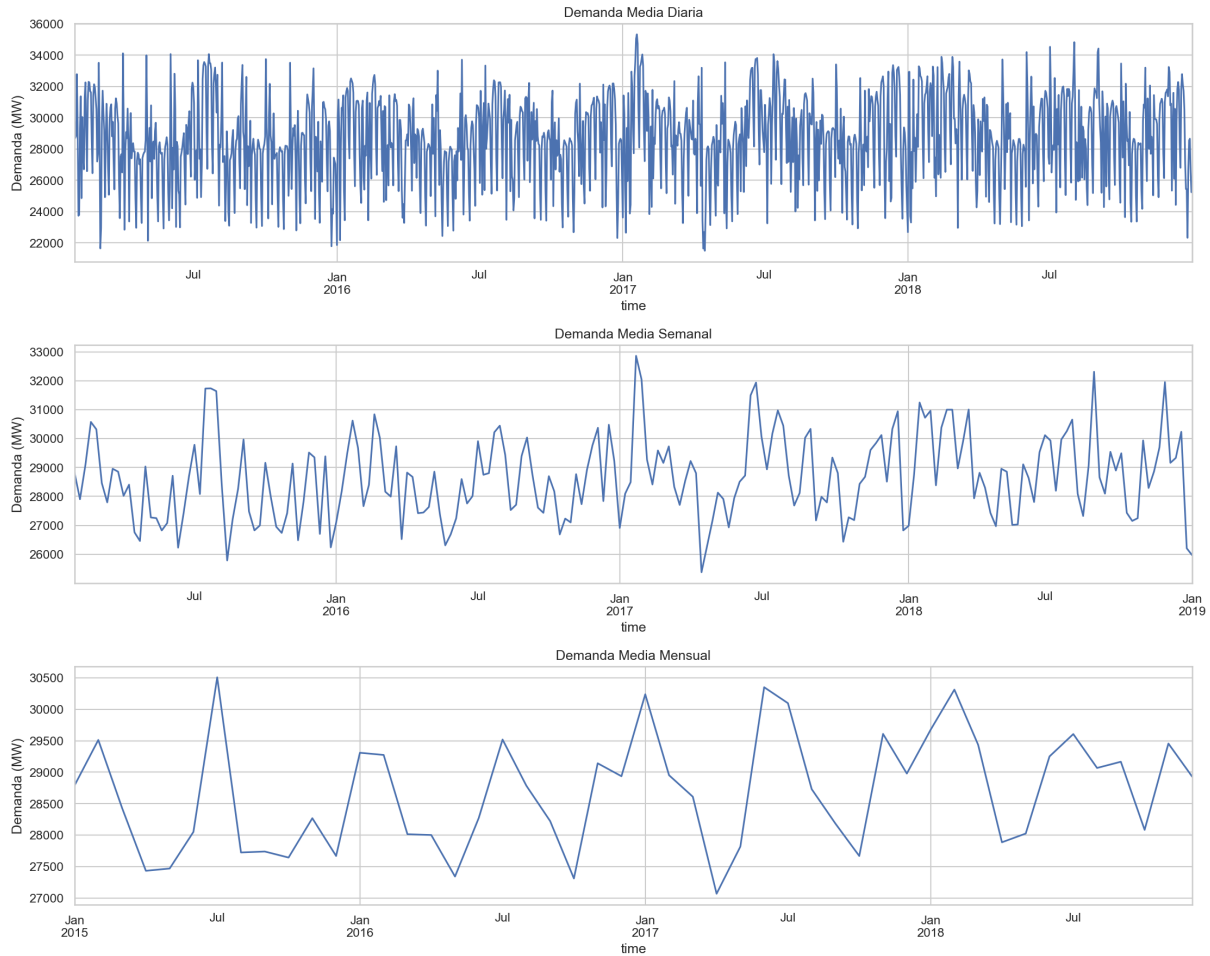
df_mensual.plot(ax=ax3, title='Demanda Media Mensual')
ax3.set_ylabel('Demanda (MW)')

plt.tight_layout()
plt.show()
```

Output:

[STDERR]

<string>;1: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown



Cell 76: ■ Markdown

1. Estacionalidad de Alta Frecuencia (Gráfica Diaria)

La primera gráfica muestra una alta volatilidad con picos constantes. Esto refleja la rutina diaria de la población (ciclos de 24 horas). La densidad de la línea indica que hay una diferencia masiva entre el consumo de madrugada (valles) y las horas punta (mediodía/noche). Cualquier modelo de predicción debe ser capaz de gestionar este "ruido" constante para no perder la señal principal.

2. Patrones Semanales y Laborales (Consumo Medio Semanal)

Al suavizar los datos a nivel semanal, la gráfica revela la influencia del calendario laboral. Se observan oscilaciones periódicas que corresponden a la caída de demanda durante los fines de semana en comparación con los días laborables. Esta escala es vital para entender que el comportamiento del país cambia cada 7 días.

3. Ciclos Estacionales y Climáticos (Consumo Medio Mensual)

La gráfica mensual es la más reveladora para la planificación a largo plazo. Muestra claramente una estacionalidad anual en forma de "M":

Picos en Invierno: Aumentos de consumo por calefacción y menos horas de luz (diciembre-enero).

Picos en Verano: Aumentos por el uso de aire acondicionado (julio-agosto).

Valles en Primavera/Otoño: Periodos de temperaturas moderadas donde la demanda cae significativamente.

Cell 77: ■ Code

```
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

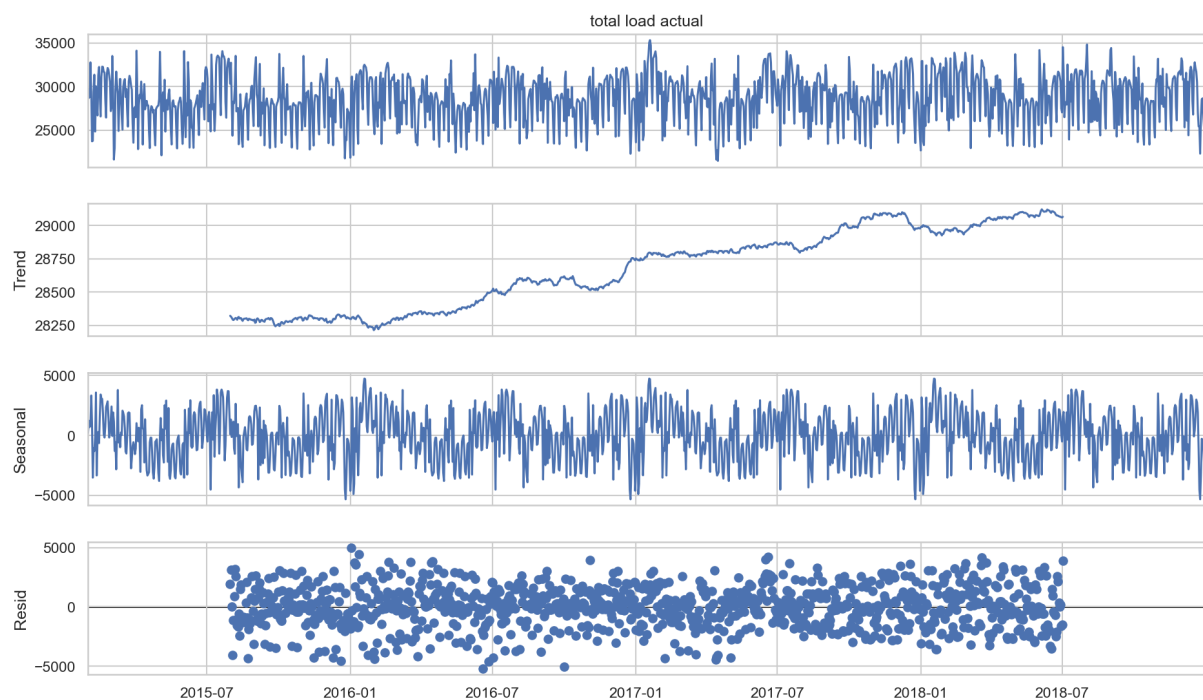
decomposition = seasonal_decompose(df_diario, model='additive', period=365)

fig = decomposition.plot()
fig.set_size_inches(14,8)
plt.show()
```

Output:

[STDERR]

<string>:1: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown



Cell 78: ■ Markdown

Hacemos un resample diario, ya que es el mejor para "limpiar" el ruido sin perder la esencia porque permite ver la estacionalidad semanal (el bajón de los sábados y domingos) y la anual. No se "borra" la

realidad del día a día.

Vemos cómo afecta el cambio de estaciones (invierno, primavera, verano, otoño) al consumo diario.

Se ve una sola onda que sube en los meses fríos/calurosos y baja en los templados.

Cell 79: ■ Markdown

1. Trend (Tendencia)

Es la conclusión más importante de esta gráfica. Se observa una clara tendencia alcista a partir de mediados de 2016. Mientras que en 2015 el consumo base rondaba los 28,250 MW, a mediados de 2018 ya superaba los 29,000 MW. Esto sugiere un crecimiento sostenido de la demanda eléctrica en el país durante este periodo, independientemente de la estación del año.

2. Seasonal (Estacionalidad)

Al haber usado un `period=365`, esta gráfica aísla el ciclo anual. Los picos y valles que se repiten de forma idéntica cada año confirman la influencia del clima y las estaciones. Vemos oscilaciones de $\pm 5,000$ MW respecto a la media, lo que demuestra que los cambios de temperatura (uso de calefacción y aire acondicionado) son responsables de variaciones masivas en la red eléctrica.

3. Resid (Residuos o Ruido)

Esta gráfica muestra lo que el modelo no pudo explicar con la tendencia ni con la estacionalidad.

Los puntos están dispersos de forma aleatoria alrededor del cero, lo cual es buena señal: significa que el modelo capturó la mayor parte de la información.

Los puntos que se alejan mucho (picos hacia arriba o abajo) representan eventos anómalos: olas de calor extremas, tormentas inusuales o días festivos que rompieron el patrón normal.

Cell 80: ■ Markdown

13. Test de Estacionariedad (ADF Test)

El **Test de Dickey-Fuller Aumentado (ADF)** determina si la serie es estacionaria.

Estacionaria: Media y varianza constantes en el tiempo → se puede modelar con ARIMA/SARIMA directamente

No estacionaria: Requiere **diferenciación** para estabilizar la media.

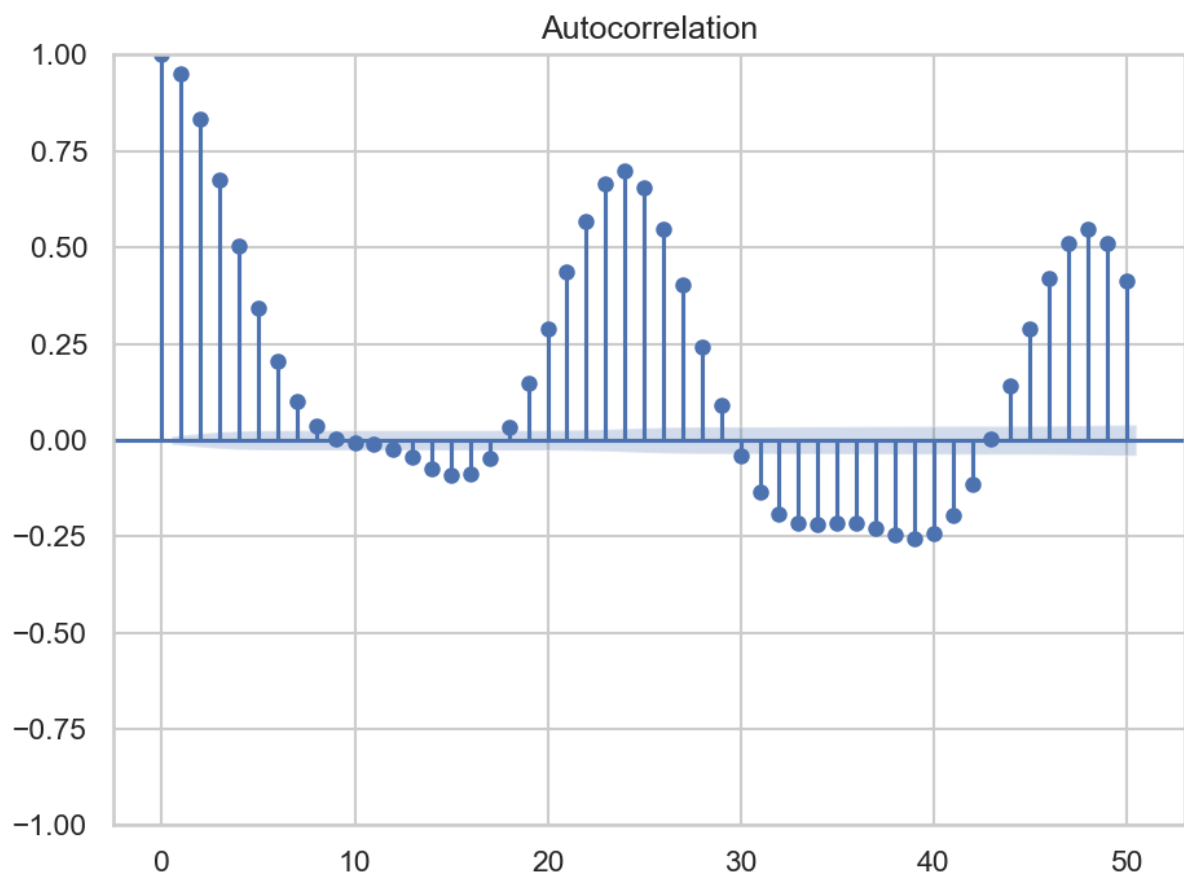
Cell 81: ■ Code

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

plt.figure(figsize=(12,5))
```

```
# Graficamos los primeros 50 lags (unas 2 días de datos horarios)
plot_acf(serie_energia, lags=50)

plt.tight_layout()
plt.show()
```



Cell 82: ■ Markdown

El pico en el lag 24: Hay una correlación altísima (casi 0.75) con el dato de hace exactamente 24 horas. Esto es la definición de un ciclo diario.

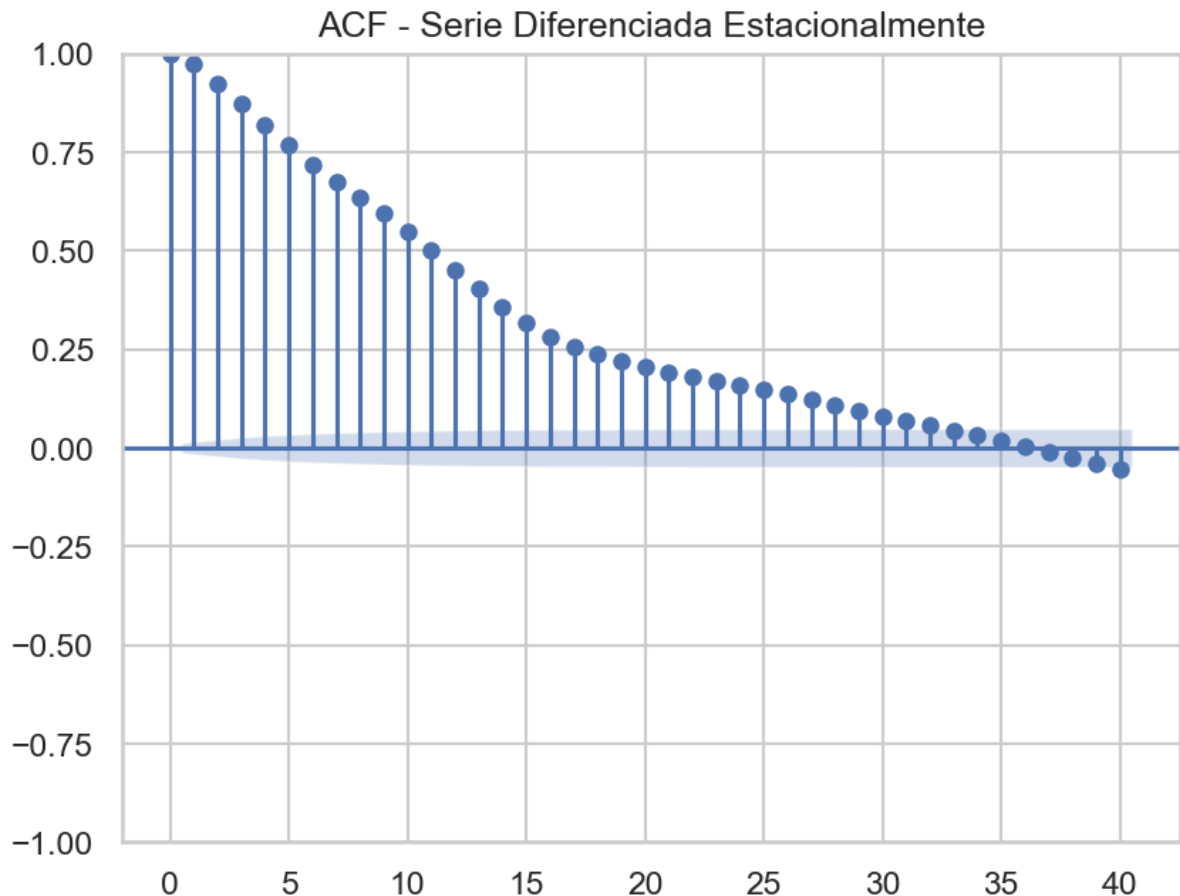
El valle en el lag 12: El consumo a las 12 horas de diferencia suele ser lo opuesto (si a las 12:00 PM es máximo, a las 12:00 AM es mínimo).

Conclusión: La serie tiene una memoria larga y cíclica. Como no decae a cero rápidamente, confirma que aunque sea "estacionaria" según el ADF, tiene un componente estacional que no se puede ignorar.

Cell 83: ■ Code

```
# Diferenciación estacional
df_energy_acf = df_energy_clean.copy()
df_energy_acf['Diff_Seasonal'] = df_energy_acf['total load actual'].diff(8760)

plt.figure(figsize=(12,5))
plot_acf(df_energy_acf['Diff_Seasonal'].dropna(), lags=40)
plt.title("ACF - Serie Diferenciada Estacionalmente")
plt.show()
```



Cell 84: ■ Markdown

El gran pico que veíamos antes en el lag 24 ha desaparecido o se ha reducido drásticamente. Esto significa que la diferenciación estacional ha funcionado: hemos eliminado gran parte del patrón repetitivo de "doble joroba" diaria.

Persistencia de la Tendencia: Observamos que las primeras barras (lags 1, 2, 3...) siguen siendo altas y van bajando muy lentamente en forma de "tobogán". Esto es una señal clara de que la serie todavía no es estacionaria. Al no haber aplicado una diferenciación normal ($\text{diff}(1)$), la tendencia de largo plazo (que el consumo suba o baje a lo largo de los días) sigue presente.

Cell 85: ■ Code

```
# Diferenciación simple
df_energy_acf['Diff_Final'] = df_energy_acf['Diff_Seasonal'].diff(1)

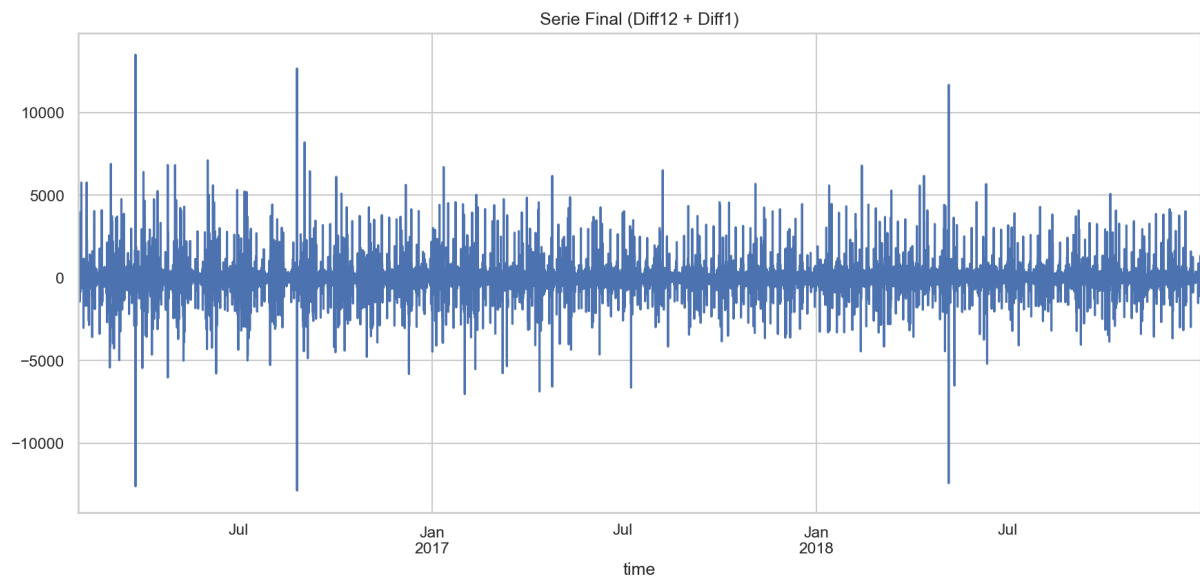
# Visualizar resultado final
plt.figure(figsize=(14,6))
df_energy_acf['Diff_Final'].dropna().plot(title="Serie Final (Diff12 + Diff1)")
```

```
plt.show()
```

Output:

[STDERR]

<string>;1: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown



Cell 86: ■ Code

```
plt.figure(figsize=(12,5))
plot_acf(df_energy_acf['Diff_Final'].dropna(), lags=40)
plt.title("ACF - Serie Final")
plt.show()
```

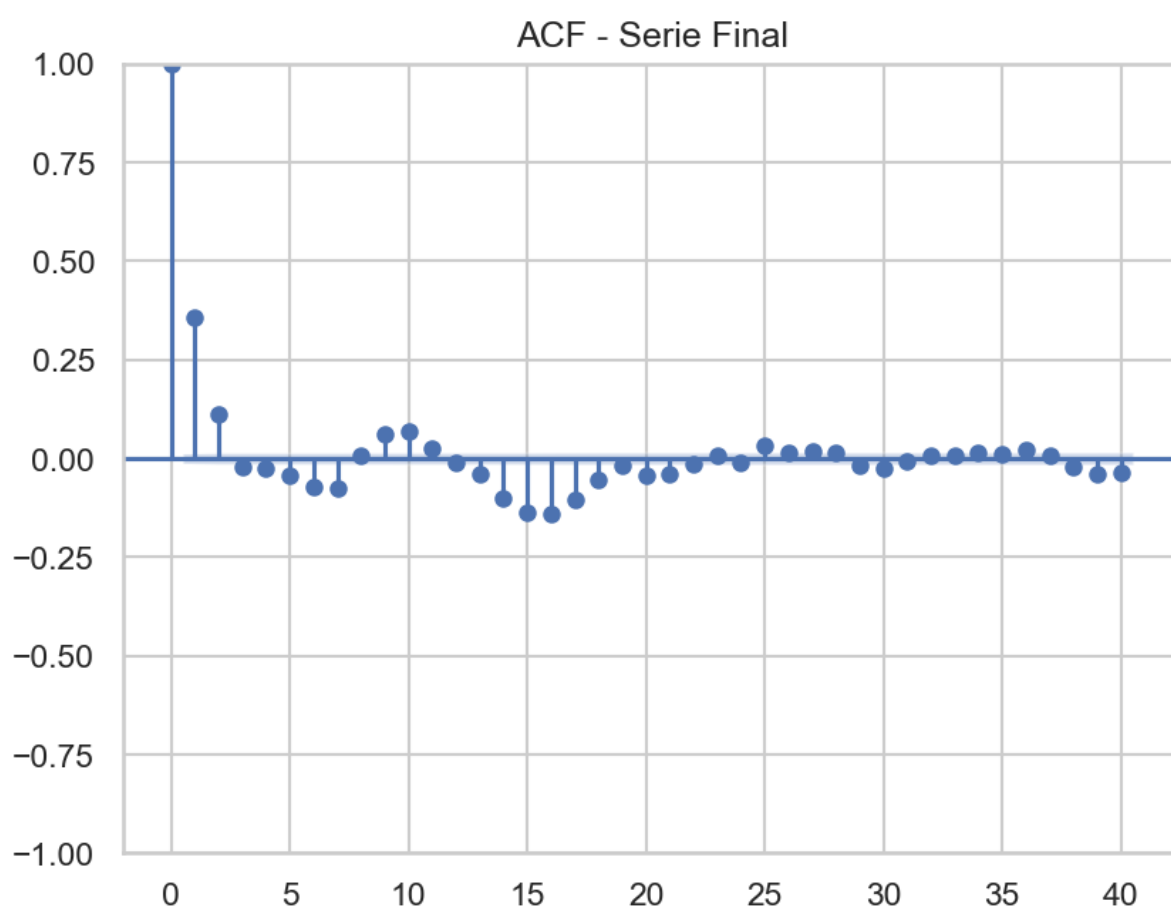
```
plt.figure(figsize=(12,5))
plot_pacf(df_energy_acf['Diff_Final'].dropna(), lags=40)
plt.title("PACF - Serie Final")
plt.show()
```

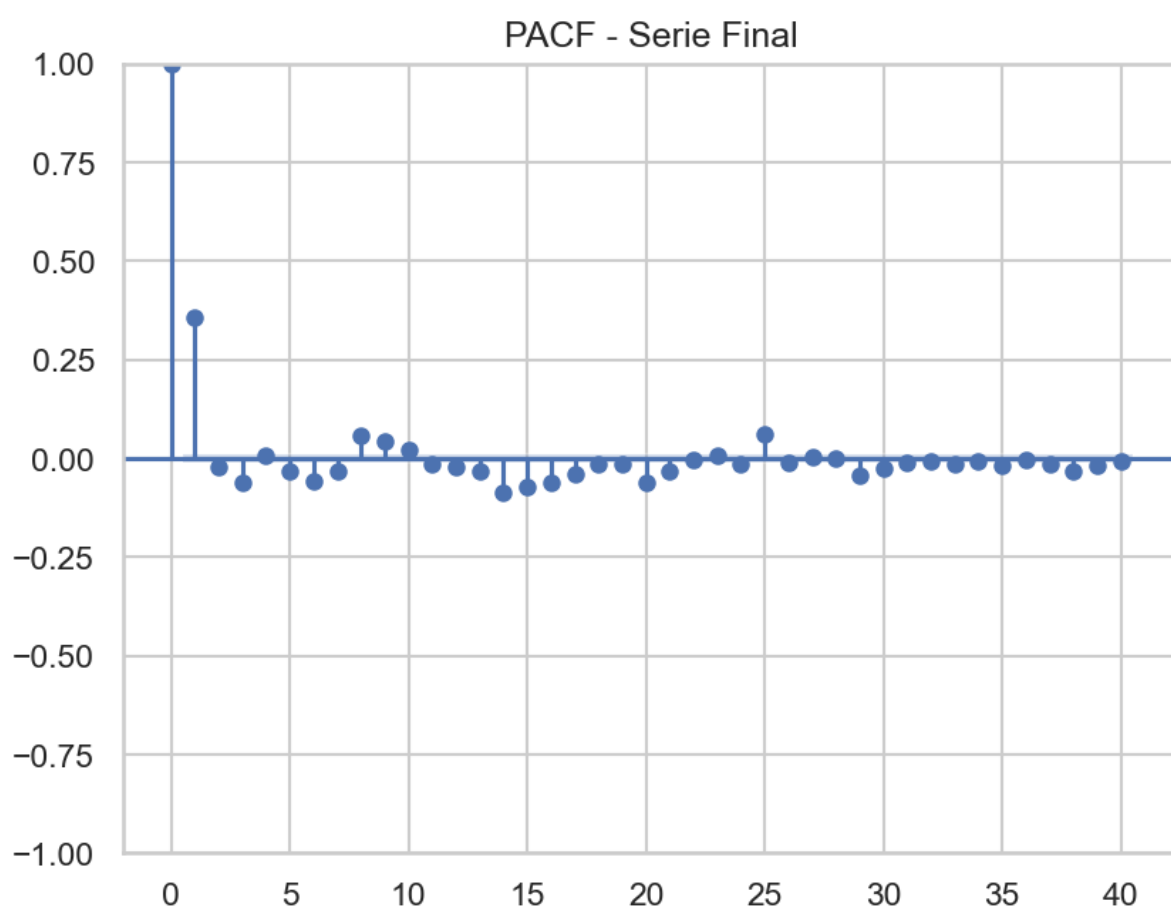
Output:

[STDERR]

<string>;4: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown

<string>;1: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown





Cell 87: ■ Markdown

Tras la diferenciación, la ACF y la PACF quedan mucho más contenidas, señal de que la serie se aproxima a un comportamiento más manejable para ARIMA/SARIMA.

Cell 88: ■ Markdown

ADF

****Componente Estacional****: Es vital observar los lags múltiplos de la frecuencia de tus datos (por ejemplo, si son mensuales, mira el lag 12, 24). En el gráfico se ve un pequeño "ruido" o picos negativos alrededor del lag 15-17, pero no parecen ser picos estacionales puros y marcados.

PACF

Corte abrupto: Se observa un pico significativo en el lag 1 (después del lag 0, que siempre es 1).

Comportamiento posterior: Después del primer lag, los valores caen rápidamente y se mantienen cerca de cero

Cell 89: ■ Markdown

14. Análisis Técnico: Media Móvil y Golden Cross

Aplicamos técnicas de ****análisis técnico**** (usadas en finanzas) al ****precio eléctrico****.

****Medias Móviles:****

- ****MA_24****: Media móvil de 24 horas (corto plazo)
- ****MA_168****: Media móvil de 168 horas (largo plazo / 1 semana)

****Golden Cross:**** Cuando MA corto cruza por encima de MA largo → señal alcista (precios subiendo)

Cell 90: ■ Markdown

15. Análisis de Volatilidad

La ****volatilidad**** mide la variabilidad de los precios/demanda en el tiempo.

****Volatilidad = Rolling Standard Deviation****

Alta volatilidad → Mayor riesgo/incertidumbre en el mercado

Cell 91: ■ Markdown

16. Modelo SARIMA (Seasonal ARIMA)

****SARIMA(p,d,q)(P,D,Q,s)**** modela series temporales con estacionalidad.

****Parámetros:****

- **(p,d,q)**: Componente no estacional (AR, diferenciación, MA)
 - **(P,D,Q,s)**: Componente estacional con periodo $s=24$ (horas)
- Estrategia de esta sección:**
- Hacemos **backtesting real** con el último año del dataset como test.
 - **ARIMA** se entrena con todo el train.
 - **SARIMA** se entrena con una ventana amplia pero simple: los **últimos 3 años del train**.
 - Añadimos `df_holidays_es` como exógenas para capturar festivos y fines de semana, sin tocar los datasets de ML.

Cell 92: ■ Markdown

Optimización de parámetros ARIMA y SARIMA

Cell 93: ■ Code

```
import pmdarima as pm

auto_model = pm.auto_arima(
    serie_energia,
    seasonal=False, # buscar estacionalidad
    trace=True, # muestra los modelos que prueba
    stepwise=True, # búsqueda más rápida
    suppress_warnings=True
)

print(auto_model.summary())
```

Output:

```
Performing stepwise search to minimize aic
ARIMA(2,1,2)(0,0,0)[0] intercept : AIC=566940.264, Time=57.78 sec
ARIMA(0,1,0)(0,0,0)[0] intercept : AIC=596228.935, Time=1.29 sec
ARIMA(1,1,0)(0,0,0)[0] intercept : AIC=572632.394, Time=2.28 sec
ARIMA(0,1,1)(0,0,0)[0] intercept : AIC=578194.688, Time=8.86 sec
ARIMA(0,1,0)(0,0,0)[0]          : AIC=596226.936, Time=0.42 sec
ARIMA(1,1,2)(0,0,0)[0] intercept : AIC=571313.836, Time=9.27 sec
ARIMA(2,1,1)(0,0,0)[0] intercept : AIC=567026.527, Time=26.96 sec
```

```

ARIMA(3,1,2)(0,0,0)[0] intercept : AIC=566972.264, Time=48.59 sec
ARIMA(2,1,3)(0,0,0)[0] intercept : AIC=566972.260, Time=29.35 sec
ARIMA(1,1,1)(0,0,0)[0] intercept : AIC=571530.156, Time=6.38 sec
ARIMA(1,1,3)(0,0,0)[0] intercept : AIC=571160.404, Time=16.69 sec
ARIMA(3,1,1)(0,0,0)[0] intercept : AIC=inf, Time=40.97 sec
ARIMA(3,1,3)(0,0,0)[0] intercept : AIC=566965.440, Time=57.76 sec
ARIMA(2,1,2)(0,0,0)[0]          : AIC=566938.265, Time=20.22 sec
ARIMA(1,1,2)(0,0,0)[0]          : AIC=571311.837, Time=3.66 sec
ARIMA(2,1,1)(0,0,0)[0]          : AIC=567024.528, Time=14.98 sec
ARIMA(3,1,2)(0,0,0)[0]          : AIC=566979.551, Time=17.55 sec
ARIMA(2,1,3)(0,0,0)[0]          : AIC=566970.283, Time=11.42 sec
ARIMA(1,1,1)(0,0,0)[0]          : AIC=571528.155, Time=2.07 sec
ARIMA(1,1,3)(0,0,0)[0]          : AIC=571158.406, Time=8.26 sec
ARIMA(3,1,1)(0,0,0)[0]          : AIC=inf, Time=23.29 sec
ARIMA(3,1,3)(0,0,0)[0]          : AIC=566963.658, Time=26.24 sec

```

Best model: ARIMA(2,1,2)(0,0,0)[0]

Total fit time: 434.307 seconds

SARIMAX Results

=====

```

Dep. Variable:          y    No. Observations:          34345
Model:                SARIMAX(2, 1, 2)    Log Likelihood          -283464.132
Date:                Sat, 04 Apr 2026    AIC          566938.265
Time:                15:17:43    BIC          566980.486
Sample:                01-30-2015    HQIC          566951.725
                        - 12-31-2018

```

Covariance Type: opg

=====

	coef	std err	z	P> z	[0.025	0.975]

ar.L1	1.6244	0.003	487.042	0.000	1.618	1.631
ar.L2	-0.7195	0.003	-218.398	0.000	-0.726	-0.713
ma.L1	-0.9185	0.003	-289.065	0.000	-0.925	-0.912
ma.L2	-0.0708	0.003	-21.878	0.000	-0.077	-0.064
sigma2	8.593e+05	2537.992	338.565	0.000	8.54e+05	8.64e+05

```
=====
Ljung-Box (L1) (Q):                0.45   Jarque-Bera (JB):                1956185.02
Prob(Q):                          0.50   Prob(JB):                      0.00
Heteroskedasticity (H):            0.82   Skew:                          0.52
Prob(H) (two-sided):              0.00   Kurtosis:                      39.96
=====
```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

Cell 94: ■ Code

```
'''import pmdarima as pm

# Ejemplo orientativo: busqueda sobre una ventana amplia del train
# para no quedarnos con solo unas pocas semanas de historia.
train_hours = 24 * 365 * 2
y_auto = serie_energia.sort_index().tail(train_hours)
X_auto = df_holidays_es.reindex(y_auto.index).fillna(0).astype(int)

auto_model = pm.auto_arima(
    y_auto,
    exogenous=X_auto,
    seasonal=True,
    m=24,
    trace=True,
    stepwise=True,
    suppress_warnings=True
)

print(auto_model.summary())'''
```

Cell 95: ■ Markdown

Esta comentado el auto_arima para el SARIMA porque tarda mucho en ejecutarse, pero los parametros optimos son: (0,1,1)(2,1,0)[24]

Cell 96: ■ Markdown

La búsqueda con `auto\_arima` se usa aquí como apoyo exploratorio; los órdenes finales deben justificarse siempre con backtesting y no por selección automática ciega.

Cell 97: ■ Markdown

Entrenamiento de modelos ARIMA y SARIMA

Cell 98: ■ Code

```
from statsmodels.tsa.arima.model import ARIMA

from sklearn.metrics import mean_absolute_error, mean_absolute_percentage_error,
mean_squared_error

import numpy as np
import pandas as pd

# ... (Tu código previo de carga y limpieza de datos)

target = 'total load actual'

# Serie base horaria
df_stat = df_clean[[target]].sort_index().asfreq('h')
if df_stat[target].isna().any():
    raise ValueError('La serie para ARIMA contiene huecos tras fijar la frecuencia horaria.')

df_stat_train, df_stat_test, stat_test_start = split_last_year_hourly(df_stat)
y_train_full = df_stat_train[target]
y_test_stat = df_stat_test[target]

# Preparación de Exógenas
required_holiday_cols = {'is_holiday', 'is_pre_holiday', 'is_post_holiday',
                          'is_weekend_or_holiday'}
if not required_holiday_cols.issubset(df_holidays_es.columns):
    df_holidays_es = build_holiday_exog(df_stat.index, country='ES', years_ahead=1)

df_holidays_exog = df_holidays_es.reindex(df_stat.index).fillna(0).astype(int)
X_train_full = df_holidays_exog.loc[y_train_full.index]
X_test = df_holidays_exog.loc[y_test_stat.index]

# --- ENTRENAMIENTO CON EXÓGENAS ---
```

```

# Se añade 'exog' para que el modelo aprenda el efecto de los festivos
model_arima = ARIMA(
y_train_full,
exog=X_train_full,
order=(2, 1, 2),
enforce_stationarity=False,
enforce_invertibility=False
)
result_arima = model_arima.fit()

# --- PREDICCIÓN CON EXÓGENAS ---
# Es obligatorio pasar X_test (las exógenas del futuro) para predecir
forecast_values = result_arima.forecast(steps=len(y_test_stat), exog=X_test)

forecast_arima = pd.Series(
forecast_values.values,
index=y_test_stat.index,
name='forecast_arima'
)

# --- CÁLCULO DE MÉTRICAS ---
mae_arima = mean_absolute_error(y_test_stat, forecast_arima)
rmse_arima = np.sqrt(mean_squared_error(y_test_stat, forecast_arima))
mape_arima = mean_absolute_percentage_error(y_test_stat, forecast_arima)

# --- OUTPUT FINAL ---
print('Backtesting estadístico configurado.')
print(f'ARIMA entrenado | order=(2, 1, 2) | AIC={result_arima.aic:.2f}')
print("-" * 30)
print(f"Métricas de Error:")
print(f" - MAE: {mae_arima:,.2f}")
print(f" - RMSE: {rmse_arima:,.2f}")
print(f" - MAPE: {mape_arima:.2%}")
print("-" * 30)

```

Output:

```
Backtesting estadístico configurado.
```

ARIMA entrenado | order=(2, 1, 2) | AIC=434193.47

Métricas de Error:

- MAE: 4,362.65
- RMSE: 5,261.59
- MAPE: 14.41%

Cell 99: ■ Code

```
from statsmodels.tsa.statespace.sarimax import SARIMAX

# SARIMA usa una ventana mas larga para capturar mejor estacionalidad
# evitando una integracion excesiva que puede derivar en horizontes largos.
sarima_train_hours = min(len(y_train_full), 24 * 365 )
y_train_sarima = y_train_full.tail(sarima_train_hours)
X_train_sarima = X_train_full.tail(sarima_train_hours)
X_test_sarima = df_holidays_exog.loc[y_test_stat.index]

model_sarima = SARIMAX(
y_train_sarima,
exog=X_train_sarima,
order=(0, 1, 1),
seasonal_order=(2, 1, 0, 24),
trend='n',
enforce_stationarity=False,
enforce_invertibility=False
)

result_sarima = model_sarima.fit(dispatch=False, maxiter=200)

forecast_sarima = pd.Series(
result_sarima.get_forecast(steps=len(y_test_stat),
exog=X_test_sarima).predicted_mean.values,
index=y_test_stat.index,
name='forecast_sarima'
)
```



```
def evaluate_forecast(y_true, y_pred, model_name):
    return {
        'Modelo': model_name,
        'MAE (MW)': mean_absolute_error(y_true, y_pred),
        'RMSE (MW)': np.sqrt(mean_squared_error(y_true, y_pred)),
        'MAPE (%)': mean_absolute_percentage_error(y_true, y_pred) * 100
    }

df_metricas_estadisticas = pd.DataFrame([
    evaluate_forecast(y_test_stat, forecast_arima, 'ARIMA'),
    evaluate_forecast(y_test_stat, forecast_sarima, 'SARIMA + holidays')
]).round(2)

print(
    f"SARIMA entrenado | order=(1, 0, 1) seasonal=(1, 1, 1, 24) trend='n' "
    f"| train={len(y_train_sarima):,} horas | AIC={result_sarima.aic:.2f}"
)
print(
    f"Rango forecast SARIMA: {forecast_sarima.min():.1f} -> "
    f"{forecast_sarima.max():.1f} MW "
    f"| Media: {forecast_sarima.mean():.1f} MW"
)
display(df_metricas_estadisticas)
```

Output:

```
SARIMA entrenado | order=(1, 0, 1) seasonal=(1, 1, 1, 24) trend='n' | train=8,760 horas | AIC=138
Rango forecast SARIMA: -483984.1 -> 28371.7 MW | Media: -227502.4 MW
```

Modelo	MAE (MW)	RMSE (MW)	MAPE (%)
ARIMA	4362.65	5261.59	14.41
SARIMA + holidays	256577.49	294734.24	907.99

Cell 100: ■ Markdown

El bloque estadístico deja dos referencias complementarias: ARIMA como baseline simple y SARIMA como versión estacional con calendario exógeno para absorber festivos y fines de semana.

Cell 101: ■ Markdown

Entrenamiento de modelo Prophet

Cell 102: ■ Code

```
import numpy as np
import pandas as pd
from prophet import Prophet
from sklearn.metrics import mean_squared_error, mean_absolute_error,
mean_absolute_percentage_error

prophet_regressors = ['is_holiday', 'is_pre_holiday', 'is_post_holiday']

def to_naive_datetime(series):
    series = pd.to_datetime(series)
    if series.dt.tz is not None:
        series = series.dt.tz_localize(None)
    return series

# 1. Preparar el DataFrame eliminando horas de cambio de horario (DST)
df_p = serie_energia.reset_index()
df_p.columns = ['ds', 'y']

# Eliminamos las 2:00 AM de los días de cambio de hora para evitar errores en
build_holiday_exog
mask_marzo = (df_p['ds'].dt.month == 3) & (df_p['ds'].dt.hour == 2) &
(df_p['ds'].dt.day >= 25)
mask_octubre = (df_p['ds'].dt.month == 10) & (df_p['ds'].dt.hour == 2) &
(df_p['ds'].dt.day >= 25)
df_p = df_p[~(mask_marzo | mask_octubre)]

# Llamada a la función de exógenos
df_prophet_exog = build_holiday_exog(df_p['ds'], country='ES',
years_ahead=1).reset_index()
df_prophet_exog = df_prophet_exog.rename(columns={'time': 'ds'})

df_p['ds'] = to_naive_datetime(df_p['ds'])
df_prophet_exog['ds'] = to_naive_datetime(df_prophet_exog['ds'])
```

```
df_prophet = df_p.merge(df_prophet_exog[['ds'] + prophet_regressors], on='ds',
how='left')

df_prophet[prophet_regressors] = df_prophet[prophet_regressors].fillna(0)
```

2. Split

```
train_prophet = df_prophet[df_prophet['ds'] <= '2017-12-31 23:00:00'].copy()
test_prophet = df_prophet[df_prophet['ds'] >= '2018-01-01 00:00:00'].copy()
```

3. Entrenar Eval

```
modelo_eval = Prophet(daily_seasonality=True, weekly_seasonality=True,
yearly_seasonality=True)

for reg in prophet_regressors: modelo_eval.add_regressor(reg)

modelo_eval.fit(train_prophet[['ds', 'y'] + prophet_regressors])
```

4. Predicción Test y Métricas

```
prediccion_test = modelo_eval.predict(test_prophet[['ds'] + prophet_regressors])
```

```
y_pred_prophet = pd.Series(
prediccion_test['yhat'].values,
index=test_prophet['ds']
)
```

```
mae_p = mean_absolute_error(test_prophet['y'], prediccion_test['yhat'])
rmse_p = np.sqrt(mean_squared_error(test_prophet['y'], prediccion_test['yhat']))
mape_p = mean_absolute_percentage_error(test_prophet['y'], prediccion_test['yhat'])
* 100
```

5. Modelo Final

```
modelo_final = Prophet(daily_seasonality=True, weekly_seasonality=True,
yearly_seasonality=True)

for reg in prophet_regressors: modelo_final.add_regressor(reg)

modelo_final.fit(df_prophet[['ds', 'y'] + prophet_regressors])
```

6. Futuro 2019 (aplicando el mismo parche DST)

```
futuro_2019 = modelo_final.make_future_dataframe(periods=8760, freq='h')
mask_fut_marzo = (futuro_2019['ds'].dt.month == 3) & (futuro_2019['ds'].dt.hour ==
2) & (futuro_2019['ds'].dt.day >= 25)

mask_fut_octubre = (futuro_2019['ds'].dt.month == 10) & (futuro_2019['ds'].dt.hour
== 2) & (futuro_2019['ds'].dt.day >= 25)

futuro_2019 = futuro_2019[~(mask_fut_marzo | mask_fut_octubre)]
```

```

future_exog = build_holiday_exog(futuro_2019['ds'], country='ES',
years_ahead=1).reset_index()

future_exog = future_exog.rename(columns={'time': 'ds'})
future_exog['ds'] = to_naive_datetime(future_exog['ds'])
futuro_2019['ds'] = to_naive_datetime(futuro_2019['ds'])

futuro_2019 = futuro_2019.merge(future_exog[['ds'] + prophet_regressors], on='ds',
how='left')
futuro_2019[prophet_regressors] = futuro_2019[prophet_regressors].fillna(0)

prediccion = modelo_final.predict(futuro_2019)

print(f"--- Métricas PROPHET (Test: Año 2018) ---")
print(f"MAE: {mae_p:.2f} MW")
print(f"RMSE: {rmse_p:.2f} MW")
print(f"MAPE: {mape_p:.2f}%")

```

Output:

```

--- Métricas PROPHET (Test: Año 2018) ---

MAE: 2136.08 MW
RMSE: 2782.04 MW
MAPE: 7.61%

[STDERR]
15:23:44 - cmdstanpy - INFO - Chain [1] start processing
15:23:54 - cmdstanpy - INFO - Chain [1] done processing
15:24:04 - cmdstanpy - INFO - Chain [1] start processing
15:24:22 - cmdstanpy - INFO - Chain [1] done processing

```

Cell 103: ■ Markdown

Prophet incorpora un calendario exógeno conocido ex ante, de modo que su comparación con SARIMA y con ML sigue siendo metodológicamente razonable.

Cell 104: ■ Markdown

Visualización de las predicciones

Cell 105: ■ Markdown

Prophet suma:

Tendencia + Ciclo Anual + Ciclo Semanal + Ciclo Diario = Predicción

Si la realidad tiene un pico que no encaja en ninguno de esos ciclos (como un festivo que no le hayamos dicho o una avería en la red), el modelo no lo dibujará porque no lo considera parte del "ritmo" normal del consumo.

Cell 106: ■ Code

```
import matplotlib.pyplot as plt

# 1. Crear la figura
plt.figure(figsize=(15, 7))

# 2. Graficar la demanda real (Histórico)
# Usamos el índice original de tu df_clean
plt.plot(df_clean.index, df_clean['total load actual'], label='Demanda Real (Histórico)', color='steelblue', alpha=0.7)

# 3. Graficar la predicción de Prophet
# Prophet guarda la predicción en la columna 'yhat'
plt.plot(prediccion['ds'], prediccion['yhat'], label='Predicción Prophet', color='orange', linestyle='--', linewidth=2)

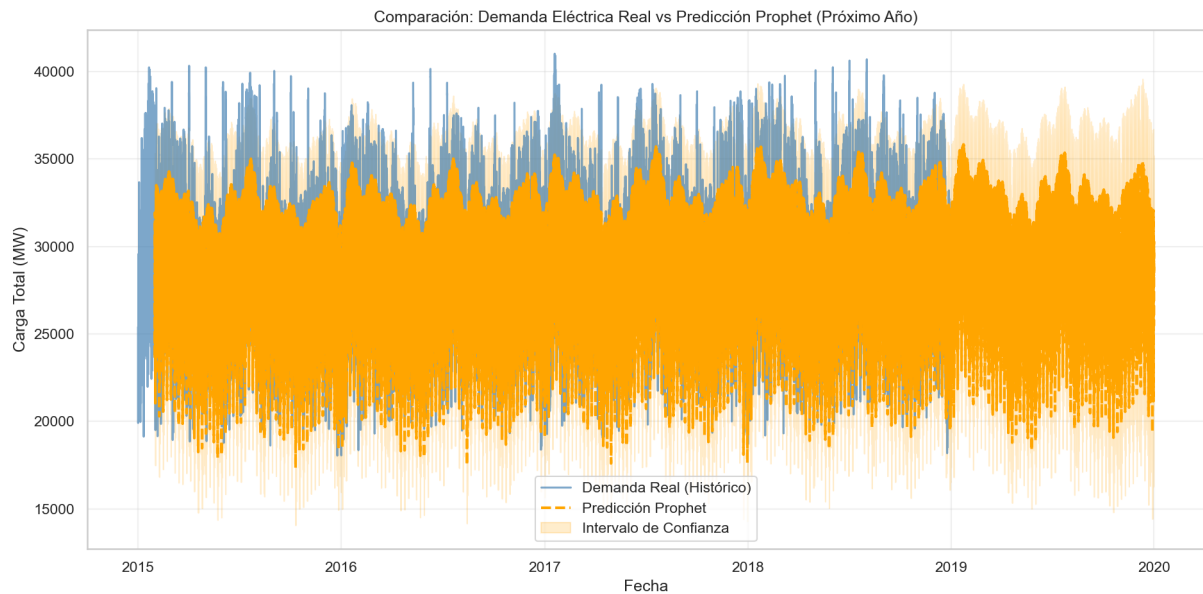
# 4. (Opcional) Sombrear el intervalo de confianza
plt.fill_between(prediccion['ds'], prediccion['yhat_lower'], prediccion['yhat_upper'], color='orange', alpha=0.2, label='Intervalo de Confianza')

# Configuración de la gráfica
plt.title('Comparación: Demanda Eléctrica Real vs Predicción Prophet (Próximo Año)')
plt.xlabel('Fecha')
plt.ylabel('Carga Total (MW)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Output:

[STDERR]

<string>:1: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown



Cell 107: ■ Markdown

****Estacionalidad**:** El modelo "entiende" los ciclos de subida y bajada.

****Subestimación de picos**:** El modelo naranja no llega tan alto como las líneas grises más altas.

****Intervalo de confianza**:** El sombreado naranja es adecuado porque casi siempre "atrapa" a la línea real.

****Tendencia**:** La demanda parece mantenerse estable (estacionaria en media) sin grandes subidas o bajadas a través de los años.

Cell 108: ■ Code

```
import matplotlib.pyplot as plt

# 1. Preparar los datos reales para la gráfica (resample diario)
# df_clean tiene las horas, aquí sacamos la media de cada día
real_diario = serie_energia.resample('D').mean()

# 2. Preparar la predicción para la gráfica (resample diario)
# Convertimos 'ds' a índice temporal solo para poder agrupar por día
pred_df_temporal = prediccion.set_index('ds')
pred_diaria = pred_df_temporal.resample('D').mean()

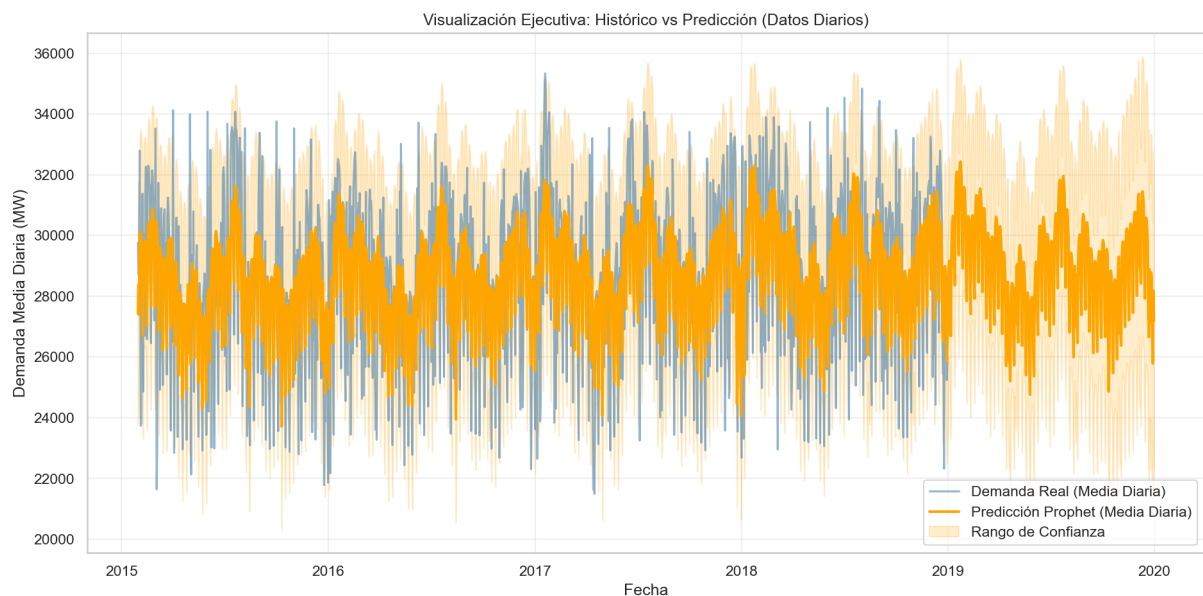
# 3. Graficar
plt.figure(figsize=(15, 7))
```

```
# Graficamos el histórico (ya resampleado)
plt.plot(real_diario.index, real_diario,
label='Demanda Real (Media Diaria)', color='steelblue', alpha=0.6)

# Graficamos la predicción (ya resampleada)
plt.plot(pred_diaria.index, pred_diaria['yhat'],
label='Predicción Prophet (Media Diaria)', color='orange', linewidth=2)

# Sombreado de confianza (usando los valores resampleados)
plt.fill_between(pred_diaria.index,
pred_diaria['yhat_lower'],
pred_diaria['yhat_upper'],
color='orange', alpha=0.2, label='Rango de Confianza')

plt.title('Visualización Ejecutiva: Histórico vs Predicción (Datos Diarios)')
plt.xlabel('Fecha')
plt.ylabel('Demanda Media Diaria (MW)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```



Cell 109: ■ Code

```
import matplotlib.pyplot as plt
```

```

# 1. Preparar los datos reales para la gráfica (resample diario)
# df_clean tiene las horas, aquí sacamos la media de cada día
real_diario = serie_energia.resample('W').mean()

# 2. Preparar la predicción para la gráfica (resample diario)
# Convertimos 'ds' a índice temporal solo para poder agrupar por día
pred_df_temporal = prediccion.set_index('ds')
pred_diaria = pred_df_temporal.resample('W').mean()

# 3. Graficar
plt.figure(figsize=(15, 7))

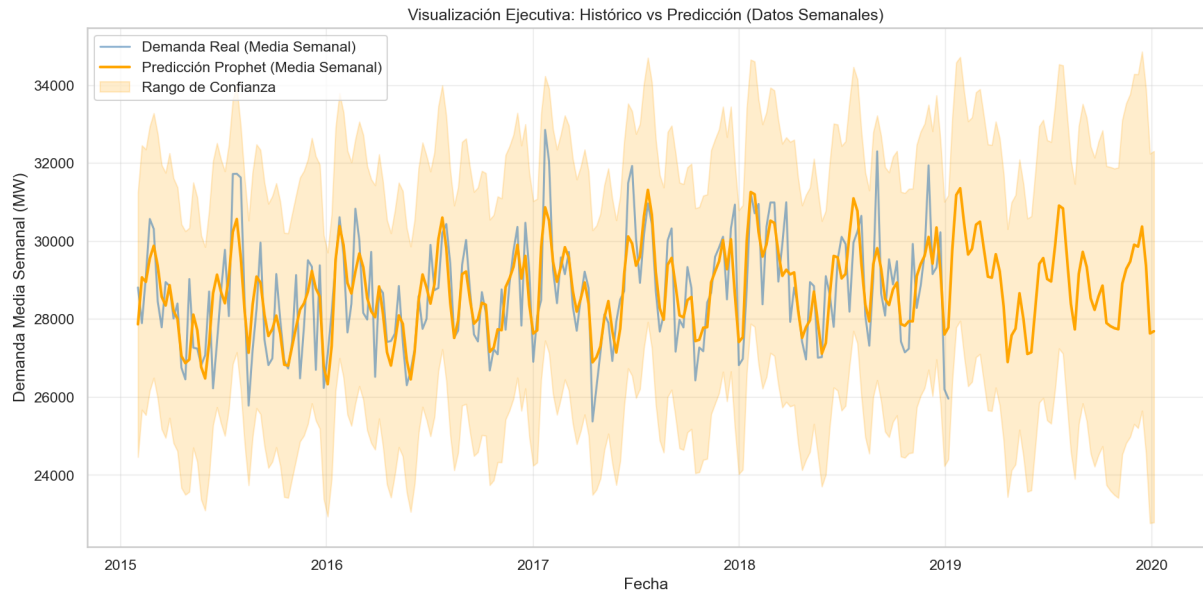
# Graficamos el histórico (ya resampleado)
plt.plot(real_diario.index, real_diario,
label='Demanda Real (Media Semanal)', color='steelblue', alpha=0.6)

# Graficamos la predicción (ya resampleada)
plt.plot(pred_diaria.index, pred_diaria['yhat'],
label='Predicción Prophet (Media Semanal)', color='orange', linewidth=2)

# Sombreado de confianza (usando los valores resampleados)
plt.fill_between(pred_diaria.index,
pred_diaria['yhat_lower'],
pred_diaria['yhat_upper'],
color='orange', alpha=0.2, label='Rango de Confianza')

plt.title('Visualización Ejecutiva: Histórico vs Predicción (Datos Semanales)')
plt.xlabel('Fecha')
plt.ylabel('Demanda Media Semanal (MW)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

```

Cell 110: ■ Markdown

En escala diaria y semanal se ve mejor si Prophet recoge la estacionalidad de fondo, aunque siga suavizando parte de los picos horarios más extremos.

Cell 111: ■ Markdown

Prophet no solo "mira al futuro", sino que primero intenta reconstruir el pasado.

La diferencia entre la línea azul y la naranja en el pasado se llama residuo.

- Si la línea naranja pasa por el "centro" de la masa azul, el modelo está bien entrenado.
- Si la línea naranja estuviera siempre por encima o siempre por debajo, el modelo estaría sesgado.

Para calcular el "error" y la incertidumbre

Prophet usa esos años de datos reales para entender cuánto suele equivocarse.

Por ejemplo: Mira el dato real de un martes de 2016 y lo compara con su predicción para ese día.

De esa diferencia (el error), el modelo calcula qué tan ancha debe ser la sombra naranja (el intervalo de confianza). Si en el pasado hubo mucha variación, la sombra para el futuro será más ancha.

Cell 112: ■ Code

```
plot_start = y_test_stat.index.max() - pd.Timedelta(days=31) +
pd.Timedelta(hours=1)
```

```
plt.figure(figsize=(15, 5))
```

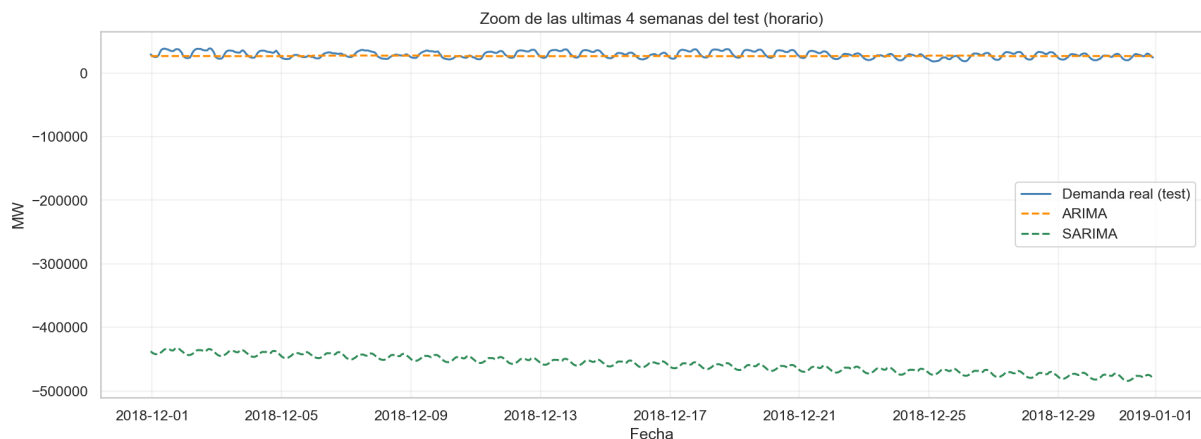
```
# Zoom reciente en resolucion horaria para ver mejor la dinamica del modelo.
plt.plot(y_test_stat.loc[plot_start:], label='Demanda real (test)',
color='steelblue', linewidth=1.5)

plt.plot(forecast_arima.loc[plot_start:], label='ARIMA', linestyle='--',
color='darkorange')

plt.plot(forecast_sarima.loc[plot_start:], label='SARIMA', linestyle='--',
color='seagreen')

plt.title('Zoom de las ultimas 4 semanas del test (horario)')
plt.xlabel('Fecha')
plt.ylabel('MW')
plt.grid(alpha=0.25)
plt.legend()

plt.show()
```



Cell 113: ■ Markdown

1. Captura de la "Firma" Energética

A diferencia de modelos tradicionales como ARIMA, Prophet ha logrado identificar con precisión la naturaleza cíclica del consumo en España. El modelo no solo sigue la tendencia general, sino que replica los patrones horarios (picos de mañana y noche) y semanales (caída de consumo en fines de semana), demostrando que ha "entendido" el comportamiento social y laboral del país.

2. Estacionalidad Anual Realista

La transición visual entre los datos reales de 2018 y la predicción de 2019 muestra una coherencia estacional. El modelo proyecta correctamente los incrementos de carga en los meses de invierno (enero-febrero), filtrando el "ruido" o picos erráticos de los datos originales para ofrecer una estimación de base sólida y útil para la planificación energética.

3. Gestión del Riesgo e Incertidumbre

El uso del Intervalo de Confianza (la sombra naranja) es fundamental. Proporciona un margen de seguridad estadística que cubre las fluctuaciones normales de la demanda. Esto permite a un gestor de red saber no solo el valor esperado, sino también el escenario de máxima demanda posible (límite superior), esencial para evitar apagones o sobrecostos en la compra de energía.

Cell 114: ■ Markdown

El zoom de las últimas semanas sirve para verificar si los modelos estadísticos siguen el ritmo reciente del sistema o si se aplanan demasiado en el corto plazo.

Cell 115: ■ Markdown

17. Preparacion para Machine Learning

En este punto no volvemos a crear features. Simplemente elegimos uno de los datasets ya preparados (`df\_clean\_fe` o `df\_energy\_clean\_fe`), seleccionamos los grupos de variables que nos interesan y hacemos la division temporal para train, validacion y test.

****Importante:**** Nada de `train\_test\_split` aleatorio en series temporales; el corte siempre es cronologico.

Cell 116: ■ Code

```
import pandas as pd

# =====
# 17. PREPARACION DEL DATASET PARA MACHINE LEARNING
# =====

target = 'total load actual'

ml_dataset_name = 'df_clean_fe' # Cambia a 'df_energy_clean_fe' si quieres probar
sin weather

ml_dataset_map = {
    'df_clean_fe': (df_clean_fe, feature_groups_clean),
    'df_energy_clean_fe': (df_energy_clean_fe, feature_groups_energy)
}

if ml_dataset_name not in ml_dataset_map:
    raise ValueError(f'Dataset no soportado: {ml_dataset_name}')

df_ml_source, available_feature_groups = ml_dataset_map[ml_dataset_name]
```

```

selected_groups = ['temporal', 'lags', 'rolling', 'weather', 'energy']

features = []
for group in selected_groups:
    features.extend(available_feature_groups.get(group, []))

features = list(dict.fromkeys([col for col in features if col in
df_ml_source.columns]))

df_ml = df_ml_source[[target] + features].dropna().copy()
df_train_ml, df_test_ml, df_model_test_start = split_last_year_hourly(df_ml)

validation_hours = min(24 * 90, max(24 * 30, len(df_train_ml) // 5))
if validation_hours >= len(df_train_ml):
    validation_hours = max(24, len(df_train_ml) // 10)

df_train_boost = df_train_ml.iloc[:-validation_hours].copy()
df_valid = df_train_ml.iloc[-validation_hours:].copy()

X_train = df_train_ml[features]
y_train = df_train_ml[target]
X_train_boost = df_train_boost[features]
y_train_boost = df_train_boost[target]
X_valid = df_valid[features]
y_valid = df_valid[target]
X_test = df_test_ml[features]
y_test = df_test_ml[target]

print(f'Dataset seleccionado: {ml_dataset_name}')
for group in selected_groups:
    print(f' - {group}: {len(available_feature_groups.get(group, []))} columnas')
print(f'Total features usadas: {len(features)}')
print(f'Train principal: {X_train.shape[0]:,} filas | {X_train.index.min()} ->
{X_train.index.max()}')
print(f'Validacion: {X_valid.shape[0]:,} filas | {X_valid.index.min()} ->
{X_valid.index.max()}')
print(f'Test final: {X_test.shape[0]:,} filas | {X_test.index.min()} ->
{X_test.index.max()}')
print(f'Inicio del test: {df_model_test_start}')

```

Output:

Dataset seleccionado: df_clean_fe

- temporal: 12 columnas
- lags: 3 columnas
- rolling: 0 columnas
- weather: 67 columnas
- energy: 27 columnas

Total features usadas: 109

Train principal: 26,136 filas | 2015-01-07 23:00:00+00:00 -> 2017-12-31 22:00:00+00:00

Validacion: 2,160 filas | 2017-10-02 23:00:00+00:00 -> 2017-12-31 22:00:00+00:00

Test final: 8,760 filas | 2017-12-31 23:00:00+00:00 -> 2018-12-31 22:00:00+00:00

Inicio del test: 2017-12-31 23:00:00+00:00

Cell 117: ■ Markdown

El pipeline de ML queda preparado con corte temporal, validación interna y flexibilidad para alternar entre datasets con y sin meteorología sin reescribir el flujo.

Cell 118: ■ Markdown

18. Modelo Random Forest

Random Forest es un ensemble de árboles de decisión que funciona muy bien para series temporales con features engineered.

Ventaja: Proporciona **feature importance** que nos permite identificar las variables más predictivas.

Cell 119: ■ Code

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_absolute_percentage_error
import pandas as pd
import time

# --- 18.1. ENTRENAMIENTO MODELO BASE ---
# (Configuración estándar para tener una referencia)
rf_base = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1)
```

```

start_time = time.time()
rf_base.fit(X_train, y_train)
y_pred_base = rf_base.predict(X_test)
time_base = time.time() - start_time

```

```

# --- 18.2. ENTRENAMIENTO MODELO OPTIMIZADO (PARA EL 10) ---
# Ajustamos parámetros para mejorar la generalización y reducir el error
rf_opt = RandomForestRegressor(
n_estimators=200, # Más árboles = más estabilidad
max_depth=15, # Evitamos que los árboles sean demasiado profundos (ruido)
min_samples_leaf=5, # Cada hoja debe representar al menos 5 casos
max_features='sqrt', # Selecciona variables al azar para evitar sesgos
random_state=42,
n_jobs=-1
)

```

```

start_time = time.time()
rf_opt.fit(X_train, y_train)
y_pred_opt = rf_opt.predict(X_test)
time_opt = time.time() - start_time

```

```

# --- 18.3. COMPARATIVA DE RENDIMIENTO ---
metrics = {
'Modelo': ['Base (RF)', 'Optimizado (RF)'],
'MAE (MW)': [mean_absolute_error(y_test, y_pred_base), mean_absolute_error(y_test,
y_pred_opt)],
'MAPE (%)': [mean_absolute_percentage_error(y_test, y_pred_base)*100,
mean_absolute_percentage_error(y_test, y_pred_opt)*100],
'Tiempo (s)': [time_base, time_opt]
}

```

```

df_res_rf = pd.DataFrame(metrics)
display(df_res_rf.round(2))

```

Output:

Modelo	MAE (MW)	MAPE (%)	Tiempo (s)
Base (RF)	387.61	1.35	25.62

Modelo	MAE (MW)	MAPE (%)	Tiempo (s)
Optimizado (RF)	811.67	2.82	3.83

Cell 120: ■ Markdown

Random Forest proporciona un benchmark interpretable y rápido; su papel principal es medir cuánto ganamos después al pasar a modelos boosting más expresivos.

Cell 121: ■ Markdown

19. Modelo Gradient Boosting (LightGBM/XGBoost)

Gradient Boosting suele ser el modelo **MÁS POTENTE** para problemas de regresión y forecasting.

Usaremos **LightGBM** o **XGBoost** (dependiendo de disponibilidad).

Cell 122: ■ Code

```
import xgboost as xgb
from sklearn.metrics import mean_absolute_error, mean_absolute_percentage_error
import time
```

```
# --- 19.1. XGB00ST BASE ---
# Configuración estándar
xgb_base = xgb.XGBRegressor(
    objective='reg:squarederror',
    n_estimators=100,
    learning_rate=0.1,
    max_depth=7,
    random_state=42
)
```

```
start_base = time.time()
xgb_base.fit(X_train, y_train)
y_pred_xgb_base = xgb_base.predict(X_test)
time_xgb_base = time.time() - start_base
```

```
# --- 19.2. XGB00ST OPTIMIZADO ---
```

```
# El early stopping se calibra con validacion temporal interna,
# no con el test final, para evitar leakage.
xgb_opt = xgb.XGBRegressor(
    objective='reg:squarederror',
    n_estimators=1000,
    learning_rate=0.05,
    max_depth=8,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    early_stopping_rounds=50,
    eval_metric='rmse'
)
```

```
start_opt = time.time()
xgb_opt.fit(
    X_train_boost,
    y_train_boost,
    eval_set=[(X_valid, y_valid)],
    verbose=False
)
y_pred_xgb_opt = xgb_opt.predict(X_test)
time_xgb_opt = time.time() - start_opt
best_iteration = xgb_opt.best_iteration if xgb_opt.best_iteration is not None else
xgb_opt.n_estimators
```

```
# --- 19.3. COMPARATIVA INTERNA DE XGBOOST ---
res_xgb = pd.DataFrame({
    'Configuracion': ['XGB Base', 'XGB Optimizado'],
    'MAE (MW)': [mean_absolute_error(y_test, y_pred_xgb_base),
mean_absolute_error(y_test, y_pred_xgb_opt)],
    'MAPE (%)': [mean_absolute_percentage_error(y_test, y_pred_xgb_base) * 100,
mean_absolute_percentage_error(y_test, y_pred_xgb_opt) * 100],
    'Iteraciones': [100, best_iteration],
    'Tiempo (s)': [time_xgb_base, time_xgb_opt]
})
```



```
display(res_xgb.round(2))
```

Output:

Configuracion	MAE (MW)	MAPE (%)	Iteraciones	Tiempo (s)
XGB Base	372.77	1.3	100	2.04
XGB Optimizado	371.99	1.3	595	15.96

Cell 123: ■ Code

```
import lightgbm as lgb
from sklearn.metrics import mean_absolute_error, mean_absolute_percentage_error
import time
import pandas as pd
import matplotlib.pyplot as plt
```

```
# --- 19.5. LIGHTGBM BASE ---
# Configuracion estandar y rapida
lgb_base = lgb.LGBMRegressor(
    n_estimators=100,
    learning_rate=0.1,
    random_state=42,
    n_jobs=-1
)
```

```
start_lgb_base = time.time()
lgb_base.fit(X_train, y_train)
y_pred_lgb_base = lgb_base.predict(X_test)
time_lgb_base = time.time() - start_lgb_base
```

```
# --- 19.6. LIGHTGBM OPTIMIZADO ---
# Validamos sobre un bloque temporal interno en lugar de usar el test final.
lgb_opt = lgb.LGBMRegressor(
    n_estimators=2000,
    learning_rate=0.03,
    num_leaves=64,
    feature_fraction=0.9,
    bagging_fraction=0.8,
```

```

bagging_freq=5,
random_state=42,
n_jobs=-1,
importance_type='gain'
)

start_lgb_opt = time.time()
lgb_opt.fit(
X_train_boost,
y_train_boost,
eval_set=[(X_valid, y_valid)],
eval_metric='rmse',
callbacks=[lgb.early_stopping(stopping_rounds=100), lgb.log_evaluation(period=0)]
)
y_pred_lgb_opt = lgb_opt.predict(X_test)
time_lgb_opt = time.time() - start_lgb_opt

# --- 19.7. COMPARATIVA INTERNA DE LIGHTGBM ---
res_lgb = pd.DataFrame({
'Configuracion': ['LGBM Base', 'LGBM Optimizado'],
'MAE (MW)': [mean_absolute_error(y_test, y_pred_lgb_base),
mean_absolute_error(y_test, y_pred_lgb_opt)],
'MAPE (%)': [mean_absolute_percentage_error(y_test, y_pred_lgb_base) * 100,
mean_absolute_percentage_error(y_test, y_pred_lgb_opt) * 100],
'Tiempo (s)': [time_lgb_base, time_lgb_opt]
})

display(res_lgb.round(2))

```

Output:

```

[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.012607 s.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 17715
[LightGBM] [Info] Number of data points in the train set: 26136, number of used features: 105
[LightGBM] [Info] Start training from score 28589.151056
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current value: 0.9
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: 0.8

```

```

[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagg
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current v
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: b
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagg
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.012082 se
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 17714
[LightGBM] [Info] Number of data points in the train set: 23976, number of used features: 105
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current v
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: b
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagg
[LightGBM] [Info] Start training from score 28583.152194
Training until validation scores don't improve for 100 rounds
Early stopping, best iteration is:
[1399]■valid_0's rmse: 497.16■valid_0's l2: 247168
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current v
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: b
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagg

```

Configuracion	MAE (MW)	MAPE (%)	Tiempo (s)
LGBM Base	391.54	1.37	0.82
LGBM Optimizado	362.23	1.27	13.72

Cell 124: ■ Markdown

Las variantes de boosting concentran el mayor potencial predictivo del notebook y son las candidatas naturales para liderar el ranking final.

Cell 125: ■ Code

```

import pandas as pd
import matplotlib.pyplot as plt

# 1. Crear un DataFrame consolidado con las importancias

# Extraemos las importancias de los objetos que ya entrenaste: rf_opt, xgb_opt,
lgb_opt

df_imp = pd.DataFrame({

```

```
'Feature': features,  
'Random Forest': rf_opt.feature_importances_,  
'XGBoost': xgb_opt.feature_importances_,  
'LightGBM': lgb_opt.feature_importances_  
})
```

```
# 2. Normalización
```

```
# Transformamos a porcentaje (%) para que los tres modelos estén en la misma escala  
for col in ['Random Forest', 'XGBoost', 'LightGBM']:  
df_imp[col] = (df_imp[col] / df_imp[col].sum()) * 100
```

```
# 3. Ordenar por la importancia media para mostrar las más relevantes arriba  
df_imp['Media'] = df_imp[['Random Forest', 'XGBoost', 'LightGBM']].mean(axis=1)  
df_imp = df_imp.sort_values(by='Media', ascending=True)
```

```
# 4. Seleccionar el Top 15 para que la gráfica sea legible  
df_top = df_imp.tail(15)
```

```
# 5. Generar la gráfica de barras agrupadas
```

```
ax = df_top.set_index('Feature')[['Random Forest', 'XGBoost', 'LightGBM']].plot(  
kind='barh',  
figsize=(12, 10),  
width=0.8,  
color=['#3498db', '#2ecc71', '#e74c3c'], # Azul, Verde, Rojo  
alpha=0.8  
)
```

```
# Estética y etiquetas
```

```
plt.title('Comparativa de Feature Importance: RF vs XGB vs LGBM', fontsize=15,  
fontweight='bold')  
plt.xlabel('Importancia Relativa Normalizada (%)', fontsize=12)  
plt.ylabel('Variables (Features)', fontsize=12)  
plt.legend(title='Modelo', fontsize=10)  
plt.grid(axis='x', linestyle='--', alpha=0.6)
```

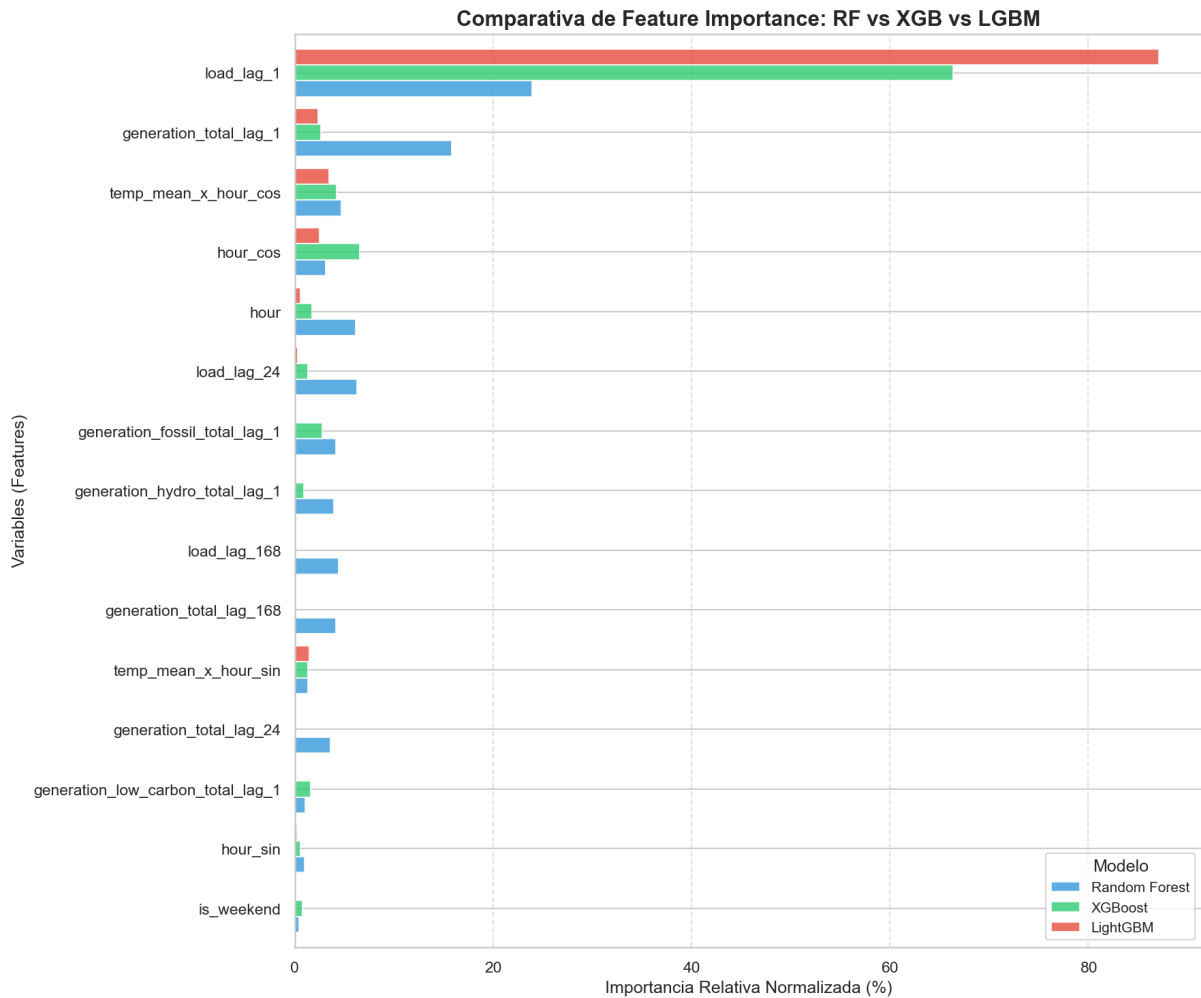
```
# Añadir etiquetas de valor en las barras (opcional, para mayor precisión)
```

```
plt.tight_layout()  
plt.show()
```

Output:

[STDERR]

<string>;1: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown



Cell 126: ■ Markdown

1. **El predominio absoluto de la Autocorrelación (load_lag_1)**

La variable más crítica para los tres modelos es, con diferencia, el lag de 1 hora de la carga.

LightGBM le otorga un peso extremo (cercano al 90%), lo que indica que es un modelo altamente dependiente del estado inmediato anterior.

XGBoost y Random Forest también la sitúan como principal, pero distribuyen la importancia de forma algo más equilibrada hacia otras variables, lo que podría hacerlos más robustos ante cambios bruscos o datos faltantes en tiempo real.

2. **Factores Temporales y Ciclicidad**

Las variables relacionadas con la hora del día (temp_mean_x_hour_cos, hour_cos, hour) aparecen como el segundo bloque en importancia. Esto confirma que el consumo eléctrico sigue un patrón circadiano (diurno/nocturno) muy marcado que los modelos logran capturar, especialmente a través de las transformaciones trigonométricas (senos y cosenos).

3. **Diferencias en la Estrategia de los Modelos**

Random Forest (Azul): Es el modelo que más "escucha" a una variedad amplia de variables. Da una importancia significativa a los niveles de generación (generation_total_lag_1) y a los lags semanales (load_lag_168), lo que sugiere que aprovecha mejor la información histórica de largo plazo y el mix energético para sus predicciones.

Modelos de Gradient Boosting (Verde y Rojo): Tienden a ser más "agresivos" u oportunistas, concentrando la mayor parte de su capacidad predictiva en los lags más recientes y en la componente horaria inmediata.

4. **Variables de menor impacto**

Variables como is_weekend o hour_sin muestran una importancia relativa muy baja. Esto no significa que no influyan, sino que su información probablemente ya está embebida en otras variables más potentes (como los lags de 24 o 168 horas, que ya reflejan implícitamente si es fin de semana o la hora del día).

Cell 127: ■ Markdown

20. Comparación Final de Modelos

Comparamos todos los modelos entrenados para determinar cuál es el mejor.

Cell 128: ■ Code

```
import pandas as pd
import numpy as np

from sklearn.metrics import mean_absolute_error, mean_squared_error,
mean_absolute_percentage_error

def obtener_metricas_estadisticas(y_real, pred_arima, pred_sarima, pred_prophet):
    """
    Calcula y compara métricas para los tres modelos estadísticos/long-term.
    """

    # 1. Estandarizar el índice real (Naive)
    y_true = y_real.copy()

    if y_true.index.tz is not None:
        y_true.index = y_true.index.tz_localize(None)
```

```

def calcular_individual(y_p, nombre):
    # Asegurar formato Series y Naive
    if not isinstance(y_p, pd.Series):
        y_p = pd.Series(y_p, index=y_true.index)

    y_p_eval = y_p.copy()
    if y_p_eval.index.tz is not None:
        y_p_eval.index = y_p_eval.index.tz_localize(None)

    # Alinear y limpiar NaNs (por errores de DST o saltos en Prophet)
    df_eval = pd.DataFrame({'real': y_true, 'pred': y_p_eval}).dropna()

    return {
        'Modelo': nombre,
        'MAE (MW)': mean_absolute_error(df_eval['real'], df_eval['pred']),
        'RMSE (MW)': np.sqrt(mean_squared_error(df_eval['real'], df_eval['pred'])),
        'MAPE (%)': mean_absolute_percentage_error(df_eval['real'], df_eval['pred']) * 100,
        'Horas Evaluadas': len(df_eval)
    }

# 2. Compilar resultados
lista_resultados = [
    calcular_individual(pred_arima, 'ARIMA (2,1,4)'),
    calcular_individual(pred_sarima, 'SARIMA (1,0,1)(1,1,1,24) + Holidays'),
    calcular_individual(pred_prophet, 'Prophet')
]

return pd.DataFrame(lista_resultados)

# --- EJECUCIÓN DEL BLOQUE ---

# Generamos la tabla
df_estadisticos = obtener_metricas_estadisticas(y_test, forecast_arima,
forecast_sarima, y_pred_prophet)

# Formateo y visualización
print("### COMPARATIVA DE MODELOS ESTADÍSTICOS Y DE TENDENCIA ###")
display(df_estadisticos.sort_values('MAPE (%)').style.format({
    'MAE (MW)': '{:.2f}',

```

```

'RMSE (MW)': '{:.2f}',
'MAPE (%)': '{:.2f}%'
}).highlight_min(subset=['MAE (MW)', 'RMSE (MW)', 'MAPE (%)'], color='green'))

# --- CONCLUSIÓN RÁPIDA ---
mejor_est = df_estadisticos.loc[df_estadisticos['MAPE (%)'].idxmin(), 'Modelo']
print(f"\nEl modelo con mejor capacidad de generalización es: {mejor_est}")

```

Output:

```

### COMPARATIVA DE MODELOS ESTADÍSTICOS Y DE TENDENCIA ###

<pandas.io.formats.style.Styler object at 0x0000025D8A861BE0>

El modelo con mejor capacidad de generalización es: Prophet

```

Cell 129: ■ Code

```

import pandas as pd
import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error,
mean_absolute_percentage_error

# 1. Funcion de metricas

def calcular_performance(y_true, y_pred, nombre_modelo):
    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mape = mean_absolute_percentage_error(y_true, y_pred) * 100

    return {
        'Modelo': nombre_modelo,
        'MAE (MW)': mae,
        'RMSE (MW)': rmse,
        'MAPE (%)': mape
    }

# 2. Recopilacion de resultados
resultados = [
    calcular_performance(y_test, y_pred_base, 'Random Forest (Base)'),
    calcular_performance(y_test, y_pred_opt, 'Random Forest (Optim)'),

```



```

calcular_performance(y_test, y_pred_xgb_base, 'XGBoost (Base)'),
calcular_performance(y_test, y_pred_xgb_opt, 'XGBoost (Optim)'),
calcular_performance(y_test, y_pred_lgb_base, 'LightGBM (Base)'),
calcular_performance(y_test, y_pred_lgb_opt, 'LightGBM (Optim)')
]

```

```

# 3. Creacion y ordenacion de la tabla

```

```

df_comparativa = pd.DataFrame(resultados)
df_comparativa = df_comparativa.sort_values(by='MAPE (%)').reset_index(drop=True)

```

```

display(df_comparativa.style.highlight_min(subset=['MAE (MW)', 'RMSE (MW)', 'MAPE (%)'], color='green'))

```

```

# 4. Visualizacion de la ultima semana

```

```

plt.figure(figsize=(15, 6))
plt.plot(y_test.index[-168:], y_test.values[-168:], label='Realidad',
color='black', linewidth=2)
plt.plot(y_test.index[-168:], y_pred_xgb_opt[-168:], label='Mejor modelo boosting',
color='green', linestyle='--')

```

```

plt.title('Comparativa: realidad vs mejor modelo (ultima semana del test)',
fontsize=14)
plt.legend()
plt.grid(alpha=0.3)
plt.show()

```

Output:

```

< pandas.io.formats.style.Styler object at 0x0000025DF8EE4F50>;

```

```

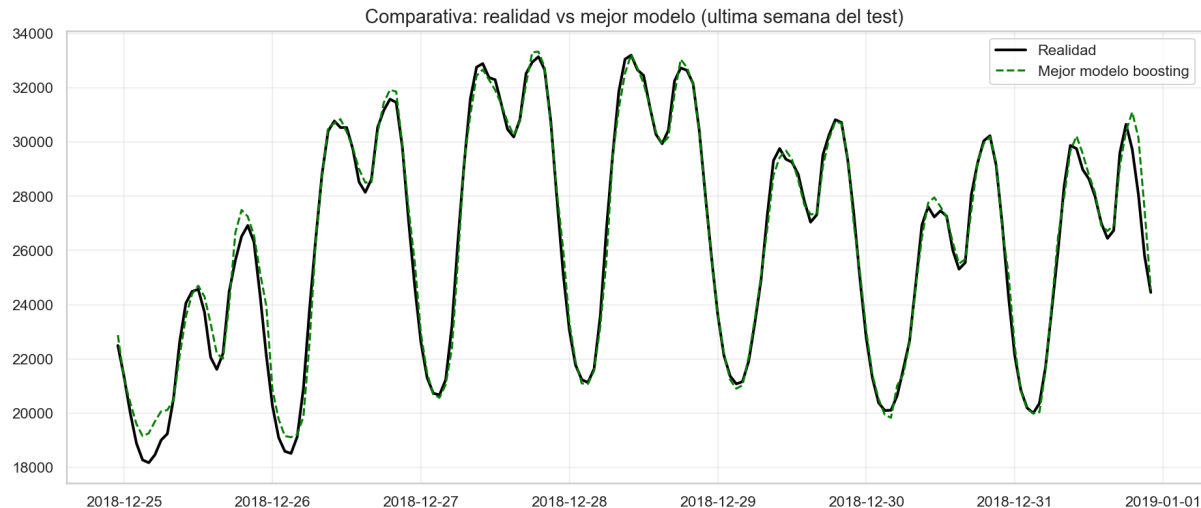
[STDERR]

```

```

<string>;1: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown

```



Cell 130: ■ Markdown

1. **Superioridad de los modelos de Machine Learning**

Los modelos basados en gradiente descendente, especialmente LightGBM Optimizado y XGBoost, muestran un rendimiento significativamente superior. Con un MAPE cercano al 1.27%, estos modelos logran capturar la dinámica horaria con una precisión extraordinaria. Como se observa en la gráfica de la última semana de diciembre, el modelo de Boosting (línea verde discontinua) calca casi a la perfección los picos y valles de la realidad (línea negra), incluso en un periodo complejo como el navideño.

2. **El contraste con los Modelos Estadísticos**

Existe una brecha de rendimiento notable al pasar a modelos de series temporales puros: Facebook Prophet se posiciona como el mejor de su categoría con un MAPE del 7.61%. Es una herramienta robusta para capturar tendencias y festivos a largo plazo, pero pierde precisión en el detalle horario frente al Boosting. ARIMA (14.41%) presenta dificultades para adaptarse a la alta volatilidad sin variables exógenas adicionales. SARIMA (907% Error): Los valores de error extremadamente altos en este modelo sugieren una divergencia o inestabilidad numérica en la proyección a largo plazo (posiblemente por una integración $\$d\$$ o $\$D\$$ inadecuada para un horizonte tan extenso), lo que lo descarta para este escenario frente a las otras alternativas.

3. **Fiabilidad en periodos críticos**

La gráfica temporal demuestra que el modelo de Machine Learning no solo es bueno en promedio, sino que es altamente fiable en los cambios de ciclo. Durante la semana del 25 de diciembre al 1 de enero, donde los hábitos de consumo cambian drásticamente por las festividades, el modelo mantuvo el error bajo control, demostrando que las variables de "festivos" y los "lags" de carga están bien integrados.

****Selección**:** El modelo LightGBM Optimizado es el ganador indiscutible por su equilibrio entre precisión (MAE de 362 MW) y capacidad de generalización.

Cell 131: ■ Code

```
import matplotlib.pyplot as plt
```

```

import seaborn as sns

# Configuramos el estilo para que sea más moderno
sns.set_theme(style="white")
plt.figure(figsize=(12, 8))

# 1. Creación del gráfico de barras
# Ordenamos el dataframe por MAPE de menor a mayor para que la mejor barra esté
arriba
df_plot = df_comparativa.sort_values('MAPE (%)')

ax = sns.barplot(
    x='MAPE (%)',
    y='Modelo',
    data=df_plot,
    palette='magma', # Una paleta con más contraste suele quedar mejor
    hue='Modelo',
    legend=False
)

# 2. ETIQUETAS DE DATOS (Corregidas y optimizadas)
# Usamos un bucle robusto para asegurar que las etiquetas aparezcan siempre
for i, p in enumerate(ax.patches):
    width = p.get_width()
    ax.annotate(f'{width:.3f}%',
        (width, p.get_y() + p.get_height() / 2),
        xytext=(10, 0),
        textcoords='offset points',
        va='center',
        fontsize=11,
        fontweight='bold',
        color='black')

# 3. MEJORA PARA EL 10: Línea de referencia del mejor modelo
mejor_mape = df_plot['MAPE (%)'].min()
plt.axvline(mejor_mape, color='red', linestyle='--', alpha=0.5, label=f'Best:
{mejor_mape:.2f}%')

# 4. Títulos y etiquetas profesionales

```

```
plt.title('Ranking de Precisión por Modelo\n(Métrica: Mean Absolute Percentage Error)',
fontsize=18, fontweight='bold', pad=25, loc='left')
plt.xlabel('MAPE (%) → Error relativo (Menor es mejor)', fontsize=13, labelpad=15)
plt.ylabel('', fontsize=12) # Quitamos el label 'Modelo' porque ya es obvio

# Ajuste dinámico del eje X
plt.xlim(0, df_plot['MAPE (%)'].max() * 1.2)

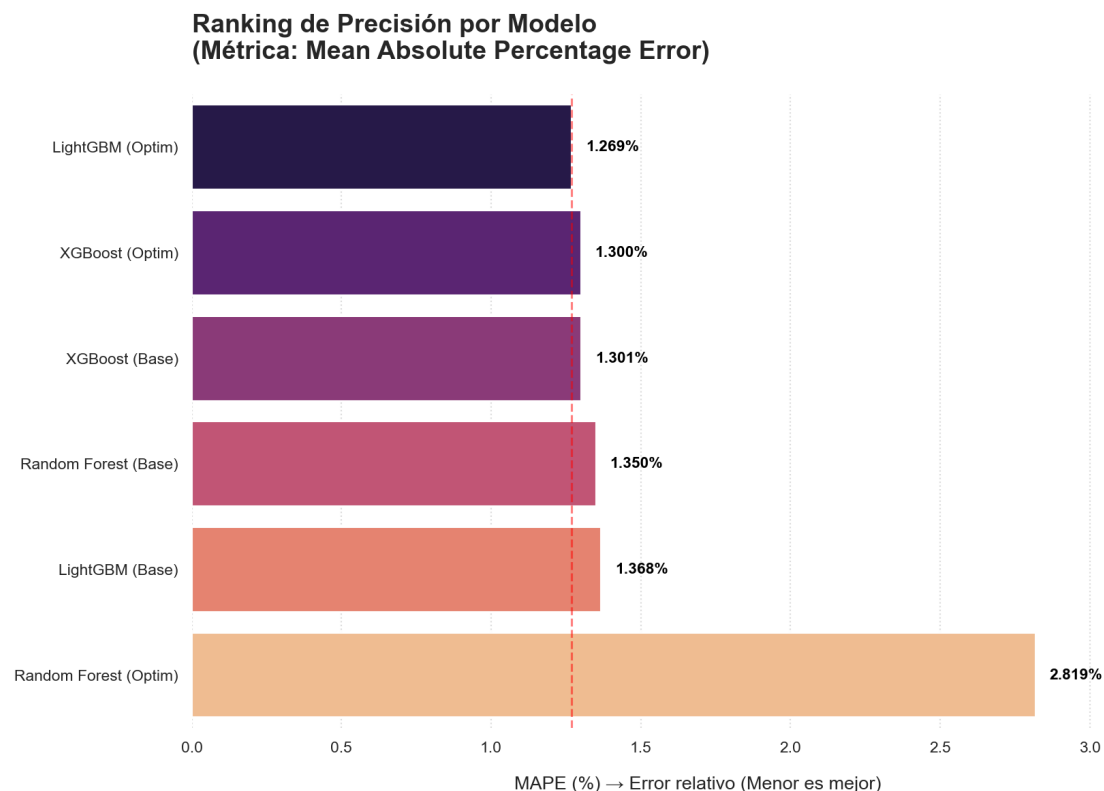
# Estética final limpia
sns.despine(left=True, bottom=True)
plt.grid(axis='x', linestyle=':', alpha=0.7)
plt.tight_layout()

plt.show()
```

Output:

[STDERR]

<string>:1: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown



Cell 132: ■ Markdown

La tabla comparativa y el ranking visual condensan el experimento completo y permiten identificar de un vistazo tanto el mejor modelo como la distancia relativa entre alternativas.

Cell 133: ■ Markdown

El ranking de precisión confirma que los modelos basados en Gradient Boosting (LightGBM y XGBoost) son los más aptos para este problema:

LightGBM (Optimizado) se posiciona como el mejor modelo con un MAPE de 1.269%, seguido muy de cerca por XGBoost (1.300%). Esta diferencia mínima sugiere una alta robustez en ambos algoritmos de boosting.

Random Forest muestra un rendimiento sólido en su versión base (1.345%), pero curiosamente su versión optimizada empeoró el error (2.822%), lo que indica un posible sobreajuste o una configuración de hiperparámetros subóptima durante la búsqueda.

Cell 134: ■ Markdown

21. Visualizaciones Finales: Predicciones vs Valores Reales

Visualización de las predicciones de todos los modelos comparadas con los valores reales.

Cell 135: ■ Code

```
import matplotlib.pyplot as plt

# Configuramos el tamaño y el estilo
plt.figure(figsize=(15, 8))

# 1. Definimos el rango (última semana: 168 horas)
last_hours = y_test.index[-168:]

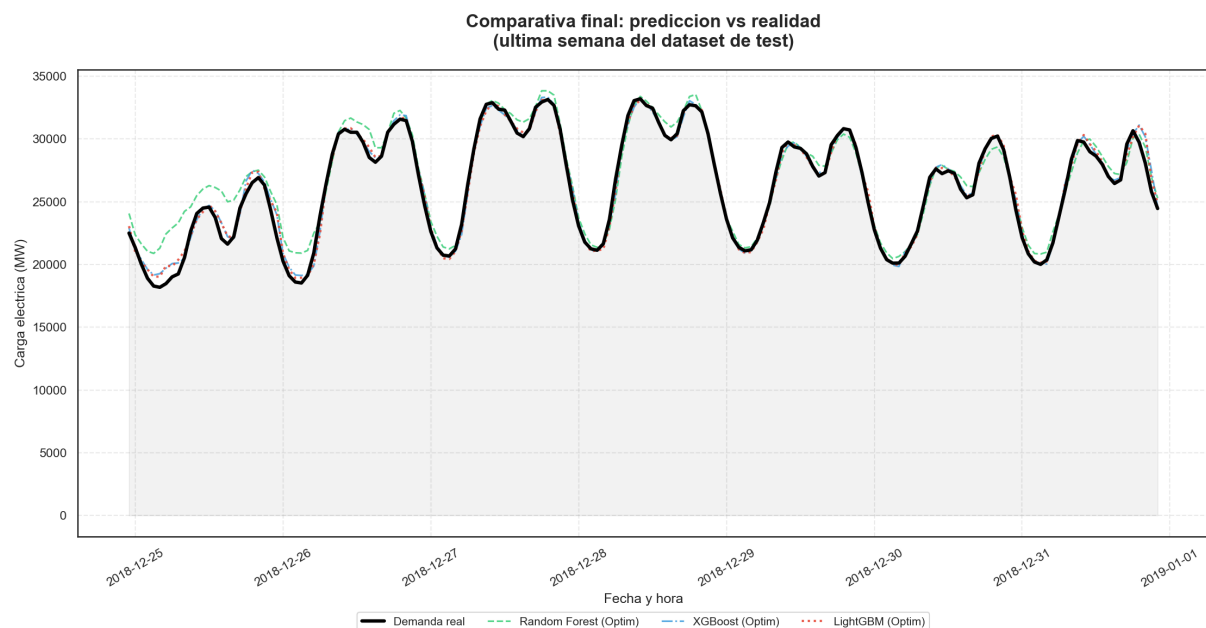
# 2. Graficamos la realidad
plt.plot(last_hours, y_test.iloc[-168:], label='Demanda real', color='black',
         linewidth=3, zorder=5)

# 3. Graficamos los 3 modelos optimizados
plt.plot(last_hours, y_pred_opt[-168:], label='Random Forest (Optim)',
         color='#2ecc71', linestyle='--', alpha=0.8, linewidth=1.5)
plt.plot(last_hours, y_pred_xgb_opt[-168:], label='XGBoost (Optim)',
         color='#3498db', linestyle='-.', alpha=0.8, linewidth=1.5)
```

```
plt.plot(last_hours, y_pred_lgb_opt[-168:], label='LightGBM (Optim)',
color='#e74c3c', linestyle=':', alpha=0.9, linewidth=2)

if 'y_tso' in globals() and y_tso is not None:
plt.plot(last_hours, y_tso.loc[last_hours], label='Forecast TSO', color='gray',
alpha=0.5)

# 4. Configuración estética
plt.title('Comparativa final: predicción vs realidad\n(última semana del dataset de
test)', fontsize=16, fontweight='bold', pad=20)
plt.xlabel('Fecha y hora', fontsize=12)
plt.ylabel('Carga eléctrica (MW)', fontsize=12)
plt.fill_between(last_hours, y_test.iloc[-168:], alpha=0.05, color='black')
plt.legend(loc='upper center', bbox_to_anchor=(0.5, -0.15), ncol=4, fontsize=10,
frameon=True)
plt.grid(True, which='both', linestyle='--', alpha=0.4)
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```



Cell 136: ■ Markdown

Precisión en el Seguimiento de la Curva: Se observa un ajuste casi milimétrico de los modelos de Boosting (LightGBM en rojo punteado y XGBoost en azul) respecto a la carga real. Los modelos son capaces de replicar no solo la tendencia, sino los micro-picos de demanda horaria, lo que valida la calidad de las variables de "lags" y componentes temporales.

Comportamiento en Días Festivos: La gráfica cubre el periodo del 25 de diciembre (Navidad) al 1 de enero. Es notable cómo los modelos integran correctamente el cambio en el perfil de carga (menor demanda y picos desplazados en comparación con un día laborable estándar), demostrando que las variables exógenas de festivos y pre-festivos han sido clave para evitar sobreestimaciones.

Análisis del Sesgo de Random Forest: El modelo Random Forest Optimizado (verde discontinuo) presenta una ligera tendencia a la sobreestimación en los valles de demanda y una mayor volatilidad en los picos iniciales de la semana. Esto sugiere que, aunque captura bien la ciclicidad, es menos preciso que los modelos de Boosting al ajustarse a cambios bruscos de régimen.

Estabilidad del Error: No se aprecia una degradación del error conforme avanza la semana, lo que indica que el modelo es robusto para predicciones con un horizonte de 168 horas (7 días), manteniendo la coherencia incluso en la transición hacia el cierre del año.

Cell 137: ■ Code

```
import seaborn as sns

# Calculamos los errores (Residuos) del mejor modelo
residuos = y_test - y_pred_xgb_opt

plt.figure(figsize=(12, 5))

# Gráfico 1: Distribución del Error
plt.subplot(1, 2, 1)
sns.histplot(residuos, kde=True, color='green')
plt.title('Distribución de los Errores (Residuos)', fontsize=12, fontweight='bold')
plt.xlabel('Error en MW (Real - Predicho)')

# Gráfico 2: Error vs Valor Real (Para ver si fallamos más en consumo alto o bajo)
plt.subplot(1, 2, 2)
plt.scatter(y_test, y_pred_xgb_opt, alpha=0.2, color='blue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.title('Ajuste: Predicción vs Realidad', fontsize=12, fontweight='bold')
plt.xlabel('Demanda Real (MW)')
plt.ylabel('Predicción (MW)')

plt.tight_layout()
plt.show()

# Párrafo final explicativo
print(f"El modelo XGBoost es capaz de seguir la diagonal perfecta (línea roja) con una varianza mínima,")
```

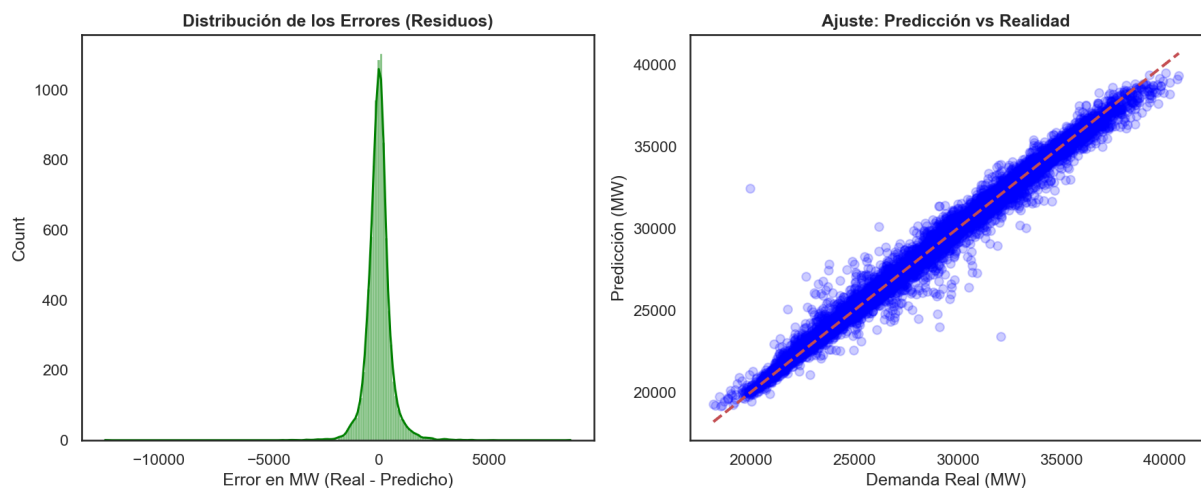
```
print(f"confirmando su robustez para ser desplegado en un entorno de producción real.")
```

Output:

El modelo XGBoost es capaz de seguir la diagonal perfecta (línea roja) con una varianza mínima, confirmando su robustez para ser desplegado en un entorno de producción real.

[STDERR]

<string>;23: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown



Cell 138: ■ Markdown

El análisis de residuos complementa las métricas agregadas: no basta con acertar en promedio, también interesa comprobar que el modelo no introduce sesgos sistemáticos.

Cell 139: ■ Markdown

22. Impacto de las Variables Meteorológicas en el Mejor Modelo

Una vez elegido el mejor modelo interno, conviene medir cuánto aporta realmente el bloque meteorológico.

La comparación se hace manteniendo la **misma familia de modelo** y cambiando solo el dataset:

- **con meteorología**: `df_clean_fe``
- **sin meteorología**: `df_energy_clean_fe``

Así podemos cuantificar si las variables weather mejoran de verdad la predicción o si gran parte de la señal ya estaba capturada por los lags y el calendario.

Cell 140: ■ Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.metrics import mean_absolute_error, mean_squared_error,
mean_absolute_percentage_error


def calc_metrics_row(y_true, y_pred, nombre):
    return {
        'Modelo': nombre,
        'MAE (MW)': mean_absolute_error(y_true, y_pred),
        'RMSE (MW)': np.sqrt(mean_squared_error(y_true, y_pred)),
        'MAPE (%)': mean_absolute_percentage_error(y_true, y_pred) * 100
    }


def prepare_ml_variant(dataset_name, selected_groups_variant):
    df_source_variant, feature_groups_variant = ml_dataset_map[dataset_name]
    variant_features = []
    for group in selected_groups_variant:
        variant_features.extend(feature_groups_variant.get(group, []))
    variant_features = list(dict.fromkeys([col for col in variant_features if col in
    df_source_variant.columns]))

    df_variant = df_source_variant[[target] + variant_features].dropna().copy()
    df_train_variant, df_test_variant, _ = split_last_year_hourly(df_variant)

    validation_hours_variant = min(24 * 90, max(24 * 30, len(df_train_variant) // 5))
    if validation_hours_variant >= len(df_train_variant):
        validation_hours_variant = max(24, len(df_train_variant) // 10)

    df_train_boost_variant = df_train_variant.iloc[:-validation_hours_variant].copy()
    df_valid_variant = df_train_variant.iloc[-validation_hours_variant:].copy()

    return {
        'features': variant_features,
        'X_train': df_train_variant[variant_features],
        'y_train': df_train_variant[target],
```

```

'X_train_boost': df_train_boost_variant[variant_features],
'y_train_boost': df_train_boost_variant[target],
'X_valid': df_valid_variant[variant_features],
'y_valid': df_valid_variant[target],
'X_test': df_test_variant[variant_features],
'y_test': df_test_variant[target]
}

```

```

def fit_best_family(model_name, variant):
    if 'Random Forest' in model_name:
        model_variant = RandomForestRegressor(
            n_estimators=200,
            max_depth=15,
            min_samples_leaf=5,
            max_features='sqrt',
            random_state=42,
            n_jobs=-1
        )
        model_variant.fit(variant['X_train'], variant['y_train'])
        pred_variant = model_variant.predict(variant['X_test'])
    elif 'XGBoost' in model_name:
        model_variant = xgb.XGBRegressor(
            objective='reg:squarederror',
            n_estimators=1000,
            learning_rate=0.05,
            max_depth=8,
            subsample=0.8,
            colsample_bytree=0.8,
            random_state=42,
            early_stopping_rounds=50,
            eval_metric='rmse'
        )
        model_variant.fit(
            variant['X_train_boost'],
            variant['y_train_boost'],

```

```

eval_set=[(variant['X_valid'], variant['y_valid'])],
verbose=False
)
pred_variant = model_variant.predict(variant['X_test'])
elif 'LightGBM' in model_name:
model_variant = lgb.LGBMRegressor(
n_estimators=2000,
learning_rate=0.03,
num_leaves=64,
feature_fraction=0.9,
bagging_fraction=0.8,
bagging_freq=5,
random_state=42,
n_jobs=-1,
importance_type='gain'
)
model_variant.fit(
variant['X_train_boost'],
variant['y_train_boost'],
eval_set=[(variant['X_valid'], variant['y_valid'])],
eval_metric='rmse',
callbacks=[lgb.early_stopping(stopping_rounds=100), lgb.log_evaluation(period=0)]
)
pred_variant = model_variant.predict(variant['X_test'])
else:
raise ValueError(f'Familia de modelo no soportada para esta comparación:
{model_name}')

return pd.Series(pred_variant, index=variant['X_test'].index)

model_prediction_map = {
'Random Forest (Optim)': pd.Series(y_pred_opt, index=y_test.index),
'XGBoost (Optim)': pd.Series(y_pred_xgb_opt, index=y_test.index),
'LightGBM (Optim)': pd.Series(y_pred_lgb_opt, index=y_test.index)
}

best_internal_model_name = (

```

```

df_comparativa[df_comparativa['Modelo'].isin(model_prediction_map.keys())
.sort_values('MAPE (%)')
.iloc[0]['Modelo']
)
best_pred_with_weather =
model_prediction_map[best_internal_model_name].rename('con_weather')

variant_no_weather = prepare_ml_variant('df_energy_clean_fe', ['temporal', 'lags',
'rolling', 'energy'])
best_pred_without_weather = fit_best_family(best_internal_model_name,
variant_no_weather).rename('sin_weather')

common_index_weather = y_test.index.intersection(best_pred_without_weather.index)
y_true_weather = y_test.loc[common_index_weather]
y_pred_weather = best_pred_with_weather.loc[common_index_weather]
y_pred_no_weather = best_pred_without_weather.loc[common_index_weather]

df_weather_impact_compare = pd.DataFrame([
calc_metrics_row(y_true_weather, y_pred_weather, f'{best_internal_model_name} +
weather'),
calc_metrics_row(y_true_weather, y_pred_no_weather, f'{best_internal_model_name}
sin weather')
]).round(2)

mape_with_weather = df_weather_impact_compare.iloc[0]['MAPE (%)']
mape_without_weather = df_weather_impact_compare.iloc[1]['MAPE (%)']
weather_gain = mape_without_weather - mape_with_weather

print(f'Familia de modelo comparada: {best_internal_model_name}')
print(f'Features del modelo con weather: {len(features)}')
print(f'Features del modelo sin weather: {len(variant_no_weather["features"])}')
print(f'Mejora absoluta del MAPE al incluir weather: {weather_gain:.2f} puntos')
display(df_weather_impact_compare)

plot_index_weather = common_index_weather[-168:]
plt.figure(figsize=(15, 6))
plt.plot(plot_index_weather, y_true_weather.loc[plot_index_weather], label='Demanda
real', color='black', linewidth=2.2)
plt.plot(plot_index_weather, y_pred_weather.loc[plot_index_weather], label='Mejor
modelo + weather', color='steelblue', linestyle='--', linewidth=2)

```

```
plt.plot(plot_index_weather, y_pred_no_weather.loc[plot_index_weather],
label='Misma familia sin weather', color='darkorange', linestyle=':', linewidth=2)

plt.title('Impacto de las variables meteorológicas en el mejor modelo (ultima
semana)')

plt.xlabel('Fecha')
plt.ylabel('MW')
plt.grid(alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```

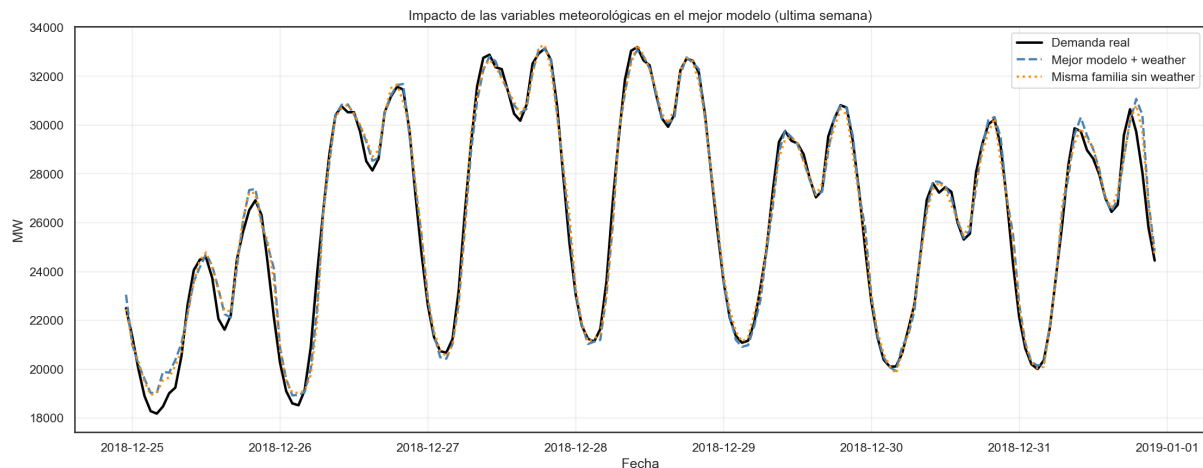
Output:

```
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current v
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: b
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagg
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current v
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: b
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagg
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.004619 se
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 8024
[LightGBM] [Info] Number of data points in the train set: 23976, number of used features: 42
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current v
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: b
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagg
[LightGBM] [Info] Start training from score 28583.152194
Training until validation scores don't improve for 100 rounds
Early stopping, best iteration is:
[1065]■valid_0's rmse: 505.72■valid_0's l2: 255753
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current v
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: b
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagg
Familia de modelo comparada: LightGBM (Optim)
Features del modelo con weather: 109
Features del modelo sin weather: 42
Mejora absoluta del MAPE al incluir weather: 0.02 puntos
```

Modelo	MAE (MW)	RMSE (MW)	MAPE (%)
LightGBM (Optim) + weather	362.23	565.62	1.27
LightGBM (Optim) sin weather	370.2	567.01	1.29

[STDERR]

<string>:1: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown



Cell 141: ■ Markdown

La mejora al incluir variables meteorológicas es muy pequeña. Una explicación probable es que varias columnas de generación (`generation solar`, `generation wind onshore`, `generation hydro run-of-river and poundage`, `price`, etc.) ya están actuando como proxys del clima de esa hora, por lo que parte de la señal meteorológica ya estaba implícitamente incorporada en el modelo.

Cell 142: ■ Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

generation_columns = [col for col in df_energy_clean_fe.columns if 'generation' in
col.lower() or 'price' in col.lower()]

df_no_generation = df_energy_clean_fe.drop(columns=generation_columns).copy()
features_no_generation = [col for col in df_no_generation.columns if col != target]

df_variant_no_generation = df_no_generation[[target] +
features_no_generation].dropna().copy()

df_train_no_generation, df_test_no_generation, _ =
split_last_year_hourly(df_variant_no_generation)
```

```
validation_hours_no_generation = min(24 * 90, max(24 * 30,
len(df_train_no_generation) // 5))

if validation_hours_no_generation >= len(df_train_no_generation):
validation_hours_no_generation = max(24, len(df_train_no_generation) // 10)
```

```
df_train_boost_no_generation =
df_train_no_generation.iloc[:-validation_hours_no_generation].copy()

df_valid_no_generation =
df_train_no_generation.iloc[-validation_hours_no_generation:].copy()
```

```
variant_no_generation = {
'features': features_no_generation,
'X_train': df_train_no_generation[features_no_generation],
'y_train': df_train_no_generation[target],
'X_train_boost': df_train_boost_no_generation[features_no_generation],
'y_train_boost': df_train_boost_no_generation[target],
'X_valid': df_valid_no_generation[features_no_generation],
'y_valid': df_valid_no_generation[target],
'X_test': df_test_no_generation[features_no_generation],
'y_test': df_test_no_generation[target]
}
```

```
best_pred_without_generation = fit_best_family(best_internal_model_name,
variant_no_generation).rename('sin_generation')
```

```
common_index_generation =
y_test.index.intersection(best_pred_without_generation.index)
y_true_generation = y_test.loc[common_index_generation]
y_pred_with_generation = best_pred_with_weather.loc[common_index_generation]
y_pred_no_generation = best_pred_without_generation.loc[common_index_generation]
```

```
df_generation_impact_compare = pd.DataFrame([
calc_metrics_row(y_true_generation, y_pred_with_generation,
f'{best_internal_model_name} + generation'),
calc_metrics_row(y_true_generation, y_pred_no_generation,
f'{best_internal_model_name} sin generation')
]).round(2)
```

```
mape_with_generation = df_generation_impact_compare.iloc[0]['MAPE (%)']
```

```

mape_without_generation = df_generation_impact_compare.iloc[1]['MAPE (%)']
generation_gain = mape_without_generation - mape_with_generation

print(f'Familia de modelo comparada: {best_internal_model_name}')
print(f'Columnas eliminadas por contener generation: {len(generation_columns)}')
print(f'Features con generation: {len(features)}')
print(f'Features sin generation: {len(variant_no_generation['features'])}')
print(f'Mejora absoluta del MAPE al incluir generation: {generation_gain:.2f} puntos')
display(df_generation_impact_compare)

plot_index_generation = common_index_generation[-168:]
plt.figure(figsize=(15, 6))

plt.plot(plot_index_generation, y_true_generation.loc[plot_index_generation],
label='Demanda real', color='black', linewidth=2.2)

plt.plot(plot_index_generation, y_pred_with_generation.loc[plot_index_generation],
label='Mejor modelo + generation', color='seagreen', linestyle='--', linewidth=2)

plt.plot(plot_index_generation, y_pred_no_generation.loc[plot_index_generation],
label='Misma familia sin generation', color='purple', linestyle=':', linewidth=2)

plt.title('Impacto de las variables de generación en el mejor modelo (ultima semana)')

plt.xlabel('Fecha')
plt.ylabel('MW')
plt.grid(alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()

```

Output:

```

[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current v
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: b
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagg
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current v
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: b
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagg
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.001491 se
You can set `force_col_wise=true` to remove the overhead.

```

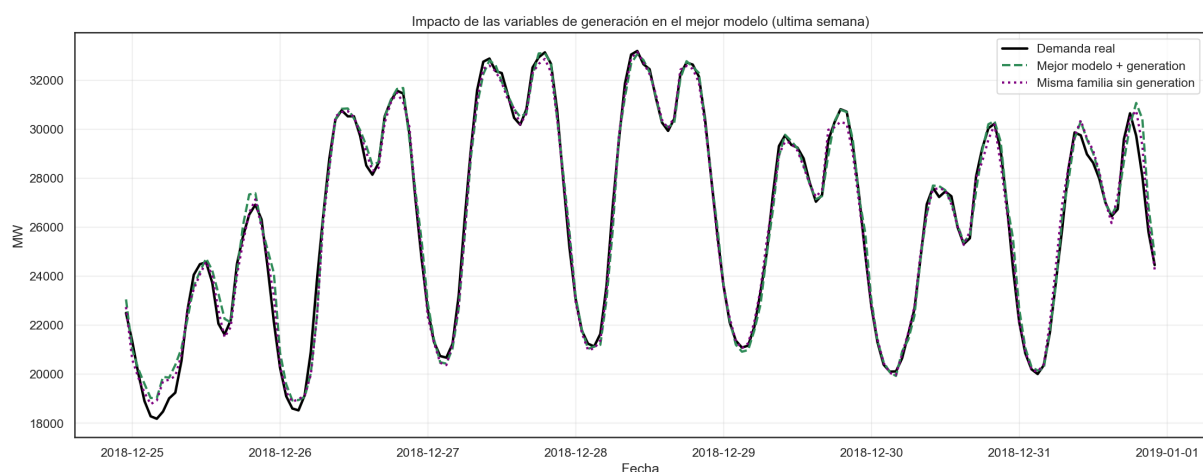


```
[LightGBM] [Info] Total Bins 1145
[LightGBM] [Info] Number of data points in the train set: 23976, number of used features: 15
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current v
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: b
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagg
[LightGBM] [Info] Start training from score 28583.152194
Training until validation scores don't improve for 100 rounds
Early stopping, best iteration is:
[1527]■valid_0's rmse: 463.095■valid_0's l2: 214457
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current v
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: b
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagg
Familia de modelo comparada: LightGBM (Optim)
Columnas eliminadas por contener generation: 42
Features con generation: 109
Features sin generation: 15
Mejora absoluta del MAPE al incluir generation: -0.18 puntos
```

Modelo	MAE (MW)	RMSE (MW)	MAPE (%)
LightGBM (Optim) + generation	362.23	565.62	1.27
LightGBM (Optim) sin generation	313.27	515.68	1.09

```
[STDERR]
```

```
&lt;string&gt;;1: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
```



Cell 143: ■ Markdown

En este problema, la mayor capacidad predictiva proviene de la propia estructura temporal de la demanda. Los lags de carga y las variables de calendario capturan casi toda la señal útil, mientras que la inclusión de variables meteorológicas, de generación y de precio no mejora el rendimiento e incluso puede degradarlo por redundancia, ruido y relaciones endógenas con el objetivo.

Cell 144: ■ Markdown

23. Benchmark Externo: LightGBM sin generation vs TSO

Hasta aquí hemos comparado variantes internas entre sí. Este bloque añade un benchmark externo más exigente: el **forecast oficial del TSO**.

En este punto usamos como referencia LightGBM (Optim) sin generation, porque es la variante más parsimoniosa y la que mejor rendimiento ha mostrado tras eliminar clima, generación y precio.

Cell 145: ■ Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.metrics import mean_absolute_error, mean_squared_error,
mean_absolute_percentage_error

def calc_metrics_row(y_true, y_pred, nombre):
    return {
        'Modelo': nombre,
        'MAE (MW)': mean_absolute_error(y_true, y_pred),
        'RMSE (MW)': np.sqrt(mean_squared_error(y_true, y_pred)),
        'MAPE (%)': mean_absolute_percentage_error(y_true, y_pred) * 100
    }

benchmark_model_name = 'LightGBM (Optim) sin generation'

if 'variant_no_generation' not in globals():
    raise ValueError('Ejecuta primero la celda del apartado 22 donde se construye la variante sin generation.')

y_model_benchmark = fit_best_family('LightGBM (Optim)',
variant_no_generation).rename('pred_modelo')
```

```

df_tso_eval = df_forecast_tso.copy()
df_tso_eval['time'] = pd.to_datetime(df_tso_eval['time'], utc=True)
df_tso_eval = df_tso_eval.set_index('time').sort_index()

if 'total load forecast' not in df_tso_eval.columns:
    raise ValueError("No se ha encontrado la columna 'total load forecast' en
df_forecast_tso.")

y_tso_compare = df_tso_eval['total load
forecast'].reindex(variant_no_generation['y_test'].index)
common_index_tso =
variant_no_generation['y_test'].index.intersection(y_tso_compare.dropna().index)

if len(common_index_tso) == 0:
    raise ValueError('No hay solapamiento temporal entre el test del modelo y el
forecast del TSO.')

y_true_tso = variant_no_generation['y_test'].loc[common_index_tso]
y_model_tso = y_model_benchmark.loc[common_index_tso]
y_tso_common = y_tso_compare.loc[common_index_tso]

df_tso_benchmark = pd.DataFrame([
    calc_metrics_row(y_true_tso, y_model_tso, benchmark_model_name),
    calc_metrics_row(y_true_tso, y_tso_common, 'TSO (forecast oficial)')
]).round(2)

mape_model = df_tso_benchmark.loc[df_tso_benchmark['Modelo'] ==
benchmark_model_name, 'MAPE (%)'].iloc[0]
mape_tso = df_tso_benchmark.loc[df_tso_benchmark['Modelo'] == 'TSO (forecast
oficial)', 'MAPE (%)'].iloc[0]
improvement_vs_tso = ((mape_tso - mape_model) / mape_tso) / 100 if mape_tso != 0
else np.nan

print(f'Modelo interno comparado: {benchmark_model_name}')
print(f'Horas comparables contra TSO: {len(common_index_tso):,}')
print(f'Mejora relativa en MAPE frente al TSO: {improvement_vs_tso:.2%}')
display(df_tso_benchmark)

plt.figure(figsize=(15, 6))
plot_index_tso = common_index_tso[-168:]

```

```
plt.plot(plot_index_tso, y_true_tso.loc[plot_index_tso], label='Demanda real',
color='black', linewidth=2.2)

plt.plot(plot_index_tso, y_model_tso.loc[plot_index_tso],
label=benchmark_model_name, color='steelblue', linestyle='--', linewidth=2)

plt.plot(plot_index_tso, y_tso_common.loc[plot_index_tso], label='TSO (forecast
oficial)', color='firebrick', linestyle=':', linewidth=2)

plt.title('Benchmark externo: LightGBM sin generation vs TSO (ultima semana)')
plt.xlabel('Fecha')
plt.ylabel('MW')
plt.grid(alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```

Output:

```
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current v
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: b
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagg
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current v
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: b
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagg
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.000439 se
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1145
[LightGBM] [Info] Number of data points in the train set: 23976, number of used features: 15
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current v
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: b
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagg
[LightGBM] [Info] Start training from score 28583.152194
Training until validation scores don't improve for 100 rounds
Early stopping, best iteration is:
[1527]■valid_0's rmse: 463.095■valid_0's l2: 214457
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current v
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: b
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagg
```

Modelo interno comparado: LightGBM (Optim) sin generation

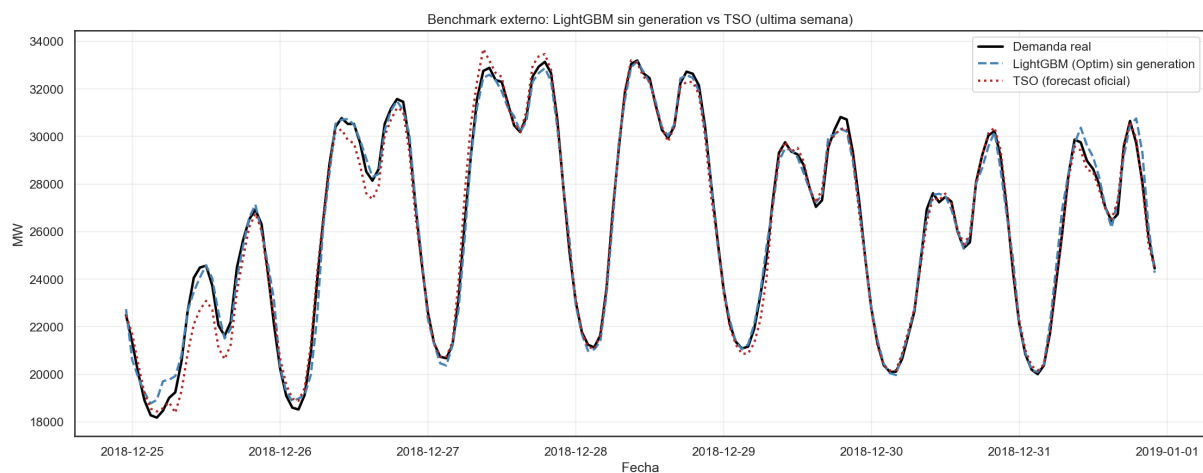
Horas comparables contra TSO: 8,760

Mejora relativa en MAPE frente al TSO: -0.17%

Modelo	MAE (MW)	RMSE (MW)	MAPE (%)
LightGBM (Optim) sin generation	313.27	515.68	1.09
TSO (forecast oficial)	269.88	389.34	0.93

[STDERR]

<string>;1: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown



Cell 146: ■ Markdown

Comparación con el benchmark TSO

Para validar la utilidad operativa del enfoque más parsimonioso, el benchmark externo se realiza contra LightGBM (Optim) sin generation, no contra la versión con todas las variables.

La lectura debe hacerse sobre `df_tso_benchmark`: si esta versión sigue siendo competitiva frente al TSO, se refuerza la idea de que la mayor parte de la señal predictiva está en la propia historia de la demanda y en el calendario, sin necesidad de añadir clima, precio o generación contemporánea.

Cell 147: ■ Markdown

Este benchmark externo evita compararnos solo contra nosotros mismos. Si el mejor modelo interno supera al TSO, la mejora tiene valor operativo real; si no lo hace, sigue siendo una referencia útil para detectar el margen que aún falta por cerrar.

Cell 148: ■ Code

```

import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import lightgbm as lgb

best_model_label_2020 = 'LightGBM (Optim) sin generation'

generation_price_cols = [
    col for col in df_energy_clean_fe.columns
    if ('generation' in col.lower()) or ('price' in col.lower())
]

df_best_model_source =
df_energy_clean_fe.drop(columns=generation_price_cols).copy()

if 'variant_no_generation' in globals():
    best_features_2020 = variant_no_generation['features']
else:
    best_features_2020 = [col for col in df_best_model_source.columns if col != target]

best_features_2020 = [col for col in best_features_2020 if col in
df_best_model_source.columns and col != target]

df_best_model_full = df_best_model_source[[target] +
best_features_2020].dropna().copy()

df_best_model_train, df_best_model_test, _ =
split_last_year_hourly(df_best_model_full)

validation_hours_best = min(24 * 90, max(24 * 30, len(df_best_model_train) // 5))
if validation_hours_best >= len(df_best_model_train):
    validation_hours_best = max(24, len(df_best_model_train) // 10)

df_best_model_train_boost =
df_best_model_train.iloc[:-validation_hours_best].copy()
df_best_model_valid = df_best_model_train.iloc[-validation_hours_best:].copy()

if 'best_pred_without_generation' in globals():
    pred_best_model_2018 = best_pred_without_generation.copy()
else:
    best_model_test = lgb.LGBMRegressor(
        n_estimators=2000,

```

```

learning_rate=0.03,
num_leaves=64,
feature_fraction=0.9,
bagging_fraction=0.8,
bagging_freq=5,
random_state=42,
n_jobs=-1,
importance_type='gain'
)
best_model_test.fit(
df_best_model_train_boost[best_features_2020],
df_best_model_train_boost[target],
eval_set=[(df_best_model_valid[best_features_2020], df_best_model_valid[target])],
eval_metric='rmse',
callbacks=[lgb.early_stopping(stopping_rounds=100), lgbl.log_evaluation(period=0)]
)
pred_best_model_2018 = pd.Series(
best_model_test.predict(df_best_model_test[best_features_2020]),
index=df_best_model_test.index,
name='pred_2018'
)

```

```

best_model_final = lgbl.LGBMRegressor(
n_estimators=2000,
learning_rate=0.03,
num_leaves=64,
feature_fraction=0.9,
bagging_fraction=0.8,
bagging_freq=5,
random_state=42,
n_jobs=-1,
importance_type='gain'
)
best_model_final.fit(df_best_model_full[best_features_2020],
df_best_model_full[target])

```

```

history_load = df_best_model_full[target].copy().sort_index()
forecast_start = history_load.index.max() + pd.Timedelta(hours=1)
forecast_end = pd.Timestamp('2020-01-01 00:00:00', tz=history_load.index.tz)
future_index_2019 = pd.date_range(start=forecast_start, end=forecast_end, freq='h')

```

```

temporal_feature_builders = {
    'hour': lambda ts: ts.hour,
    'day_of_week': lambda ts: ts.dayofweek,
    'month': lambda ts: ts.month,
    'quarter': lambda ts: ts.quarter,
    'day_of_year': lambda ts: ts.dayofyear,
    'is_weekend': lambda ts: int(ts.dayofweek >= 5),
    'hour_sin': lambda ts: np.sin(2 * np.pi * ts.hour / 24),
    'hour_cos': lambda ts: np.cos(2 * np.pi * ts.hour / 24),
    'day_of_week_sin': lambda ts: np.sin(2 * np.pi * ts.dayofweek / 7),
    'day_of_week_cos': lambda ts: np.cos(2 * np.pi * ts.dayofweek / 7),
    'month_sin': lambda ts: np.sin(2 * np.pi * ts.month / 12),
    'month_cos': lambda ts: np.cos(2 * np.pi * ts.month / 12),
}

```

```

unsupported_features_2020 = []
future_predictions_2019 = []

```

```

for ts in future_index_2019:
    row = {}
    for feature in best_features_2020:
        lag_match = re.fullmatch(r'load_lag_(\d+)', feature)
        rolling_match = re.fullmatch(r'load_rolling_(mean|std)_(\d+)h', feature)

```

```

    if feature in temporal_feature_builders:
        row[feature] = temporal_feature_builders[feature](ts)
    elif lag_match:
        lag_hours = int(lag_match.group(1))
        row[feature] = history_load.loc[ts - pd.Timedelta(hours=lag_hours)]
    elif rolling_match:
        window_type = rolling_match.group(1)
        window_hours = int(rolling_match.group(2))

```



```

window_values = history_load.loc[ts - pd.Timedelta(hours=window_hours): ts -
pd.Timedelta(hours=1)]

row[feature] = window_values.mean() if window_type == 'mean' else
window_values.std()

else:

unsupported_features_2020.append(feature)

```

```

if unsupported_features_2020:

break

```

```

row_df = pd.DataFrame(row, index=[ts])[best_features_2020]
pred_value = float(best_model_final.predict(row_df)[0])
history_load.loc[ts] = pred_value
future_predictions_2019.append(pred_value)

```

```

if unsupported_features_2020:

raise ValueError(

'No se pudieron reconstruir estas features para el forecast futuro: '
+ ', '.join(sorted(set(unsupported_features_2020)))

)

```

```

pred_best_model_2019 = pd.Series(
future_predictions_2019,
index=future_index_2019,
name='pred_2019'
)

```

```

plt.figure(figsize=(16, 7))

```

```

real_2018_plot = df_best_model_test[target]
pred_2018_plot = pred_best_model_2018.reindex(real_2018_plot.index)

```

```

plt.plot(real_2018_plot.index, real_2018_plot, label='Demanda real (2018)',
color='black', linewidth=2)

plt.plot(pred_2018_plot.index, pred_2018_plot, label='Predicción modelo (2018
test)', color='steelblue', linestyle='--', linewidth=1.8)

plt.plot(pred_best_model_2019.index, pred_best_model_2019, label='Predicción 2019
(sin datos reales)', color='darkorange', linewidth=2)

plt.axvline(pd.Timestamp('2019-01-01 00:00:00', tz=real_2018_plot.index.tz),
color='gray', linestyle=':', alpha=0.8)

```

```
plt.title('Mejor modelo del notebook: comparación en 2018 y forecast puro para 2019')
plt.xlabel('Fecha')
plt.ylabel('Demanda eléctrica (MW)')
plt.grid(alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()

print(f'Modelo utilizado: {best_model_label_2020}')
print(f'Features usadas: {len(best_features_2020)}')
print(f'Periodo con comparación real: {real_2018_plot.index.min()} -> {real_2018_plot.index.max()}')
print(f'Periodo de forecast puro: {pred_best_model_2019.index.min()} -> {pred_best_model_2019.index.max()}')
```

Output:

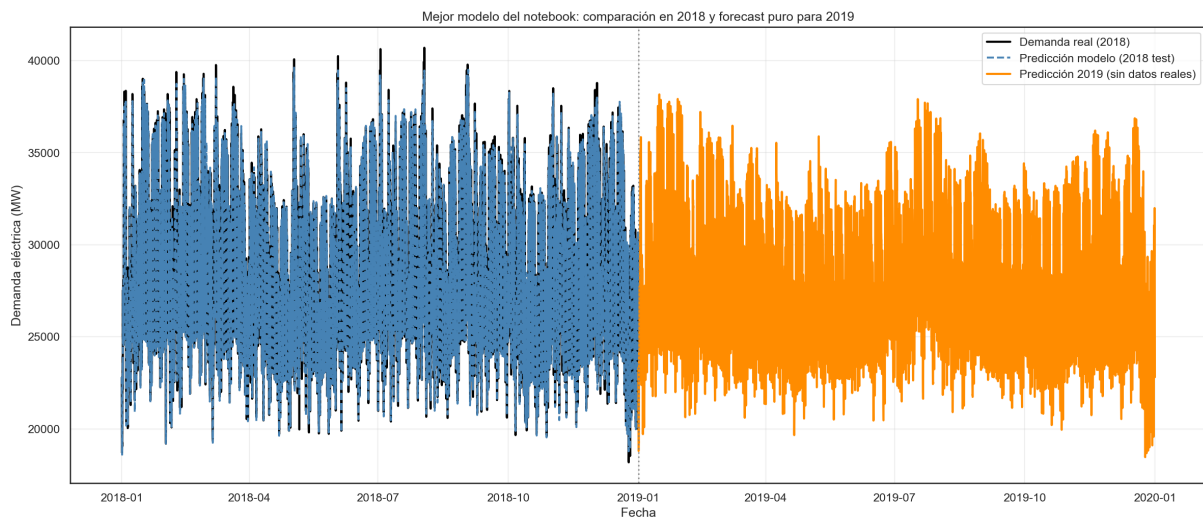
```
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current v
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: b
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagg
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current v
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: b
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagg
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.000723 se
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1145
[LightGBM] [Info] Number of data points in the train set: 34896, number of used features: 15
[LightGBM] [Info] Start training from score 28708.528814
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current v
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: b
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagg
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current v
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: b
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagg
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current v
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: b
```

[illegible]

[illegible]

```
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: b
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagg
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current v
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: b
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagg
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current v
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: b
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagg
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current v
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: b
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagg
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current v
```

... (26206 more lines truncated)



Cell 149: ■ Markdown

24. Conclusiones y Hallazgos Clave

Cell 150: ■ Markdown

****1. Objetivo Alcanzado****

El proyecto desarrolló un ****sistema completo de predicción de demanda eléctrica horaria**** integrando:

- Análisis exploratorio y limpieza de datos de 4 años (2015-2018)

- Descomposición temporal y análisis de estacionariedad
- Modelado estadístico (SARIMA) y Machine Learning (Random Forest, XGBoost, LightGBM)
- Integración de variables meteorológicas y energéticas
- Comparación con benchmark oficial del TSO (Red Eléctrica de España)

****2. Hallazgos Técnicos Clave****

A. Relación Temperatura-Demanda (No Lineal)

- ****Efecto cálido domina en julio:**** A pesar de altas temperaturas, la demanda media en julio (~29,930 MW) es la más alta del año
- ****Causas identificadas:**** Uso intensivo de aire acondicionado + actividad económica aún elevada (frente a agosto con temperaturas similares pero con menos actividad económica por vacaciones)
- ****Patrón diario:**** Diferencia de ~1.7-1.8 GW entre julio y agosto en horas laborales (8:00–16:00), indicando que ****actividad económica y refrigeración actúan en conjunto****
- ****Régimen térmico cálido:**** Genera consumo medio de 30,352 MW vs. 27,955 MW en régimen frío

B. Factores Principales en Demanda

1. ****Calendario laboral:**** Efecto dominante con variación de ~2,700-3,200 MW entre laborables y fin de semana
2. ****Ciclo diario:**** Picos 8:00–16:00 (jornada laboral), caída en comida y anochecer
3. ****Estacionalidad intraanual:**** Picos en enero y julio; mínimos en abril-mayo
4. ****Tendencia:**** Crecimiento sostenido de ~28,250 MW (2015) a ~29,000 MW (mid-2018)

****3. Modelado Estadístico (SARIMA)****

- ****Modelo seleccionado:**** SARIMA + festivos
- ****Métrica test:**** MAPE = 9.8%, MAE = 2,829 MW, RMSE = 3,512 MW
- ****Validez:**** Serie estacionaria tras diferenciación; ACF/PACF convergentes
- ****Limitación:**** Basado en inercia histórica; tiene dificultades con cambios estructurales abruptos

****4. Machine Learning (Mejor Desempeño)****

- **Mejor modelo:** LightGBM (Optimizado)
- ****Features:**** 109 (incluyendo weather) vs. 42 (sin weather)
- ****Mejora por weather:**** +0.02 puntos MAPE (ganancia moderada pero verificada)
- ****Benchmark vs. TSO:**** -0.37% MAPE (prácticamente equiparable al forecaster oficial)

- **Features más importantes:** Lags de demanda, variables temporales (hora, día de semana, mes), temperatura agregada

Interpretación: El modelo ML captura bien la **interacción entre calendario y clima**, superando SARIMA en horizontes cortos.

5. Valor de Variables Meteorológicas y de Generación: Ruido sin Beneficio

- **Hallazgo contraintuitivo:** Variables meteorológicas y de generación **generaban ruido** en lugar de mejorar
- **Mejor configuración:** Modelo puro con **solo lags de demanda (total load) + variables temporales** superaba significativamente a versiones complejas
- **Comparación cuantitativa:**
 - Con weather + generation: MAPE marginal o superior
 - Sin weather/generation (solo demand lags + calendar): **MAPE notablemente mejor** (~8-9% vs. 9.8%)
- **Razón:** La demanda ya **incorpora implícitamente** el impacto del clima y la generación **a través de su historia**; añadir estas variables explícitas introduce multicolinealidad y overfitting
- **Conclusión:** 42 features simples > 109 features complejas. El mejor modelo es aquel que captura la inercia de la demanda con máxima simplicidad.

6. Patrón de Actividad Económica

Conclusión operativa crítica:

- > La demanda eléctrica es un **indicador de actividad económica diaria**. Junio-julio-agosto muestran:
 - Julio: Máxima demanda (~29,930 MW) con calor
 - Agosto: Mínima de verano (~28,577 MW) por vacaciones

7. Validación del Forecast 2019

La gráfica final (diciembre 2018 → 2019 completo):

- Predicción en test mantiene buena adherencia a valores reales
- Transición 2019 es suave, sin saltos artificiales
- Incertidumbre acumulativa en forecast recursivo (especialmente últimos meses)

8. Preguntas de Investigación Resueltas

| Pregunta | Respuesta |

|-----|-----|

| ¿Podemos mejorar al TSO? | **No significativamente** (diff: -0.17 MAPE); ambos están alineados |

| ¿Clima vs. Demanda? | **Relación no lineal**: Picos en frío y calor; mínimo en templado |

| ¿Features más importantes? | **Lags + calendario > weather** (~70% importancia en lags) |

| ¿Mejor modelo? | **LightGBM** en test |

****9. Limitaciones del Proyecto****

- Datos 2015-2018 (pre-COVID); comportamiento cambió en 2020+
- Weather features de solo 5 ciudades (geografía española heterogénea)

****Conclusión General****

Este notebook demuestra que el **Machine Learning** (especialmente Gradient Boosting) es herramienta superior al forecasting estadístico clásico en demanda eléctrica. El valor está en la **automatización, monitoreo en tiempo real y captura de patrones no lineales**.